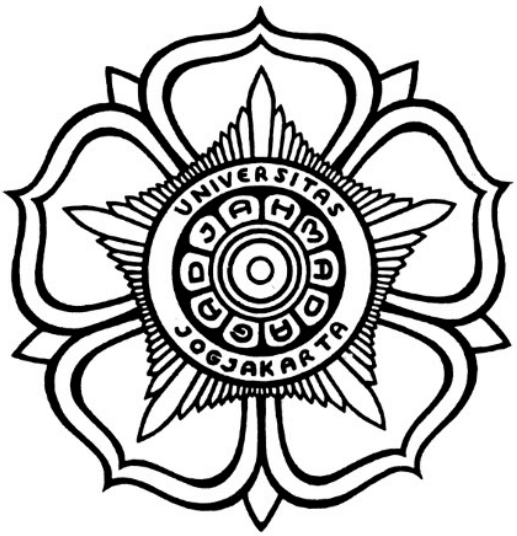
PERANCANGAN SYSTEM ON CHIP (SOC) RISC-V MENGGUNAKAN CORE RI5CY

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Quality Education

Disusun oleh:

Achmad Muhajir
22/500339/TK/54839

PROGRAM SARJANA PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

HALAMAN PENGESAHAN

PERANCANGAN SYSTEM ON CHIP (SOC) RISC-V MENGGUNAKAN CORE RI5CY

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Achmad Muhajir
22/500339/TK/54839

Telah disetujui dan disahkan

Pada tanggal . . . . . .

Dosen Pembimbing I	Dosen Pembimbing II
Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.	**Enas Dhuhri Kusuma, S.T., M.Eng.**
NIP 198001012015041002	**NIP 198211082010121002**

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama	: Achmad Muhajir
NIM	: 22/500339/TK/54839
Tahun terdaftar	: 2022
Program	: Sarjana
Program Studi	: Teknik Elektro
Fakultas	: Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 1 Juli 2026



Achmad Muhajir
22/500339/TK/54839

HALAMAN PERSEMBAHAN



Dengan menyebut nama Allah Yang Maha Pengasih lagi Maha Penyayang,

Dengan penuh rasa syukur kepada Allah Subhanahu Wa Ta'ala atas segala rahmat, nikmat, kemudahan, kekuatan, dan kesempatan yang telah diberikan, sehingga setiap proses dan perjalanan yang telah dilalui dapat terselesaikan atas izin-Nya. Karya sederhana ini saya persembahkan kepada:

Kedua orang tua tercinta, Bapak Hary Suhadi dan Ibu Siti Alvi Khamimi, yang senantiasa menjadi sumber kekuatan, semangat, dan doa dalam setiap langkah kehidupan saya. Terima kasih atas kasih sayang yang tulus, doa yang tidak pernah terputus, pengorbanan yang tidak ternilai, serta dukungan yang selalu diberikan dalam keadaan apa pun. Setiap langkah dan pencapaian yang saya raih tidak pernah terlepas dari perjuangan, doa, dan ridha Bapak dan Ibu.

Abang saya, Muhammad Ainun Najib, adik saya, Alhumairo Nirva Suhadi, serta seluruh keluarga besar yang selalu memberikan dukungan, semangat, dan kebersamaan. Terima kasih telah menjadi bagian penting dalam perjalanan hidup saya, tempat berbagi cerita, serta pengingat untuk terus tumbuh menjadi pribadi yang lebih baik.

Terakhir, untuk diri saya sendiri, Achmad Muhajir. Terima kasih telah memilih untuk terus berusaha, belajar, dan melangkah hingga sampai pada titik ini. Terima kasih atas setiap perjuangan, kesabaran, dan keteguhan dalam menghadapi berbagai proses yang tidak selalu mudah. Semoga perjalanan ini menjadi awal dari langkah-langkah yang lebih baik serta membawa kebermanfaatan di masa mendatang.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, Allah Swt., atas segala limpahan rahmat, karunia, dan nikmat-Nya sehingga penulis senantiasa diberikan kemudahan, kelancaran, kesempatan, serta kekuatan dalam setiap perjalanan hidup hingga mampu menyelesaikan tugas akhir berupa skripsi ini dengan baik. Setiap proses, tantangan, pengalaman, dan pencapaian yang telah dilalui menjadi bagian dari perjalanan berharga yang tidak terlepas dari pertolongan, kehendak, serta kasih sayang-Nya.

Dalam proses penyusunan tugas akhir ini, penulis menyadari bahwa penyelesaian skripsi tidak terlepas dari bantuan, bimbingan, doa, serta dukungan dari berbagai pihak. Oleh karena itu, dengan penuh rasa hormat dan terima kasih, penulis menyampaikan apresiasi kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE. selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Iswandi, S.T., M.Eng. selaku Ketua Program Studi Sarjana Teknik Elektro Departemen Teknik Elektro dan Teknologi Informasi Universitas Gadjah Mada.
3. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM., selaku dosen pembimbing pertama yang telah meluangkan waktu, tenaga, serta ilmu untuk senantiasa memberikan bimbingan, arahan, dan saran kepada penulis selama proses penyusunan tugas akhir ini. Segala masukan yang diberikan sejak tahap perancangan, implementasi dan pengujian, hingga penyelesaian naskah skripsi ini menjadi bagian yang sangat berharga bagi penulis.
4. Bapak Enas Dhuhri Kusuma, S.T., M.Eng., selaku dosen pembimbing kedua yang telah memberikan bimbingan, arahan, masukan, serta evaluasi yang membangun selama proses penelitian. Berbagai ilmu, saran, dan dukungan yang diberikan telah membantu penulis dalam menghadapi berbagai tantangan, menyempurnakan hasil penelitian, serta menyelesaikan penyusunan tugas akhir ini dengan baik.
5. Bapak dan Ibu Dosen Penguji yang telah memberikan waktu, perhatian, saran, serta evaluasi yang sangat berharga dalam proses pengujian tugas akhir ini. Setiap masukan yang diberikan menjadi pembelajaran penting bagi penulis dalam memperbaiki dan menyempurnakan kualitas penelitian serta penulisan skripsi ini.
6. Ibu tercinta, Siti Alvi Khamimi, S.E., yang senantiasa memberikan kasih sayang, doa, perhatian, dukungan, serta semangat yang tiada henti kepada penulis. Terima kasih atas segala ketulusan, kesabaran, pengorbanan, dan kasih sayang yang telah diberikan dalam merawat, mendidik, serta mendampingi penulis sejak kecil hingga mampu menyelesaikan salah satu tahap penting dalam perjalanan hidup ini.
7. Bapak tercinta, Hary Suhadi, S.Si., M.T., yang senantiasa memberikan doa, nasihat, dukungan, serta motivasi kepada penulis dalam setiap perjalanan hidup. Terima kasih atas segala perjuangan, kerja keras, pengorbanan, dan kasih sayang yang telah diberikan dalam mendidik, membimbing, serta membesarkan penulis hingga mampu berada pada tahap ini dan menyelesaikan penyusunan skripsi ini dengan baik.
8. Abang tercinta, Muhammad Ainun Najib, A.Md.Pnl., dan adik tersayang, Alhumairo Nirva Suhadi, yang senantiasa memberikan doa, dukungan, semangat, serta ke-

bersamaan kepada penulis. Terima kasih atas segala perhatian, motivasi, dan cerita yang telah menjadi bagian berharga dalam perjalanan hidup penulis hingga terselesaikannya skripsi ini.

9. Seluruh keluarga besar yang telah memberikan doa, dukungan, perhatian, serta semangat kepada penulis. Terima kasih atas segala bentuk kasih sayang dan dukungan yang diberikan sehingga penulis dapat terus berusaha, berkembang, dan menyelesaikan penyusunan skripsi ini dengan baik.
10. Teman-teman dekat penulis, khususnya Querencia Visesa Asvathavida Vandeveria dan Punten Goput, yang telah menjadi bagian penting dalam perjalanan penulis selama menjalani masa perkuliahan di Yogyakarta. Terima kasih atas segala kebersamaan, cerita, dukungan, bantuan, serta hiburan yang diberikan dalam berbagai keadaan sehingga masa perkuliahan menjadi perjalanan yang lebih berwarna dan penuh kenangan.
11. Teman-teman tim KKN Pesona Natuna 2025 yang telah memberikan pengalaman, kerja sama, kebersamaan, serta kenangan berharga selama menjalani masa pengabdian masyarakat. Berbagai cerita, tantangan, dan pembelajaran yang dilalui bersama menjadi pengalaman yang tidak terlupakan dan memberikan banyak pelajaran berharga bagi penulis.
12. William Gunawan serta rekan-rekan seperjuangan tugas akhir di Faslab IC Design yang telah memberikan bantuan, dukungan, serta menjadi teman diskusi selama proses penyelesaian tugas akhir ini.
13. Seluruh dosen, tenaga kependidikan, serta keluarga besar Departemen Teknik Elektro dan Teknologi Informasi yang telah memberikan ilmu, bantuan, dukungan, serta fasilitas selama penulis menjalani masa perkuliahan. Berbagai ilmu, pengalaman, dan kesempatan yang diberikan menjadi bekal berharga bagi penulis dalam mengembangkan kemampuan akademik maupun pribadi hingga terselesaikannya skripsi ini.
14. Seluruh pihak yang tidak dapat penulis sebutkan satu per satu yang telah memberikan bantuan, dukungan, inspirasi, serta doa selama masa perkuliahan hingga penyusunan skripsi ini. Setiap bentuk kebaikan dan dukungan yang diberikan memiliki arti besar bagi penulis dalam menyelesaikan perjalanan pendidikan ini.

Penulis menyadari bahwa skripsi ini masih memiliki kekurangan dan jauh dari kesempurnaan. Oleh karena itu, penulis dengan terbuka menerima segala bentuk kritik dan saran yang membangun sebagai bahan evaluasi dan perbaikan di masa mendatang.

Akhir kata, penulis berharap semoga skripsi ini dapat memberikan manfaat bagi pembaca serta berkontribusi dalam pengembangan ilmu pengetahuan dan teknologi, khususnya di bidang teknik elektro. Semoga hasil penelitian ini juga dapat menjadi referensi, inspirasi, serta langkah awal bagi penelitian-penelitian selanjutnya. Aamiin.

Yogyakarta, 1 Juli 2026

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN ii
PERNYATAAN BEBAS PLAGIASI iii
HALAMAN PERSEMBAHAN iv
KATA PENGANTAR vi
DAFTAR ISI vii
DAFTAR TABEL x
DAFTAR GAMBAR xi
DAFTAR SINGKATAN xiv
INTISARI xv
ABSTRACT xvi
BAB I Pendahuluan 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Penelitian 2
1.4 Batasan Penelitian 2
1.5 Manfaat Penelitian 3
1.6 Sistematika Penulisan 3
BAB II Tinjauan Pustaka 5
2.1 Tinjauan Pustaka 5
2.2 Landasan Teori 6
2.2.1 Arsitektur Set Instruksi (*Instruction Set Architecture* atau ISA) RISC-V 6
2.2.2 *System on Chip* (SoC) 8
2.2.3 Platform PULP dan PULPino 9
2.2.4 Peta Memori (*Memory Map*) PULPino 10
2.2.4.1 Organisasi Ruang Alamat 10
2.2.4.2 Memory-Mapped I/O 11
2.2.5 Arsitektur *Core* RI5CY 12
2.2.5.1 Pipeline dan Modul Utama Arsitektur RI5CY 13
2.2.5.2 Arsitektur Harvard 14
2.2.5.3 Mekanisme Penanganan *Data Hazard* 15
2.2.5.4 Antarmuka Memori Eksternal 15
2.2.5.5 Logika *Handshaking* dan Kendali Antar-Tahapan (*Inter-Stage*) 15
2.2.6 Protokol Komunikasi Register Memori dan Arsitektur Bus Interkoneksi 15

2.2.7 Antarmuka Periferal 16
2.2.8 FPGA Zybo *development board* dan SoC Zynq-7000 17
2.2.8.1 Mekanisme Booting pada SoC Zynq-7000 19
2.2.9 Tahapan Sintesis dan Implementasi FPGA 19
2.2.10 Penggunaan Sumber Daya FPGA 20
2.2.11 Analisis *Timing* FPGA 21
2.2.12 Ekosistem Perangkat Lunak Pendukung 22
BAB III Metode Penelitian 23
3.1 Alat dan Bahan Tugas Akhir 23
3.1.1 Alat Tugas Akhir 23
3.1.2 Bahan Tugas Akhir 25
3.2 Metode yang Digunakan 26
3.3 Alur Penelitian 27
3.4 Rancangan Hierarki Perangkat Keras 29
3.4.1 Struktur Direktori dan Komposisi Proyek PULPino 32
3.5 Tahapan Modifikasi Repositori 35
3.5.1 Modifikasi Perangkat Keras FPGA 35
3.5.1.1 Penyesuaian Manajemen IP 35
3.5.1.2 Penyesuaian Skrip Otomasi FPGA 36
3.5.1.3 Pembaruan Modul IP FPGA 36
3.5.1.4 Penyesuaian Proyek PULPino 37
3.5.1.5 Penyesuaian Proyek Emulasi Zybo 38
3.5.2 Modifikasi Lingkungan Perangkat Lunak Linux (`fpga/sw/`) 39
3.5.2.1 Penyesuaian Sistem Kompilasi 39
3.5.2.2 Penyesuaian Device Tree 41
3.5.2.3 Penyesuaian Konfigurasi U-Boot 41
3.5.2.4 Penyesuaian Buildroot 41
3.5.2.5 Penyesuaian Program Spiload 42
3.5.3 Modifikasi Perangkat Lunak Bare-metal (`sw/`) 43
3.5.3.1 Penyesuaian Skrip Konfigurasi 43
3.5.3.2 Penyesuaian Konfigurasi `CMakeLists.txt` 44
3.5.3.3 Penyesuaian Berkas Inisialisasi dan *Linker* 45
3.6 Tahapan Sintesis, Implementasi, dan *Deployment* 46
3.6.1 Sintesis dan Implementasi Perangkat Keras 47
3.6.2 Pembangunan Sistem Operasi pada *Processing System* 48
3.6.3 Kompilasi Perangkat Lunak RISC-V 48
3.6.4 Pembuatan Berkas Boot 49
3.6.5 Persiapan Media *Boot* dan *Deployment* 50
3.6.6 Proses *Booting* dan Eksekusi Program 50

3.7 Skenario dan Rancangan Pengujian 51
3.8 Masalah dan Keterbatasan Penelitian 53
3.9 Penggunaan Kecerdasan Buatan dalam Penulisan Naskah 53
BAB IV Hasil dan Pembahasan 55
4.1 Keberhasilan Sintesis dan *Deployment* 55
4.2 Evaluasi Laporan Hasil Sintesis 55
4.2.1 Analisis Penggunaan Sumber Daya (*Resource Utilization*) 56
4.2.2 Analisis *Timing* dan Frekuensi Kerja Maksimum 58
4.3 Hasil Validasi Pengujian Operasional 60
4.3.1 Validasi Operasional Sistem 60
4.3.2 Pengujian Operasional Periferal Masukan/Keluaran (GPIO) 61
4.3.2.1 Pengujian Saklar (*Switch*) Fisik dan Lampu LED 61
4.3.2.2 Pengujian Interaksi Tombol (*Push Button*) 63
4.3.2.3 Pengujian Port Modul Periferal (*Peripheral Module* atau PMOD) 63
4.4 Pengujian Integrasi Periferal Tambahan Sensor PIR dan Antarmuka Layar I2C OLED 64
4.4.1 Perbandingan Hasil Implementasi dengan Penelitian Terdahulu ... 66
BAB V Kesimpulan dan Saran 68
5.1 Kesimpulan 68
5.2 Saran 69
DAFTAR PUSTAKA 70
LAMPIRAN L-72
L.1 Repositori Modifikasi L-72
L.2 Kode Program L-73
L.2.1 `helloworld.c` L-73
L.2.2 `pre_opt.tcl` L-84
L.2.3 `pre_place.tcl` L-90

DAFTAR TABEL

Tabel 2.1 Peta Memori (*Memory Map*) Default PULPino 11

Tabel 3.1 Perangkat Keras pada Penelitian 24
Tabel 3.2 Perangkat Lunak pada Penelitian 25
Tabel 3.3 Bahan Penelitian 26
Tabel 3.4 Skenario dan rancangan pengujian perangkat keras 52

Tabel 4.1 Utilisasi sumber daya berdasarkan hierarki modul SoC PULPino 56
Tabel 4.2 Ringkasan hasil analisis *timing* pada domain *clock* `clk` 58
Tabel 4.3 Interpretasi keluaran utilitas `spiload` selama validasi operasional sistem 61
Tabel 4.4 Tabel Skenario Pengujian Aplikasi Periferal Tambahan 64
Tabel 4.5 Perbandingan Kinerja Sintesis SoC Berbasis Zero-Riscy dan RI5CY. 66

DAFTAR GAMBAR

Gambar 2.1 Format Dasar Instruksi RISC-V 7
Gambar 2.2 Logo PULPino 9
Gambar 2.3 Diagram Sistem Keseluruhan PULPino (*Overview*) 10
Gambar 2.4 Diagram *Core* RI5CY pada Arsitektur PULPino 12
Gambar 2.5 Diagram *Pipeline Core* RI5CY 14
Gambar 2.6 Zybo *Development Board* Zynq-7010 17
Gambar 2.7 Arsitektur Internal SoC Xilinx Zynq-7000 18
Gambar 3.8 Diagram alur penelitian 27
Gambar 3.9 Diagram Blok Hierarki Top-Level Arsitektur Zynq PS dan PULPino PL 29
Gambar 3.10 Modul tingkat teratas `pulpemu_top` 30
Gambar 3.11 Blok Diagram Verilog PULPino 31
Gambar 3.12 Penyesuaian versi IP core pada `ips_list.yml` 35
Gambar 3.13 Penambahan blok *try-except* untuk *bypass* perintah `chmod` pada `IPDatabase.py` 36
Gambar 3.14 Modifikasi `fpga/Makefile` untuk menonaktifkan proses *clean* dan kompilasi perangkat lunak bawaan 36
Gambar 3.15 Pemindahan parameter target penyalinan fail *output* ke direktori `.gen` 37
Gambar 3.16 Injeksi perbaikan rilis versi IP *Floating Point* ke rilis terbaru 7.1 . 37
Gambar 3.17 Penyesuaian sintaks operasi dasar folder agar kompatibel pada lingkungan *host* Windows 38
Gambar 3.18 Penambahan aturan *checkpoint* dan penyimpanan kendala otomatis pasca integrasi skema EDN 38
Gambar 3.19 Penyisipan instruksi pra-optimasi dan pra-penempatan perangkat 39
Gambar 3.20 Klausa pelemahan penolakan ketidakcocokan versi rilis blok fungsional *Processing System* 39
Gambar 3.21 Perbaikan orientasi repositori pengunduhan Linux dan penyisipan perintah `sed` 40
Gambar 3.22 Penambahan tanda petik pada deklarasi variabel `PATH` 40
Gambar 3.23 Integrasi *patch* pada `fakeroot` 40
Gambar 3.24 Penyisipan berkas `pl.dtsi` 40
Gambar 3.25 Penambahan alias `serial1` 41
Gambar 3.26 Penonaktifan fitur verifikasi citra pada U-Boot 41
Gambar 3.27 Deklarasi minimalisasi spesifikasi perangkat 42
Gambar 3.28 Penambahan flag `-DZYBO` pada program `spiload` 42
Gambar 3.29 Perubahan alamat awal eksekusi pada program `spiload` 43
Gambar 3.30 Koreksi rujukan direktori target aplikasi *toolchain* pada skrip *configure* 43
Gambar 3.31 Injeksi spesifikasi target kompilasi mesin pada skrip *configure* ... 44
Gambar 3.32 Pembenahan argumen perintah pemanggilan utilitas CMake 44
Gambar 3.33 Deklarasi target sistem `Generic` untuk lingkungan *bare-metal* . 45
Gambar 3.34 Penonaktifan *implicit link libraries* 45
Gambar 3.35 Penyesuaian compiler dan linker flags 45

Gambar 3.36 Penonaktifan pemanggilan inisialisasi awal `libc` pada blok *assembly* 46
Gambar 3.37 Penyingkiran parameter pustaka OS pendukung dari skrip *linker* RI5CY 46
Gambar 3.38 Diagram alir tahapan sintesis, implementasi, dan *deployment* 46
Gambar 3.39 Alur sintesis dan implementasi perangkat keras 47
Gambar 3.40 Alur pembangunan sistem operasi pada *Processing System* 48
Gambar 3.41 Alur kompilasi perangkat lunak *bare-metal* RISC-V 48
Gambar 3.42 Alur pembuatan berkas *boot* 49
Gambar 3.43 Alur persiapan media *boot* dan *deployment* 50
Gambar 4.44 Hasil validasi proses *bootloading* dan eksekusi program pada SoC PULPino 60
Gambar 4.45 Dokumentasi Pengujian Fisik Saklar-LED pada Board Zybo 62
Gambar 4.46 Tangkapan Layar Log Interaksi Umpan Balik *Switch* 62
Gambar 4.47 Tangkapan Layar Log Respon *Push Button* pada UART 63
Gambar 4.48 Tangkapan Layar Deteksi Log Perubahan Transisi Level Tegangan PMOD 63
Gambar 4.49 Tangkapan Layar Deteksi Log Sensor PIR 64
Gambar 4.50 Dokumentasi Pengujian Periferal Lengkap pada Board Zybo 65

DAFTAR SINGKATAN

DAFTAR LAMBANG DAN SINGKATAN

Lambang

F_{max}	=	frekuensi kerja maksimum (MHz)
T_{clock}	=	periode *clock* desain (ns)

Singkatan

APB	=	*Advanced Peripheral Bus*
ASIC	=	*Application Specific Integrated Circuit*
AXI	=	*Advanced eXtensible Interface*
BRAM	=	*Block Random Access Memory*
CLB	=	*Configurable Logic Block*
CRLF	=	*Carriage Return Line Feed*
DDR	=	*Double Data Rate*
DSP	=	*Digital Signal Processor*
ELF	=	*Executable and Linkable Format*
EX	=	*Execute* (tahap *pipeline*)
EXT4	=	*Fourth Extended Filesystem*
FAT32	=	*File Allocation Table 32*
FF	=	*Flip-Flop*
FPGA	=	*Field Programmable Gate Array*
FSBL	=	*First Stage Bootloader*
GCC	=	*GNU Compiler Collection*
GPIO	=	*General Purpose Input/Output*
GUI	=	*Graphical User Interface*
HDL	=	*Hardware Description Language*
I2C	=	*Inter-Integrated Circuit*
ID	=	*Instruction Decode* (tahap *pipeline*)
IDE	=	*Integrated Development Environment*
IF	=	*Instruction Fetch* (tahap *pipeline*)
IoT	=	*Internet of Things*
IP	=	*Intellectual Property*
ISA	=	*Instruction Set Architecture*
JTAG	=	*Joint Test Action Group*
LED	=	*Light Emitting Diode*
LF	=	*Line Feed*

LUT	=	*Look-Up Table*
MAC	=	*Multiply-Accumulate*
MMIO	=	*Memory-Mapped Input/Output*
OCM	=	*On-Chip Memory*
OLED	=	*Organic Light-Emitting Diode*
OS	=	*Operating System*
PIR	=	*Passive Infrared*
PL	=	*Programmable Logic*
PMOD	=	*Peripheral Module*
POR	=	*Power-On Reset*
POSIX	=	*Portable Operating System Interface*
PS	=	*Processing System*
PULP	=	*Parallel Ultra-Low-Power*
QSPI	=	*Quad Serial Peripheral Interface*
RISC	=	*Reduced Instruction Set Computer*
ROM	=	*Read-Only Memory*
RTL	=	*Register Transfer Level*
RV32C	=	*RISC-V 32-bit Compressed*
RV32I	=	*RISC-V 32-bit Integer*
RV32M	=	*RISC-V 32-bit Multiply*
SoC	=	*System on Chip*
SPI	=	*Serial Peripheral Interface*
SSBL	=	*Second Stage Boot Loader*
STA	=	*Static Timing Analysis*
THS	=	*Total Hold Slack*
TNS	=	*Total Negative Slack*
TPWS	=	*Total Pulse Width Slack*
UART	=	*Universal Asynchronous Receiver-Transmitter*
WB	=	*Write Back* (tahap *pipeline*)
WHS	=	*Worst Hold Slack*
WNS	=	*Worst Negative Slack*
WPWS	=	*Worst Pulse Width Slack*
WSL	=	*Windows Subsystem for Linux*
XSA	=	*Xilinx Support Archive*
Zicsr	=	*Control and Status Register* (Ekstensi RISC-V)

INTISARI

PULPino merupakan platform *System-on-Chip* (SoC) berbasis arsitektur RISC-V yang dikembangkan sebagai bagian dari proyek *Parallel Ultra Low Power* (PULP). Namun, repositori PULPino dikembangkan menggunakan perangkat lunak dan alur pengembangan yang telah usang sehingga tidak lagi kompatibel dengan versi terbaru perangkat lunak AMD/Xilinx. Kondisi tersebut menyulitkan proses sintesis, implementasi, dan reproduksi penelitian pada platform FPGA modern. Penelitian ini bertujuan melakukan adaptasi, sintesis, implementasi, serta validasi operasional SoC PULPino berbasis *core* RI5CY pada FPGA Digilent Zybo Z-7010 menggunakan Xilinx Vivado 2025.1 dan Vitis Unified IDE 2025.1. Proses adaptasi meliputi penyesuaian repositori, skrip sintesis, konfigurasi sistem, kompilasi program RISC-V, penyusunan *boot image*, serta implementasi pada perangkat keras. Evaluasi dilakukan melalui analisis penggunaan sumber daya FPGA, analisis *timing*, dan pengujian fungsional terhadap antarmuka UART, GPIO, PMOD, sensor PIR, serta layar OLED berbasis I^2C. Hasil implementasi menunjukkan bahwa rancangan berhasil diimplementasikan menggunakan 13.400 LUT (76,14%), 9.924 *flip-flop* (28,19%), 16 BRAM (26,67%), dan 11 DSP (13,75%). Analisis *timing* menunjukkan seluruh kendala waktu terpenuhi dengan nilai *Worst Negative Slack* (WNS) sebesar 26,587 ns pada frekuensi target 20 MHz, yang memberikan estimasi frekuensi kerja maksimum sebesar 42,71 MHz. Seluruh pengujian operasional berhasil dijalankan sesuai rancangan, menunjukkan bahwa SoC PULPino dapat diadaptasi dan diimplementasikan secara andal pada lingkungan pengembangan FPGA modern. Hasil penelitian ini diharapkan dapat menjadi referensi yang mudah direproduksi bagi pengembangan dan penelitian lanjutan berbasis RISC-V pada platform FPGA.

Kata kunci : FPGA, RISC-V, SoC, Implementasi Perangkat Keras

ABSTRACT

PULPino is a System-on-Chip (SoC) platform based on the RISC-V architecture, developed as part of the Parallel Ultra Low Power (PULP) project. However, the original PULPino repository was built using deprecated tools and development workflows, rendering it incompatible with the latest versions of AMD/Xilinx software. This incompatibility severely hinders the synthesis, implementation, and reproduction of research on modern FPGA platforms. This study aims to adapt, synthesize, implement, and operationally validate the RI5CY-core-based PULPino SoC on the Digilent Zybo Z-7010 FPGA using Xilinx Vivado 2025.1 and Vitis Unified IDE 2025.1. The adaptation process comprises repository refactoring, synthesis script adjustments, system configuration tuning, RISC-V program compilation, boot image generation, and hardware-level implementation. Evaluation was conducted through FPGA resource utilization profiling, static timing analysis, and functional verification of the UART, GPIO, and PMOD interfaces, alongside a PIR sensor and an I2C-based OLED display. The implementation results demonstrate that the design was successfully realized using 13,400 LUTs (76.14%), 9,924 flip-flops (28.19%), 16 BRAMs (26.67%), and 11 DSP blocks (13.75%). Static timing analysis confirms that all timing constraints were met, yielding a Worst Negative Slack (WNS) of 26.587 ns at a target frequency of 20 MHz, which translates to an estimated maximum operating frequency of 42.71 MHz. All operational tests executed successfully as designed, indicating that the PULPino SoC can be reliably adapted and deployed within modern FPGA development environments. The outcomes of this research are expected to serve as a highly reproducible reference for future development and advanced research based on RISC-V on FPGA platforms.

Keywords : FPGA, RISC-V, SoC, *Hardware Implementation*.

BAB I
PENDAHULUAN

1.1 Latar Belakang

Seiring dengan berkembangnya teknologi komputasi dan *Internet of Things* (IoT), kebutuhan akan prosesor yang hemat daya, efisien, dan fleksibel semakin meningkat. Dalam dekade terakhir, *Instruction Set Architecture* (ISA) yang *open-source* mendapatkan perhatian besar dari kalangan akademisi maupun industri. RISC-V hadir sebagai salah satu ISA *open-source* yang menawarkan fleksibilitas tinggi tanpa batasan lisensi komersial. Keterbukaan ini memungkinkan peneliti dan pengembang untuk mengeksplorasi, memodifikasi, dan mengimplementasikan arsitektur prosesor sesuai dengan kebutuhan spesifik aplikasi.

Salah satu implementasi platform mikrokontroler berbasis arsitektur set instruksi RISC-V yang dirancang untuk penggunaan daya rendah adalah platform PULP (*Parallel Ultra Low Power*). Platform PULP merupakan proyek kolaboratif oleh grup peneliti ETH Zurich dan University of Bologna, dengan salah satu produk turunannya yaitu *System on Chip* (SoC) PULPino. SoC PULPino menggunakan *core* RI5CY sebagai unit pemrosesan utamanya [1]. *Core* RI5CY merupakan *core* RISC-V 32-bit dengan (*4-stage pipeline*) yang dirancang khusus untuk aplikasi komputasi berdaya sangat rendah (*ultra-low power*) [2]. PULPino sendiri mewadahi integrasi *core* RI5CY ini ke FPGA (*Field Programmable Gate Array*) dengan menyatukannya bersama subsistem periferal dan memori. Implementasi fisik perangkat kerasnya diperlukan untuk memvalidasi performa serta kinerja operasional periferal pada *board* dalam kondisi nyata.

Field Programmable Gate Array (FPGA) merupakan platform yang sangat tepat untuk tujuan purwarupa (*prototyping*) dan implementasi perangkat keras. FPGA menawarkan kemampuan untuk dikonfigurasi ulang, sehingga proses modifikasi dan pengujian arsitektur prosesor dapat dilakukan dengan siklus pengembangan yang lebih singkat dibandingkan dengan pembuatan *Application Specific Integrated Circuit* (ASIC).

Oleh karena itu, penelitian ini akan berfokus pada implementasi arsitektur *core* RI5CY pada SoC PULPino ke dalam *board* FPGA (khususnya Zybo development board Z-7010) untuk mengevaluasi proses modifikasi dan memvalidasi kebenaran operasionalnya melalui pengujian langsung periferal pada FPGA yang telah diimplementasikan PULPino berbasis *core* RI5CY dengan menggunakan *toolchain* dan perangkat lunak, yaitu Xilinx Vivado dan Vitis versi terbaru.

Oleh karena itu, penelitian ini berfokus pada modifikasi, sintesis, implementasi, dan validasi operasional SoC PULPino berbasis *core* RI5CY pada FPGA Digilent Zybo

development board Z-7010 menggunakan perangkat lunak Xilinx Vivado dan Vitis versi terbaru. Selain mengevaluasi penggunaan sumber daya FPGA dan karakteristik *timing*, penelitian ini juga memvalidasi operasional periferal melalui pengujian langsung pada perangkat keras.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana tahapan modifikasi dan sintesis SoC PULPino berbasis *core* RISC-V RI5CY agar dapat diimplementasikan ke dalam FPGA Zybo *development board* (xc7z010clg400-1) menggunakan perangkat lunak Xilinx Vivado versi terbaru?
2. Bagaimana profil penggunaan sumber daya (*resource utilization*) serta karakteristik *timing* dari implementasi SoC PULPino berbasis *core* RI5CY pada FPGA Zybo?
3. Bagaimana memvalidasi operasional SoC PULPino pada FPGA Zybo melalui pengujian komunikasi UART, antarmuka GPIO, konektor PMOD, sensor PIR, serta modul OLED berbasis I2C?

1.3 Tujuan Penelitian

Adapun tujuan yang ingin dicapai melalui penelitian ini adalah:

1. Melakukan modifikasi, sintesis, dan implementasi SoC PULPino berbasis *core* RISC-V RI5CY pada FPGA Zybo *development board* (xc7z010clg400-1) menggunakan konfigurasi dan *toolchain* terbaru.
2. Menganalisis penggunaan sumber daya (*resource utilization*) serta karakteristik *timing* hasil implementasi SoC PULPino pada FPGA Zybo.
3. Memvalidasi operasional SoC PULPino melalui pengujian komunikasi UART, GPIO, konektor PMOD, sensor PIR, dan modul OLED berbasis I2C pada FPGA Zybo Z-7010.

1.4 Batasan Penelitian

Agar penelitian ini lebih terfokus dan terarah, ditetapkan beberapa batasan masalah sebagai berikut:

1. Implementasi dan pengujian perangkat keras secara eksklusif menggunakan *board* FPGA Zybo *development board* Z-7010.
2. Perangkat lunak yang digunakan untuk tahap sintesis dan implementasi adalah Xilinx Vivado, sedangkan kompilasi program uji menggunakan Xilinx Vitis dan *toolchain* GCC RISC-V.

3. Penelitian berfokus pada implementasi platform SoC PULPino dengan penyesuaian operasional pada aplikasi perangkat lunak terbaru, tanpa mengubah secara ekstrem arsitektur internal dari *core* RI5CY atau instruksi ISA bawaannya.
4. Analisis yang dilakukan terbatas pada hasil penggunaan sumber daya (*resource utilization*) dan analisis *timing* yang dihasilkan dari implementasi Vivado.
5. Pengujian operasional dibatasi pada komunikasi UART, GPIO, sensor PIR, dan modul OLED berbasis I2C.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1. **Manfaat Akademis**: Menambah referensi dan literatur terkait perancangan sistem digital, khususnya dalam hal implementasi dan replikasi prosesor *open-source* berbasis RISC-V pada platform FPGA menggunakan piranti modern.
2. **Manfaat Praktis**: Menjadi panduan praktis bagi pengembang atau mahasiswa lain yang ingin membangun sistem tertanam menggunakan platform SoC PULPino dengan *core* RISC-V RI5CY pada perangkat FPGA bersumber daya terbatas seperti seri Zynq-7000.

1.6 Sistematika Penulisan

Untuk mempermudah pemahaman mengenai isi laporan skripsi, maka disusunlah sistematika penulisan sebagai berikut:

Bab I Pendahuluan
Bab ini membahas tentang latar belakang masalah, rumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan.

Bab II Tinjauan Pustaka
Bab ini berisi mengenai tinjauan pustaka dari penelitian terdahulu serta landasan teori yang mendukung penelitian, meliputi arsitektur RISC-V, platform PULP dan PULPino, struktur internal *core* RI5CY, konsep dasar FPGA Zynq-7000, antarmuka periferal, serta aliran desain sintesis menggunakan Xilinx Vivado dan Vitis.

Bab III Metode Penelitian
Bab ini menguraikan alat dan bahan yang digunakan, metode modifikasi yang dilakukan selama proses sintesis dan implementasi, alur tahapan tugas akhir dari inisialisasi repositori hingga *deployment* fisik, serta skenario pengujian dan pengambilan data.

Bab IV Hasil dan Pembahasan
Bab ini menyajikan hasil sintesis dan implementasi platform SoC PULPino, analisis penggunaan sumber daya FPGA, analisis *timing* dan estimasi frekuensi kerja maksimum,

validasi operasional sistem melalui pengujian UART, GPIO, sensor PIR, serta modul OLED berbasis I2C, kemudian membandingkan hasil implementasi dengan penelitian terdahulu.

Bab V Kesimpulan dan Saran

Bab ini berisi kesimpulan yang diperoleh dari serangkaian proses sintesis, implementasi, dan pengujian serta memberikan saran-saran untuk pengembangan penelitian selanjutnya.

BAB II
TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Penelitian mengenai implementasi arsitektur RISC-V pada *Field Programmable Gate Array* (FPGA) telah berkembang pesat dalam beberapa tahun terakhir. Sifat RISC-V yang bersifat *open-source* memungkinkan pengembangan *System on Chip* (SoC) tanpa ketergantungan pada lisensi komersial sehingga banyak dimanfaatkan sebagai platform penelitian maupun pengembangan sistem *embedded*. Berbagai penelitian terdahulu menunjukkan bahwa implementasi RISC-V pada FPGA mampu memberikan keseimbangan antara performa, fleksibilitas, dan efisiensi penggunaan sumber daya.

Höller *et al.* [3] melakukan evaluasi terhadap beberapa *core* RISC-V sumber terbuka, yaitu Freedom, RI5CY, dan PicoRV32, yang diimplementasikan pada FPGA Xilinx Artix-7. Penelitian tersebut membandingkan karakteristik penggunaan sumber daya, frekuensi operasi maksimum, dan performa prosesor. Hasil sintesis menunjukkan bahwa RI5CY mampu mencapai frekuensi operasi hingga 50 MHz dengan kinerja sebesar 1,10 DMIPS/MHz serta penggunaan sumber daya yang relatif moderat dibandingkan *core* lainnya. Hasil tersebut menunjukkan bahwa RI5CY merupakan salah satu alternatif *softcore* RISC-V yang memiliki keseimbangan antara performa komputasi dan kompleksitas implementasi.

Heinz *et al.* [4] mengevaluasi berbagai implementasi *softcore* RISC-V pada beberapa keluarga FPGA Xilinx, termasuk Zynq-7000. Penelitian tersebut menunjukkan bahwa karakteristik performa setiap *core* sangat dipengaruhi oleh spesifikasi perangkat FPGA yang digunakan. Perbedaan kapasitas logika, memori, dan blok DSP menyebabkan nilai frekuensi maksimum serta utilisasi sumber daya yang diperoleh setiap implementasi menjadi berbeda meskipun menggunakan arsitektur prosesor yang sama. Hasil tersebut menunjukkan bahwa platform Zynq-7000 merupakan lingkungan yang relevan untuk mengevaluasi implementasi berbagai *core* RISC-V.

Verma *et al.* [5] memodifikasi *core* RI5CY dengan mengganti unit perkalian menggunakan *approximate multiplier* untuk aplikasi yang bersifat *error-tolerant*. Implementasi pada FPGA Xilinx Zedboard berbasis Zynq-7000 menunjukkan adanya penurunan konsumsi daya dan peningkatan efisiensi *power-delay product* dibandingkan implementasi RI5CY standar. Penelitian tersebut memperlihatkan bahwa struktur mikroarsitektur RI5CY relatif mudah dimodifikasi untuk memenuhi kebutuhan aplikasi tertentu tanpa mengubah keseluruhan arsitektur SoC.

Kırali dan Fidan [6] membandingkan implementasi beberapa *core* RISC-V, terma-

suk RI5CY dan CV32E40P, pada FPGA Zybo Z7-20. Hasil penelitian menunjukkan bahwa RI5CY memerlukan penggunaan sumber daya FPGA yang lebih besar dibandingkan beberapa *core* lainnya, terutama pada penggunaan LUT, sebagai konsekuensi dari kompleksitas mikroarsitektur dan dukungan fitur yang lebih lengkap. Oleh karena itu, RI5CY banyak digunakan sebagai acuan dalam penelitian yang mengevaluasi keseimbangan antara performa dan penggunaan sumber daya pada implementasi RISC-V di FPGA.

Selain penelitian yang berfokus pada inti prosesor, pengembangan SoC PULPino oleh ETH Zurich dan University of Bologna menyediakan platform RISC-V berbasis RI5CY yang mengintegrasikan memori, interkoneksi bus, serta berbagai periferal dalam satu sistem [7]. PULPino banyak dimanfaatkan sebagai platform penelitian karena menyediakan lingkungan perangkat keras dan perangkat lunak yang lengkap untuk pengembangan sistem *embedded*. Namun, repositori PULPino dikembangkan menggunakan ekosistem perangkat lunak yang relatif lama sehingga proses implementasinya pada lingkungan pengembangan terbaru, seperti Vivado 2025.1, Vitis 2025.1, serta FPGA Zybo Z-7010, memerlukan serangkaian penyesuaian pada sistem sintesis, proses *build*, maupun konfigurasi perangkat lunak pendukung.

Berdasarkan tinjauan pustaka tersebut, sebagian besar penelitian terdahulu berfokus pada evaluasi performa arsitektur RI5CY maupun modifikasi mikroarsitektur prosesor. Sementara itu, pembahasan mengenai proses adaptasi dan implementasi penuh SoC PULPino pada lingkungan pengembangan terbaru masih relatif terbatas. Oleh karena itu, penelitian ini berfokus pada proses adaptasi repositori PULPino agar dapat diimplementasikan pada FPGA Digilent Zybo Z-7010 menggunakan Vivado 2025.1 dan Vitis 2025.1, kemudian mengevaluasi hasil sintesis, implementasi, serta validasi operasional sistem yang dihasilkan.

2.2 Landasan Teori

2.2.1 Arsitektur Set Instruksi (*Instruction Set Architecture* atau ISA) RISC-V

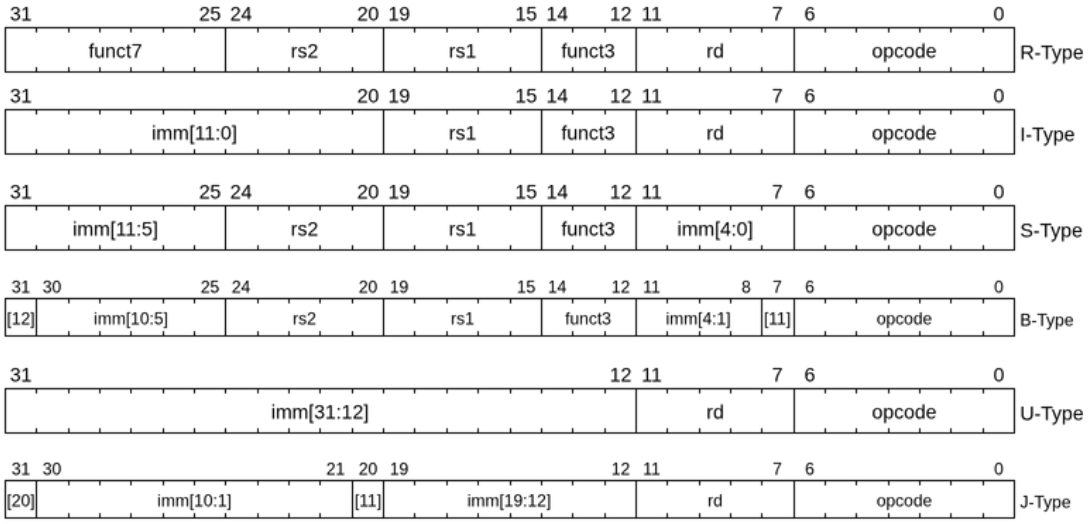
RISC-V adalah arsitektur set instruksi berbasis *Reduced Instruction Set Computer* (RISC) yang dikembangkan di University of California, Berkeley [8]. Filosofi utama dari arsitektur RISC berfokus pada kesederhanaan desain perangkat keras (seperti penggunaan arsitektur *load/store* terpisah dan jumlah format instruksi yang minim) guna mencapai siklus eksekusi yang konsisten dan frekuensi *clock* yang tinggi.

Tidak seperti sebagian besar arsitektur prosesor komersial yang bersifat proprietari, RISC-V menggunakan spesifikasi *open-source*. Hal ini memberikan kebebasan bagi peneliti, akademisi, maupun industri untuk merancang, memodifikasi, dan memproduksi prosesor berbasis RISC-V tanpa membayar royalti atau terikat lisensi tertutup.

Selain itu, RISC-V dirancang dengan pendekatan yang modular. Berdasarkan

spesifikasi ISA resmi [**?**], RISC-V tidak menggunakan satu set instruksi yang besar, melainkan terdiri atas satu set instruksi dasar (*integer base*) yang dapat dipadukan dengan berbagai ekstensi opsional, seperti `M`, `A`, `F`, `D`, `C`, dan ekstensi lainnya sesuai kebutuhan aplikasi.

Untuk arsitektur 32-bit, spesifikasi dasar tersebut dikenal sebagai RV32I. ISA ini menyediakan 40 instruksi dasar dan 32 register, yang sudah memadai untuk membangun sebuah core serta menjalankan perangkat lunak standar. Seluruh instruksi pada RV32I dikodekan dalam enam format dasar dengan panjang tetap 32-bit, seperti ditunjukkan pada Gambar 2.1.



Gambar 2.1. Format Dasar Instruksi RISC-V

- Format R (*Register*): Digunakan untuk operasi aritmatika dan logika yang melibatkan tiga register (dua register sumber, `rs1` dan `rs2`, serta satu register tujuan, `rd`).
- Format I (*Immediate*): Digunakan untuk instruksi yang melibatkan satu register sumber, satu register tujuan, dan satu nilai konstanta numerik (*immediate*) 12-bit, misalnya pada operasi *load* memori dan aritmatika *immediate*.
- Format S (*Store*): Dirancang khusus untuk memindahkan nilai register ke dalam memori. Format ini mengandalkan dua register sumber (data yang akan disimpan dan alamat referensi) serta nilai *immediate* untuk pergeseran alamat (*offset*).
- Format B (*Branch*): Sangat serupa dengan format S, namun varian ini difokuskan pada operasi lompatan bersyarat (*conditional jump*). Format penyandian ini merepresentasikan alamat tujuan ke dalam bentuk kelipatan dua bit.
- Format U (*Upper Immediate*): Berfungsi untuk memuat nilai *immediate* besar selebar 20-bit langsung ke posisi 20 bit paling signifikan dari register tujuan.
- Format J (*Jump*): Didedikasikan khusus untuk operasi lompatan tak bersyarat (*uncon-*

ditional jump) secara absolut maupun relatif, karena mampu membawa bentang alamat *immediate* 20-bit.

Selain instruksi dasar RV32I, RISC-V juga menyediakan berbagai ekstensi opsional yang memungkinkan penyesuaian set instruksi dengan kebutuhan aplikasi tanpa mengubah spesifikasi arsitektur utamanya. Beberapa ekstensi yang umum digunakan pada sistem *embedded* meliputi:

- Ekstensi `M` (*Integer Multiplication and Division*): Menambahkan instruksi perangkat keras murni untuk perkalian dan pembagian bilangan bulat.
- Ekstensi `A` (*Atomic Instructions*): Menambahkan instruksi untuk sinkronisasi data pada prosesor *multicore*, sehingga beberapa inti dapat mengakses data yang sama dengan aman tanpa menimbulkan konflik.
- Ekstensi `F` dan `D` (*Floating-Point*): Menambahkan 32 register khusus untuk operasi bilangan pecahan (*floating-point*). Ekstensi `F` mendukung perhitungan dengan presisi tunggal (*single precision*), sedangkan `D` mendukung presisi ganda (*double precision*). Keduanya mengikuti standar IEEE 754-2008 sehingga hasil perhitungannya konsisten dan kompatibel dengan sistem lain.
- Ekstensi `C` (*Compressed Instructions*): Menyediakan format instruksi berukuran 16-bit sebagai alternatif dari instruksi standar 32-bit. Dengan ukuran instruksi yang lebih kecil, kebutuhan memori program dapat berkurang sekitar 25–30%, sehingga bermanfaat untuk mikrokontroler yang memiliki kapasitas memori terbatas.
- Ekstensi `Zicsr` (*Control and Status Register*): Menyediakan instruksi untuk mengakses dan mengelola *Control and Status Register* (CSR), yaitu register yang menyimpan informasi penting mengenai kondisi dan pengaturan prosesor. Pada spesifikasi RISC-V terbaru, ekstensi ini dipisahkan dari instruksi dasar (RV32I), sehingga harus dinyatakan secara eksplisit sebagai flag arsitektur saat proses kompilasi.

2.2.2 *System on Chip* (SoC)

System on Chip (SoC) adalah sirkuit terintegrasi (*Integrated Circuit* atau IC) yang menggabungkan prosesor, *cache*, memori, dan berbagai perangkat masukan/keluaran (I/O) ke dalam satu keping silikon tunggal [9]. Pada komputer konvensional, komponen-komponen seperti CPU (Central Processing Unit), memori, pengendali penyimpanan, dan antarmuka I/O dipasang sebagai perangkat terpisah di atas papan sirkuit cetak (PCB). Konsep SoC mengintegrasikan seluruh komponen tersebut dalam satu chip, sehingga menghasilkan sistem yang lebih ringkas, hemat daya, dan efisien. Sebuah SoC umumnya mencakup beberapa komponen utama sebagai berikut:

1. Prosesor (*Cores*) merupakan unit pemrosesan utama yang bertindak sebagai pengen-

dali logik. Prosesor ini mengeksekusi instruksi perangkat lunak utama.

2. Memori (*Memory*) merupakan ruang penyimpanan data dan instruksi yang diperlukan oleh prosesor untuk beroperasi. Memori dapat berupa RAM (Random Access Memory) atau ROM (Read-Only Memory).
3. Blok Periferal atau I/O merupakan berbagai macam antarmuka piranti keras untuk berinteraksi dengan sensor dan aktuator (seperti ADC, DAC, UART, SPI, I2C, GPIO, Ethernet, dan lain sebagainya).
4. Sistem Interkoneksi merupakan sistem yang menghubungkan prosesor, memori, dan periferal lainnya. Sistem interkoneksi ini dapat berupa bus, switch, atau jaringan. Sistem interkoneksi ini juga dapat berupa jaringan yang menghubungkan beberapa prosesor, memori, dan periferal lainnya.

2.2.3 Platform PULP dan PULPino

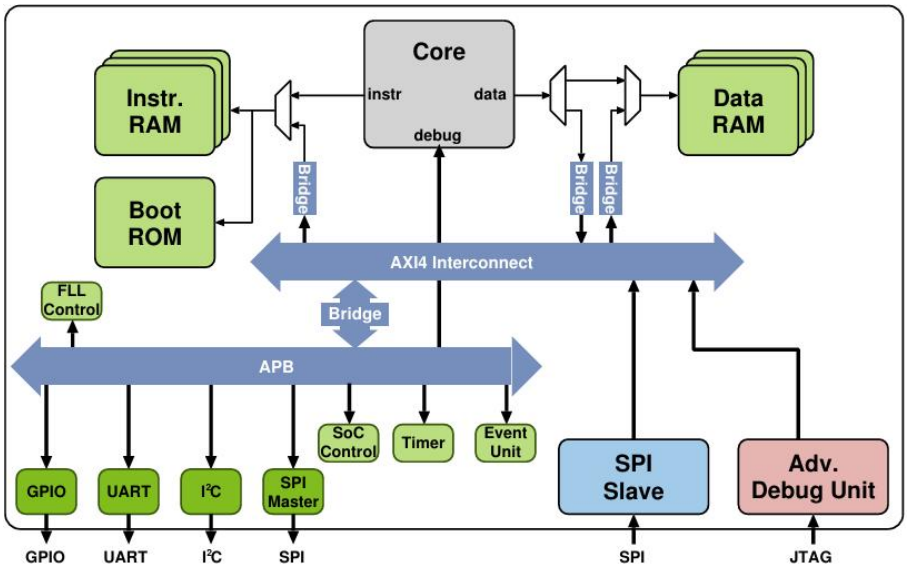
PULP (*Parallel Ultra-Low-Power*) merupakan platform sumber terbuka *open-source* berbasis RISC-V yang dikembangkan oleh kelompok riset dari ETH Zurich dan University of Bologna. Platform ini awalnya ditujukan untuk menghasilkan sistem komputasi berdaya rendah bagi aplikasi *Internet of Things* (IoT), kemudian berkembang menjadi ekosistem untuk penelitian dan pengembangan berbagai implementasi prosesor RISC-V.

Salah satu implementasi dari ekosistem tersebut adalah PULPino, yaitu *System on Chip* (SoC) *single-core* yang dirancang sebagai platform referensi untuk pengembangan dan evaluasi sistem berbasis RISC-V [1].



Gambar 2.2. Logo PULPino

PULPino mendukung penggunaan inti prosesor RI5CY maupun zero-riscy serta mengintegrasikan berbagai periferal penting, seperti memori, UART, SPI, I2C, GPIO, PWM, timer, dan pengendali interupsi, dalam satu sistem yang dihubungkan melalui Advanced eXtensible Interface (AXI) dan Advanced Peripheral Bus (APB).



Gambar 2.3. Diagram Sistem Keseluruhan PULPino (*Overview*)

Dalam konteks implementasi di FPGA, memori program tidak disimpan pada *Boot ROM* karena *Boot ROM* bersifat *read-only* dan ditujukan untuk urutan *booting* bawaan. Alih-alih, program utama dikirim secara dinamis oleh prosesor *host* (yakni aplikasi utilitas yang berjalan di *Processing System* / PS ARM) lalu disimpan ke dalam memori *Instruction RAM* dan *Data RAM* di dalam *Programmable Logic* (PL). Aliran data ini bermula dari komunikasi SPI antara ARM PS menuju *SPI Slave* di dalam PULPino. Dari *SPI Slave*, data ditransfer melalui *AXI4 Interconnect* secara langsung (*direct memory access*) ke *Instruction RAM* dan *Data RAM* tanpa intervensi instruksi dari *core* RISC-V. Setelah memori berhasil dimuat sepenuhnya, *core* RISC-V baru diaktifkan agar dapat mulai melakukan *fetch* instruksi dari RAM tersebut.

2.2.4 Peta Memori (*Memory Map*) PULPino

2.2.4.1 Organisasi Ruang Alamat

Untuk mengelola tata letak ruang penyimpanan, aliran instruksi, serta pemetaan periferal (*memory-mapped I/O*), PULPino mendefinisikan struktur peta memori yang spesifik. Tabel 2.1 menjabarkan rentang alamat dasar dari subsistem memori dan modul periferal bawaan yang dialokasikan dalam arsitektur SoC PULPino.

Tabel 2.1. Peta Memori (*Memory Map*) Default PULPino

Blok Memori / Modul	**Alamat Dasar (*Base*)**	**Keterangan / Ukuran**
Instruction Memory	`0x0000_0000`	`0x0000_8000` (32 KB RAM)
Boot ROM (Read-Only)	`0x0008_0000`	`0x0008_0200` (512 B ROM)
Data Memory	`0x0010_0000`	`0x0010_8000` (32 KB RAM)
UART	`0x1A10_0000`	Antarmuka Periferal
GPIO	`0x1A10_1000`	Antarmuka Periferal
SPI Master	`0x1A10_2000`	Antarmuka Periferal
Timer	`0x1A10_3000`	Antarmuka Periferal
Event/Interrupt Unit	`0x1A10_4000`	Antarmuka Periferal
I2C	`0x1A10_5000`	Antarmuka Periferal
FLL	`0x1A10_6000`	Antarmuka Periferal
SoC Control	`0x1A10_7000`	Antarmuka Periferal
Debug Port	`0x1A11_0000`	Antarmuka Periferal

2.2.4.2 Memory-Mapped I/O

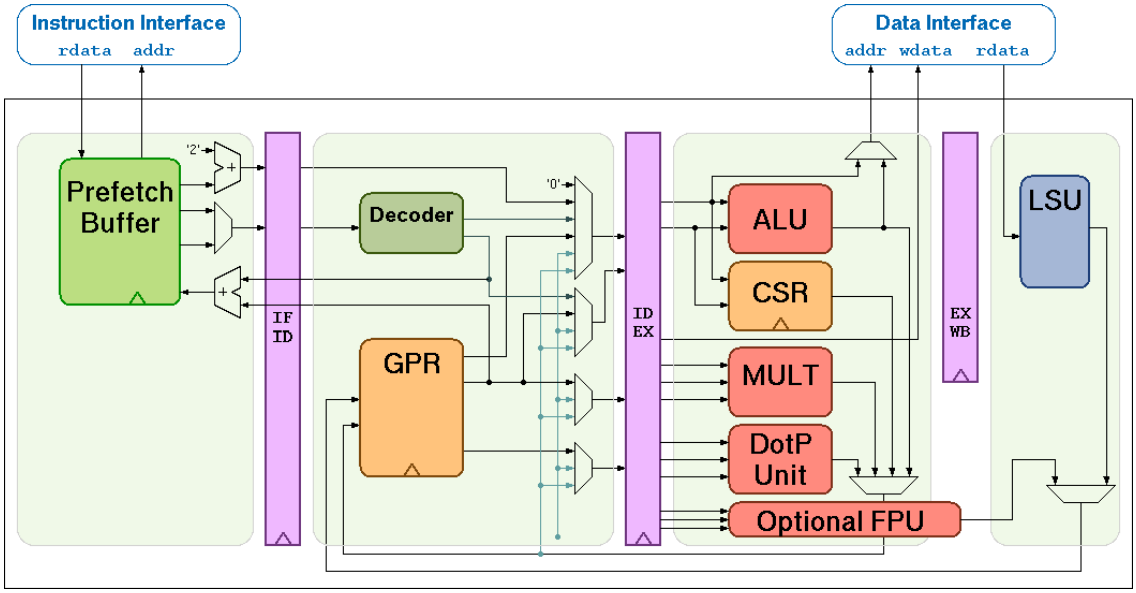
Memory-Mapped Input/Output (MMIO) merupakan teknik pengalamatan periferal dengan memetakan register kendali, register status, maupun register data perangkat keras ke dalam ruang alamat memori prosesor [10]. Melalui pendekatan ini, prosesor tidak memerlukan ruang alamat I/O maupun instruksi I/O khusus karena akses terhadap periferal dilakukan menggunakan instruksi baca dan tulis memori yang sama, seperti instruksi *load* dan *store* pada arsitektur RISC-V [10].

Pada platform SoC PULPino, periferal seperti GPIO, UART, SPI, dan I2C ditempatkan pada alamat tetap (*fixed address*) di dalam ruang alamat sistem [1]. Ketika *core* RI5CY melakukan operasi baca atau tulis pada alamat tersebut, transaksi memori terlebih dahulu diteruskan melalui antarmuka AXI menuju interkoneksi sistem. Selanjutnya, modul *AXI-to-APB Bridge* menerjemahkan transaksi tersebut ke bus APB yang digunakan oleh periferal berkecepatan rendah. Penggunaan bus APB dipilih karena memiliki protokol yang lebih sederhana dan kebutuhan sumber daya perangkat keras yang lebih kecil dibandingkan bus AXI, sehingga sesuai untuk komunikasi dengan periferal [1].

Keunggulan pendekatan MMIO adalah penyederhanaan pengembangan perangkat lunak tingkat rendah karena seluruh periferal dapat diakses menggunakan mekanisme pengalamatan memori yang seragam. Pada bahasa C, register periferal umumnya diakses melalui pointer yang dideklarasikan menggunakan kata kunci `volatile` agar setiap

operasi baca maupun tulis benar-benar diteruskan ke perangkat keras tanpa dioptimasi oleh kompilator. Pada penelitian ini, mekanisme tersebut digunakan untuk mengendalikan GPIO, UART, SPI, dan I2C melalui alamat register yang telah ditentukan pada *memory map* PULPino.

2.2.5 Arsitektur *Core* RI5CY



Gambar 2.4. Diagram *Core* RI5CY pada Arsitektur PULPino

Di dalam SoC PULPino, unit pemrosesan utamanya menggunakan *core* RI5CY (saat ini dikenal dengan CV32E40P [2]). Arsitektur RI5CY mengimplementasikan eksekusi instruksi ke dalam (*pipeline*) sepanjang 4 tahap (*4-stage pipeline*) [7]. Keempat tahapan komputasi ini dipisahkan secara struktural oleh *pipeline registers*, yang memungkinkan instruksi yang saling berurutan dapat diproses secara bersamaan pada tahapan yang berbeda dalam satu siklus *clock*. Adapun penjabaran fungsi dari masing-masing tahapan adalah sebagai berikut:

1. *Instruction Fetch* (IF) adalah tahapan paling awal di mana blok *Prefetch Buffer* bertugas mengambil instruksi dari memori program secara terus-menerus.
2. *Instruction Decode* (ID) adalah tahapan di mana instruksi yang telah diambil diterjemahkan oleh unit *Decoder*. Pada tahap ini, nilai operan dibaca dari *General Purpose Register* (GPR) dan unit *forwarding* diatur guna mencegah *data hazard*.
3. *Execute* (EX) adalah tahapan eksekusi aktual dari operasi komputasi (seperti unit aritmatika/ALU dan unit perkalian/*Multiplier*). Selain komputasi, operasi pembacaan dan penulisan variabel ke memori secara fisik juga ditangani di tahap ini oleh blok *Load Store Unit* (LSU).

4. *Write Back* (WB) adalah tahapan akhir dari siklus instruksi di mana hasil operasi komputasi maupun data yang baru saja dibaca dari memori disimpan kembali (*write back*) ke dalam *General Purpose Register* tujuan.

2.2.5.1 Pipeline dan Modul Utama Arsitektur RI5CY

Berikut adalah fungsi dari setiap komponen utama pada arsitektur RI5CY:

a. Prefetch buffer bertugas mengambil instruksi dari memori lebih awal dan menyimpannya sementara agar pasokan instruksi ke *pipeline* tetap lancar.

b. Decoder menerjemahkan instruksi menjadi sinyal kendali yang mengatur operasi setiap unit fungsional di dalam prosesor.

c. GPR (*General Purpose Register*) menyediakan 32 register berukuran 32-bit sebagai tempat penyimpanan *operand* dan hasil komputasi sementara.

d. ALU (*Arithmetic Logic Unit*) menjalankan operasi aritmatika dan logika, seperti penjumlahan, pengurangan, serta operasi *bitwise*.

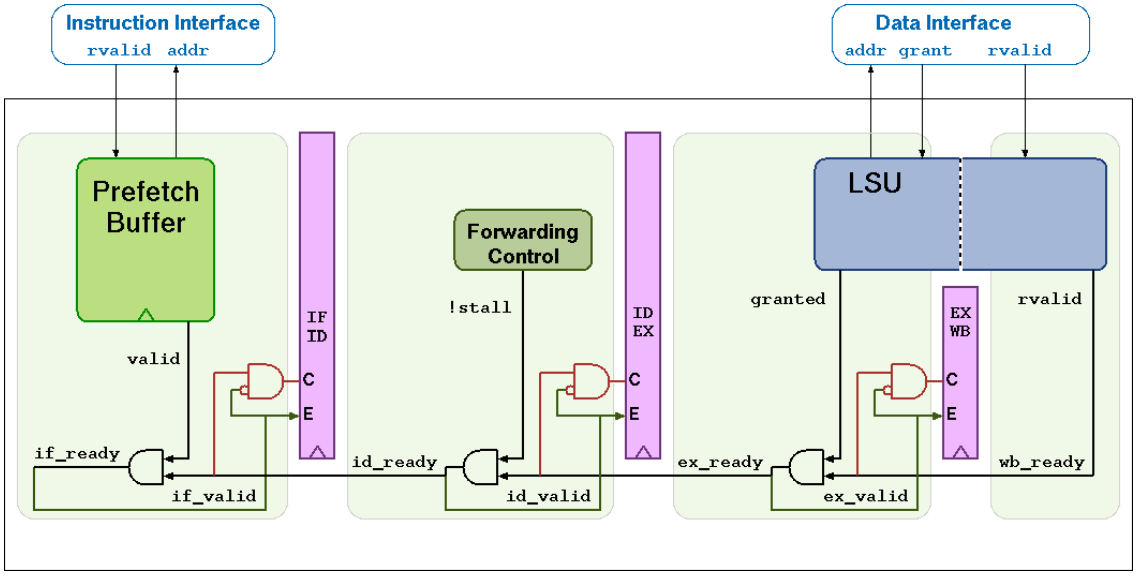
e. CSR (*Control and Status Register*) menyimpan konfigurasi dan status prosesor, termasuk pengaturan interupsi dan kontrol sistem.

f. MULT/DIV merupakan unit khusus yang menangani operasi perkalian dan pembagian sehingga beban komputasi ALU dapat dikurangi.

g. Optional FPU (*Floating Point Unit*) menyediakan dukungan perangkat keras untuk operasi bilangan floating-point sesuai standar IEEE 754 sehingga perhitungan numerik dapat dilakukan dengan lebih efisien.

h. *Dot Product Unit* menjalankan operasi perkalian titik (dot product) menggunakan pendekatan SIMD sehingga beberapa operasi dapat diproses secara paralel dalam satu instruksi.

i. LSU (*Load Store Unit*) mengelola proses pembacaan dan penulisan data antara register di dalam prosesor dengan memori melalui antarmuka data.



Gambar 2.5. Diagram *Pipeline Core* RI5CY

2.2.5.2 Arsitektur Harvard

Arsitektur Harvard merupakan organisasi komputer yang memisahkan ruang penyimpanan beserta jalur akses untuk instruksi dan data. Berbeda dengan arsitektur Von Neumann yang menggunakan satu ruang memori dan satu jalur bus bersama sehingga berpotensi menimbulkan *Von Neumann bottleneck*, arsitektur Harvard menyediakan memori instruksi (*Instruction Memory*) dan memori data (*Data Memory*) yang dapat diakses melalui jalur independen [11]. Pemisahan tersebut memungkinkan prosesor melakukan pengambilan instruksi (*instruction fetch*) secara bersamaan dengan operasi akses data (*load/store*), sehingga meningkatkan efisiensi eksekusi instruksi serta mengurangi kemungkinan terjadinya *structural hazard* akibat perebutan akses memori [12].

Core RI5CY yang digunakan pada platform PULPino mengimplementasikan prinsip *Modified Harvard Architecture* [7]. Pada tingkat inti prosesor, RI5CY menyediakan antarmuka memori instruksi dan memori data yang terpisah untuk mendukung proses *pipeline* empat tahap. Dengan organisasi tersebut, tahap *Instruction Fetch* (IF) dapat mengambil instruksi berikutnya secara paralel ketika *Load-Store Unit* (LSU) melakukan operasi baca atau tulis data. Meskipun demikian, pada tingkat *System-on-Chip* kedua antarmuka tersebut tetap terhubung ke ruang alamat sistem yang sama melalui interkoneksi bus AXI sehingga akses menuju memori dan periferal dapat dikelola secara terintegrasi.

Pada implementasi FPGA, pemisahan memori instruksi dan memori data umumnya direalisasikan menggunakan blok *Block RAM* (BRAM) yang terpisah. Organisasi ini memungkinkan kedua memori diakses secara independen sesuai kebutuhan masing-masing jalur antarmuka tanpa menimbulkan konflik akses. Pada penelitian ini, konsep tersebut diwujudkan melalui implementasi memori instruksi dan memori data yang

masing-masing dipetakan ke blok BRAM tersendiri pada FPGA Zybo Z-7010 sehingga mendukung eksekusi SoC PULPino secara efisien.

2.2.5.3 Mekanisme Penanganan *Data Hazard*

Data hazard terjadi ketika suatu instruksi membutuhkan hasil komputasi dari instruksi sebelumnya yang belum tersedia. RI5CY mengatasi kondisi tersebut melalui mekanisme *forwarding*, yaitu meneruskan hasil komputasi secara langsung ke instruksi berikutnya, serta mekanisme *stall* yang menghentikan sementara aliran pipeline apabila data yang diperlukan belum tersedia. Kedua mekanisme tersebut membantu menjaga kebenaran eksekusi instruksi sekaligus mempertahankan kinerja prosesor.

2.2.5.4 Antarmuka Memori Eksternal

RI5CY menggunakan arsitektur Harvard yang memisahkan jalur akses memori instruksi dan memori data. Dengan pemisahan tersebut, proses pengambilan instruksi (*instruction fetch*) dapat dilakukan secara bersamaan dengan akses data, sehingga mengurangi potensi structural hazard dan meningkatkan efisiensi eksekusi.

Arsitektur ini menyediakan dua antarmuka memori, yaitu *Instruction Interface* yang digunakan oleh *Prefetch Buffer* untuk mengambil instruksi, serta *Data Interface* yang digunakan oleh *Load Store Unit* (LSU) untuk melakukan operasi *load* dan *store*.

2.2.5.5 Logika *Handshaking* dan Kendali Antar-Tahapan (*Inter-Stage*)

Komunikasi antar tahapan pipeline pada RI5CY menggunakan mekanisme *handshaking* berbasis sinyal *valid* dan *ready*. Instruksi hanya diteruskan ke tahapan berikutnya apabila data telah *valid* dan tahapan tujuan siap menerima data. Mekanisme ini menjaga sinkronisasi aliran instruksi sekaligus mencegah terjadinya kehilangan data selama proses eksekusi.

2.2.6 Protokol Komunikasi Register Memori dan Arsitektur Bus Interkoneksi

Di dalam SoC PULPino, komunikasi data antar-komponen diatur melalui protokol bus berbasis standar AMBA (*Advanced Microcontroller Bus Architecture*) dari Arm. Terdapat dua standar interkoneksi utama yang digunakan, yaitu AXI (*Advanced eXtensible Interface*) dan APB (*Advanced Peripheral Bus*).

Bus AXI difungsikan sebagai interkoneksi utama berkinerja tinggi (*main interconnect*) yang menangani lalu lintas pita lebar tinggi antara inti *core* RI5CY, memori RAM, dan pengontrol akses langsung (*Direct Memory Access*). Sementara itu, bus APB diimplementasikan sebagai jaringan antarmuka sekunder yang lebih hemat daya, dikhususkan untuk melayani modul-modul subsistem I/O yang lebih lambat, seperti UART,

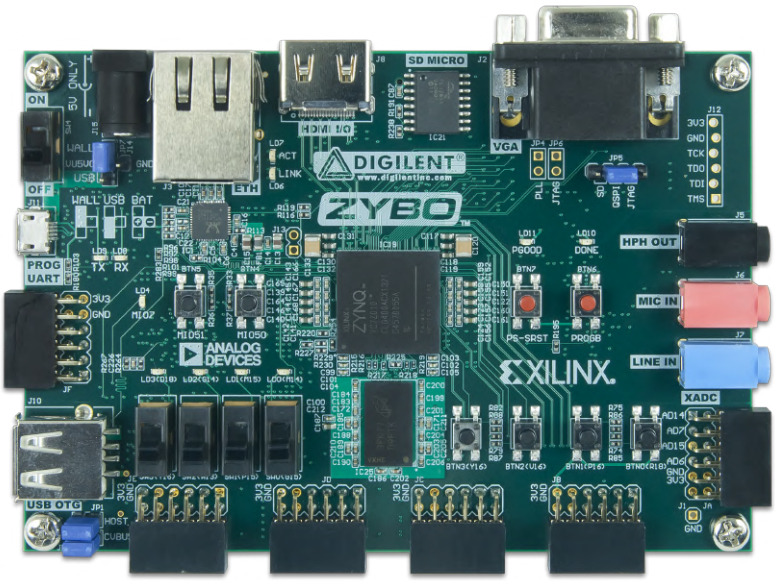
SPI, GPIO, dan I2C. Untuk menjembatani perbedaan kecepatan operasi dan kompleksitas sinyal antara jaringan AXI dan APB, arsitektur PULPino menyertakan modul jembatan penerjemah penurun kecepatan (*AXI-to-APB bridge*).

2.2.7 Antarmuka Periferal

Dalam SoC PULPino, komponen periferal berperan krusial dalam komunikasi data ke dunia luar. Beberapa periferal utama yang diimplementasikan meliputi:

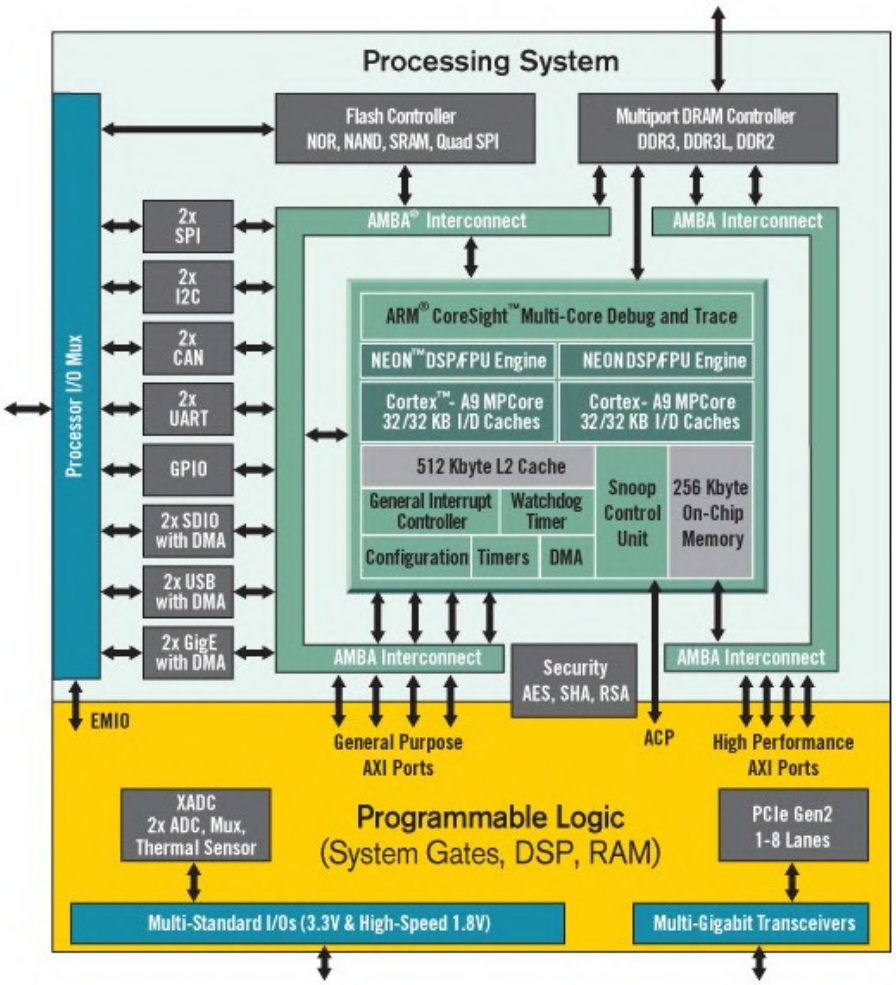
- UART (*Universal Asynchronous Receiver-Transmitter*) : Merupakan antarmuka komunikasi serial asinkron. UART sangat penting untuk tujuan *debugging*, memungkinkan SoC PULPino untuk mengirim dan menerima teks dari perangkat host (komputer) melalui jalur RX (*Receive*) dan TX (*Transmit*) [13].
- GPIO (*General Purpose Input/Output*): Merupakan pin digital fleksibel yang perilakunya (apakah sebagai input atau output) dapat dikendalikan saat *runtime* oleh perangkat lunak [9]. GPIO digunakan untuk membaca sinyal dari tombol/saklar atau mengendalikan lampu LED.
- SPI (*Serial Peripheral Interface*): Merupakan protokol komunikasi serial sinkron berkecepatan tinggi yang berjalan pada mode *full-duplex* [14]. Protokol ini menggunakan arsitektur *master-slave* dan biasanya digunakan oleh PULPino untuk berkomunikasi dengan memori eksternal (seperti *flash* SPI) atau sensor-sensor.
- I2C (*Inter-Integrated Circuit*): Merupakan protokol komunikasi serial sinkron yang berjalan pada mode *half-duplex* [15]. Protokol ini menggunakan arsitektur *master-slave* dan biasanya digunakan oleh PULPino untuk berkomunikasi dengan sensor-sensor.

2.2.8 FPGA Zybo *development board* dan SoC Zynq-7000



Gambar 2.6. Zybo *Development Board* Zynq-7010

Field Programmable Gate Array (FPGA) adalah perangkat semikonduktor dan merupakan bentuk paling umum dari perangkat keras yang dapat dikonfigurasi ulang (*re-configurable hardware*) [15]. Pada penelitian ini digunakan Zybo *development board* dari Digilent yang mengadopsi SoC Xilinx Zynq-7000 dengan nomor xc7z010-1clg400c [16]. Papan pengembangan ini dipilih karena mengintegrasikan prosesor ARM dan FPGA dalam satu chip sehingga mendukung implementasi serta pengujian SoC PULPino.



Gambar 2.7. Arsitektur Internal SoC Xilinx Zynq-7000

Zynq-7000 terdiri atas dua bagian utama, yaitu Processing System (PS) dan Programmable Logic (PL), sebagaimana ditunjukkan pada Gambar 2.7. PS berisi prosesor dual-core ARM Cortex-A9 beserta pengendali memori dan periferal bawaan yang mampu menjalankan sistem operasi maupun aplikasi bare-metal. Sementara itu, PL menyediakan sumber daya FPGA yang digunakan untuk mengimplementasikan perangkat keras kustom. Pada penelitian ini, keseluruhan SoC PULPino diimplementasikan pada wilayah PL, sedangkan PS berperan sebagai *host* yang menjalankan Linux untuk mendukung proses *bootloading*, komunikasi UART, serta pengelolaan sistem selama pengujian.

Komunikasi antara PS dan PL menggunakan antarmuka AXI (*Advanced eXtensible Interface*) sehingga prosesor ARM dapat mengakses logika yang berada di wilayah FPGA. Mekanisme ini dimanfaatkan untuk mendukung proses pengunggahan program ke PULPino serta pertukaran data antara sistem *host* dan SoC yang diimplementasikan. Selain itu, Zybo juga menyediakan berbagai periferal *on-board* seperti LED, *switch*, *push button*, port PMOD, USB-UART, dan slot microSD yang dimanfaatkan sebagai media implementasi dan pengujian pada penelitian ini.

2.2.8.1 Mekanisme Booting pada SoC Zynq-7000

Proses *booting* pada *System-on-Chip* (SoC) Zynq-7000 berlangsung melalui beberapa tahapan yang melibatkan *BootROM*, *First Stage Boot Loader* (FSBL), dan *Second Stage Boot Loader* (SSBL) sebelum sistem operasi dapat dijalankan. Mekanisme ini memungkinkan inisialisasi bertahap pada sisi *Processing System* (PS) serta konfigurasi wilayah *Programmable Logic* (PL) sesuai kebutuhan aplikasi [17].

Tahap pertama diawali oleh *BootROM*, yaitu program permanen yang tersimpan pada memori internal PS dan dieksekusi secara otomatis setelah perangkat mengalami *Power-On Reset* (POR). BootROM membaca konfigurasi *boot mode* untuk menentukan media penyimpanan yang digunakan, seperti kartu SD, QSPI Flash, maupun JTAG. Selanjutnya, BootROM memuat *boot image* dari media tersebut ke dalam *On-Chip Memory* (OCM) dan mengeksekusi *First Stage Boot Loader* (FSBL) [17].

FSBL bertanggung jawab melakukan inisialisasi subsistem PS, termasuk konfigurasi *clock*, antarmuka *Multiplexed I/O* (MIO), pengendali memori DDR, serta periferal dasar yang diperlukan selama proses *boot*. Apabila *boot image* memuat berkas *bitstream*, FSBL juga mengonfigurasi wilayah PL sebelum menyerahkan kendali kepada *Second Stage Boot Loader*, yang pada umumnya menggunakan U-Boot [17].

Selanjutnya, U-Boot mempersiapkan lingkungan eksekusi sistem operasi dengan memuat citra kernel Linux (`uImage`), berkas *device tree* (`devicetree.dtb`), dan *root filesystem* ke dalam memori DDR. Setelah seluruh komponen berhasil dimuat, kendali diberikan kepada kernel Linux untuk melanjutkan proses inisialisasi sistem hingga sistem operasi siap menjalankan aplikasi pengguna [17].

2.2.9 Tahapan Sintesis dan Implementasi FPGA

Proses implementasi suatu rancangan digital pada FPGA diawali dengan tahap sintesis (*synthesis*), yaitu proses menerjemahkan deskripsi perangkat keras yang ditulis menggunakan bahasa HDL (*Hardware Description Language*) ke dalam bentuk *gate-level netlist*. Pada tahap ini, perangkat lunak sintesis melakukan optimasi terhadap logika digital yang dibangun serta menghasilkan laporan awal mengenai estimasi penggunaan sumber daya perangkat dan karakteristik *timing* rancangan [18].

Selanjutnya, *netlist* hasil sintesis diproses pada tahap implementasi (*implementation*) untuk dipetakan ke sumber daya fisik FPGA. Tahap implementasi mencakup optimasi fisik (*physical optimization*), penempatan elemen logika pada lokasi yang tersedia di dalam perangkat (*placement*), serta pembentukan jalur interkoneksi antar elemen logika (*routing*). Setelah proses tersebut selesai, perangkat lunak melakukan analisis *Static Timing Analysis* (STA) guna memverifikasi bahwa seluruh lintasan sinyal memenuhi batasan waktu (*timing constraints*) yang telah ditetapkan [18].

Apabila hasil implementasi menunjukkan bahwa seluruh kendala desain telah terpenuhi, termasuk tidak ditemukannya pelanggaran *timing*, maka proses dilanjutkan dengan pembangkitan berkas konfigurasi FPGA (*bitstream*). Berkas *bitstream* tersebut selanjutnya digunakan untuk mengonfigurasi perangkat FPGA sehingga seluruh sumber daya logika, interkoneksi, dan elemen memori bekerja sesuai dengan fungsi yang telah dirancang.

2.2.10 Penggunaan Sumber Daya FPGA

FPGA tersusun atas berbagai sumber daya perangkat keras yang dapat diprogram untuk mengimplementasikan suatu rangkaian digital. Tingkat penggunaan setiap sumber daya tersebut ditampilkan pada laporan *resource utilization* yang dihasilkan selama proses implementasi. Laporan ini memberikan informasi mengenai efisiensi pemanfaatan sumber daya FPGA oleh rancangan yang diimplementasikan.

Look-Up Table (LUT) merupakan elemen logika dasar yang berada di dalam *Configurable Logic Block* (CLB) dan digunakan untuk merealisasikan fungsi logika kombinasional berdasarkan tabel kebenaran Boolean. Pada keluarga FPGA Xilinx 7-Series, sebagian *slice* diklasifikasikan sebagai *SLICEM*, yaitu *slice* yang memungkinkan LUT dikonfigurasi sebagai *distributed RAM* maupun *shift register*, sedangkan *SLICEL* hanya digunakan untuk fungsi logika dan register [19]. Semakin kompleks fungsi logika yang diimplementasikan, semakin besar pula jumlah LUT yang dibutuhkan.

Flip-Flop (FF) merupakan elemen penyimpan data sekuensial yang bekerja secara sinkron terhadap sinyal *clock*. FF digunakan untuk membangun register, jalur *pipeline*, maupun *finite state machine* (FSM) pada suatu sistem digital. Jumlah FF yang digunakan umumnya dipengaruhi oleh banyaknya register, kedalaman *pipeline*, serta lebar jalur data yang diterapkan pada rancangan [19].

Block RAM (BRAM) merupakan memori internal berbasis blok khusus yang disediakan oleh FPGA untuk menyimpan data berkapasitas relatif besar. BRAM umumnya dimanfaatkan untuk mengimplementasikan memori instruksi, memori data, maupun *buffer* komunikasi tanpa mengonsumsi sumber daya LUT dalam jumlah yang signifikan [20].

Selain elemen logika dasar, FPGA juga menyediakan blok *Digital Signal Processing* (DSP), seperti arsitektur DSP48E1 pada FPGA Xilinx 7-Series, yang dioptimalkan untuk melaksanakan operasi aritmetika berkecepatan tinggi [21]. Blok DSP48E1 menyediakan perangkat keras khusus yang mendukung operasi perkalian, penjumlahan, pengurangan, akumulasi, serta operasi *multiply-accumulate* (MAC) yang banyak digunakan pada aplikasi pemrosesan sinyal digital. Pemanfaatan blok DSP memungkinkan operasi aritmetika direalisasikan menggunakan sumber daya khusus sehingga dapat mengurangi penggunaan LUT serta berpotensi meningkatkan performa implementasi dibandingkan

apabila seluruh operasi direalisasikan menggunakan logika umum [20, 21].

Pada penelitian ini, parameter penggunaan LUT, FF, BRAM, dan DSP digunakan sebagai indikator utama untuk mengevaluasi karakteristik implementasi SoC PULPino pada FPGA Zybo Z-7010. Analisis terhadap keempat sumber daya tersebut dilakukan berdasarkan laporan *Resource Utilization* yang dihasilkan oleh perangkat lunak Xilinx Vivado setelah proses implementasi selesai.

2.2.11 Analisis *Timing* FPGA

Selain penggunaan sumber daya perangkat, keberhasilan implementasi suatu rancangan pada FPGA juga ditentukan oleh terpenuhinya kendala waktu (*timing constraints*). Analisis *timing* dilakukan untuk memastikan bahwa seluruh lintasan data memenuhi persyaratan *setup* dan *hold* terhadap sinyal *clock* yang digunakan sehingga rangkaian dapat beroperasi secara andal pada frekuensi yang ditentukan [22].

Evaluasi karakteristik *timing* umumnya dilakukan menggunakan metode *Static Timing Analysis* (STA), yaitu metode analisis yang menghitung waktu propagasi seluruh lintasan sinyal tanpa memerlukan simulasi berbasis stimulus. Metode ini digunakan secara luas pada proses implementasi FPGA karena mampu mengidentifikasi lintasan kritis (*critical path*) serta mengevaluasi apakah seluruh kendala *timing* telah terpenuhi [22].

Beberapa parameter utama yang dihasilkan oleh STA meliputi *Worst Negative Slack* (WNS), *Total Negative Slack* (TNS), *Worst Hold Slack* (WHS), dan *Total Hold Slack* (THS). WNS menunjukkan margin waktu minimum terhadap kendala *setup* pada lintasan paling kritis, sedangkan WHS menunjukkan margin waktu minimum terhadap kendala *hold*. Adapun TNS dan THS menyatakan akumulasi pelanggaran *setup* maupun *hold* pada seluruh lintasan yang gagal memenuhi batasan waktu [22].

Nilai WNS yang bernilai positif menunjukkan bahwa seluruh lintasan masih memenuhi kendala *setup* yang diberikan. Sebaliknya, nilai WNS yang bernilai negatif mengindikasikan adanya pelanggaran *setup timing*, yaitu kondisi ketika data tidak tiba di register tujuan sebelum tepi aktif *clock* berikutnya. Kondisi tersebut dapat menyebabkan data yang tersimpan pada register menjadi tidak valid dan pada kondisi tertentu berpotensi memicu metastabilitas.

Apabila seluruh kendala *timing* berhasil dipenuhi (*timing closure*), lintasan kritis memiliki waktu propagasi sebesar $T_{\text{clock}} - WNS$. Oleh karena itu, estimasi frekuensi kerja maksimum rancangan dapat dihitung menggunakan persamaan berikut [18]:

$$F_{\max} = \frac{1}{T_{\text{clock}} - WNS} \tag{2-1}$$

dengan T_{clock} merupakan periode *clock* target yang digunakan sebagai kendala

desain, sedangkan WNS merupakan margin waktu pada lintasan kritis hasil analisis implementasi.

Pada penelitian ini, parameter WNS, WHS, TNS, dan THS digunakan untuk mengevaluasi keberhasilan implementasi SoC PULPino setelah proses *placement* dan *routing* selesai dilakukan. Selain itu, nilai WNS dimanfaatkan untuk mengestimasi frekuensi kerja maksimum dari rancangan yang diimplementasikan.

2.2.12 Ekosistem Perangkat Lunak Pendukung

Implementasi SoC PULPino pada FPGA tidak hanya melibatkan proses sintesis perangkat keras, tetapi juga kompilasi perangkat lunak, pembangunan sistem operasi, hingga proses *bootloading*. Oleh karena itu, penelitian ini memanfaatkan beberapa perangkat lunak pendukung yang saling terintegrasi. Xilinx Vivado digunakan untuk sintesis, implementasi, dan menghasilkan *bitstream* FPGA, sedangkan Xilinx Vitis digunakan untuk membangun *First Stage Bootloader* (FSBL) serta mendukung proses pengembangan perangkat lunak pada platform Zynq. Program aplikasi yang dijalankan pada inti prosesor RI5CY dikompilasi menggunakan *toolchain* RISC-V GCC dengan sistem pembangunan berbasis CMake.

Selain perangkat lunak tersebut, implementasi sistem juga memanfaatkan beberapa komponen pendukung pada lingkungan Linux. Perangkat lunak yang digunakan dalam penelitian ini meliputi:

1. WSL 2 (Windows Subsystem for Linux) [23] digunakan untuk menyediakan lingkungan GNU/Linux pada sistem operasi Windows sehingga proses kompilasi dan eksekusi skrip berbasis Unix dapat dilakukan tanpa instalasi sistem operasi terpisah.
2. U-Boot (*Das U-Boot*) [24] digunakan sebagai *bootloader* yang bertugas memuat kernel Linux dari media penyimpanan ke memori utama setelah proses inisialisasi perangkat keras oleh FSBL selesai dilakukan.
3. *Buildroot* [25] digunakan untuk membangun sistem Linux *embedded*, meliputi *toolchain*, *root filesystem*, pustaka dasar, dan utilitas yang diperlukan agar sistem operasi dapat dijalankan pada Processing System Zynq.
4. *Device Tree* [26] digunakan untuk mendeskripsikan konfigurasi perangkat keras kepada kernel Linux, seperti alokasi memori dan periferal yang tersedia, sehingga kernel dapat mengenali perangkat keras tanpa memerlukan perubahan pada kode sumber.
5. *CMake* [27] digunakan sebagai sistem *build automation* untuk mengelola konfigurasi kompilasi proyek PULPino, termasuk pengaturan *toolchain*, pustaka, serta arsitektur target RISC-V yang digunakan pada penelitian ini.

BAB III
METODE PENELITIAN

Bab ini menjelaskan metodologi yang digunakan untuk mengimplementasikan *System on Chip* (SoC) PULPino pada FPGA Digilent Zybo *development board*. Pembahasan meliputi spesifikasi alat dan bahan, tahapan penelitian, arsitektur sistem yang diimplementasikan, proses implementasi perangkat keras dan perangkat lunak, serta skenario pengujian yang digunakan untuk memvalidasi operasional sistem.

3.1 Alat dan Bahan Tugas Akhir

3.1.1 Alat Tugas Akhir

Penelitian ini menggunakan berbagai perangkat keras dan perangkat lunak yang saling mendukung untuk melakukan eksperimen serta pengolahan data. Rincian mengenai spesifikasi dan fungsi dari masing-masing alat yang digunakan dalam penelitian ini ditunjukkan secara berturut-turut pada Tabel 3.1 dan Tabel 3.2.

Tabel 3.1. Perangkat Keras pada Penelitian

No.	Alat	Spesifikasi	Fungsi
1	Laptop Pribadi	Prosesor AMD 2500U @ 2.0 GHz, RAM 8 GB, SSD 256 GB	Sebagai stasiun kerja utama untuk instalasi perangkat lunak Vivado dan *build environment* kompilasi lintas arsitektur.
2	FPGA Digilent Zybo *development board*	System on Chip (SoC) Xilinx Zynq-7000 (seri `xc7z010clg400-1`)	Papan pengembangan silikon untuk penanaman, pengujian, dan implementasi sirkuit digital SoC PULPino.
3	Kartu microSD	Kapasitas penyimpanan 8 GB dengan kecepatan antarmuka Class 10	Media penyimpanan mandiri untuk menampung *bootloader*, *kernel* Linux, dan *root filesystem* sistem *host* ARM.
4	Kabel micro-USB	Kabel transmisi data ganda (*Data & Power*)	Menyediakan koneksi komunikasi UART serial antara komputer *host* dengan lingkungan Zybo.
5	Sensor PIR (HC-SR501)	Sensor inframerah pasif dengan tegangan kerja 5-15 V	Digunakan sebagai input untuk mendeteksi gerakan manusia di sekitar Zybo.
6	OLED GM009605v4	Layar OLED dengan resolusi 96x96 piksel dan antarmuka SPI	Digunakan sebagai output untuk menampilkan informasi status sistem.
7	Kabel Jumper	Kabel penghubung dengan panjang 10 cm	Digunakan untuk menghubungkan sensor PIR dan OLED GM009605v4 dengan FPGA Zybo.
8	Adaptor 12V 2A	Adaptor dengan output 12 V dan arus 2 A	Menyediakan daya untuk sensor PIR.

Tabel 3.2. Perangkat Lunak pada Penelitian

No.	Alat	Fungsi
1	OS Windows & WSL)	Windows bertindak sebagai OS *host* alat rancang Vivado, sedangkan WSL 2 Ubuntu 24.04 LTS memfasilitasi kompilasi *cross-platform* yang membutuhkan *case-sensitivity*.
2	Xilinx Vivado Design Suite 2025.1	*Integrated Development Environment* untuk menyintesis logika RTL SoC PULPino, penempatan, perutean, dan pencetakan *bitstream*.
3	Xilinx Vitis Unified IDE 2025.1	*Platform* terpadu untuk menyusun modul *First Stage Bootloader* (FSBL) dan memaketkannya ke dalam `BOOT.BIN`.
4	Python 2.7.18	*Interpreter* untuk mengeksekusi skrip penarikan arsitektur IP murni dari repositori *legacy* PULPino.
5	PuTTY / Tera Term	Terminal serial interaktif untuk memantau keluaran konsol Linux ARM dan membaca log komunikasi *core* RI5CY.
6	GNU RISC-V GCC (riscv64-unknown-elf-)	*Toolchain* untuk mengompilasi kode program dan *firmware* yang dijalankan di *core* RISC-V.
7	ARM GNU/Linux GCC (arm-xilinx-linux-gnueabi-)	*Toolchain* untuk mem-*build* sistem *host* ARM seperti U-Boot, *kernel* Linux, serta program `spiload`.

3.1.2 Bahan Tugas Akhir

Bahan yang digunakan dalam penelitian ini berupa dokumen spesifikasi, manual teknis, dan repositori kode sumber yang menjadi acuan dalam proses perancangan, implementasi, serta pengujian sistem. Daftar bahan penelitian ditunjukkan pada Tabel 3.3.

Tabel 3.3. Bahan Penelitian

No.	Bahan	Keterangan
1	*The RISC-V Instruction Set Manual* [8]	Dokumen resmi spesifikasi arsitektur set instruksi (ISA) standar RISC-V yang menjadi landasan eksekusi program dan acuan kompilasi perangkat lunak.
2	*RI5CY Core Reference Manual* [7]	Dokumen spesifikasi arsitektur inti prosesor RI5CY yang digunakan sebagai acuan implementasi, meliputi set instruksi khusus, organisasi register, serta fitur-fitur internal prosesor.
3	*PULPino System Description* [1]	Dokumen referensi yang menjelaskan arsitektur SoC PULPino, termasuk pemetaan memori, interkoneksi bus, dan periferal bawaan sistem.
4	*Zynq-7000 SoC Technical Reference Manual* [17]	Dokumen teknis arsitektur Xilinx Zynq-7000 yang digunakan sebagai panduan alokasi perutean I/O, antarmuka silang *Processing System* (PS), dan konfigurasi *Programmable Logic* (PL).
5	Repositori RTL PULPino [28]	Kode sumber RTL (*Register-Transfer Level*) berbasis SystemVerilog/Verilog yang digunakan sebagai acuan utama sebelum dimodifikasi dan disintesis.

3.2 Metode yang Digunakan

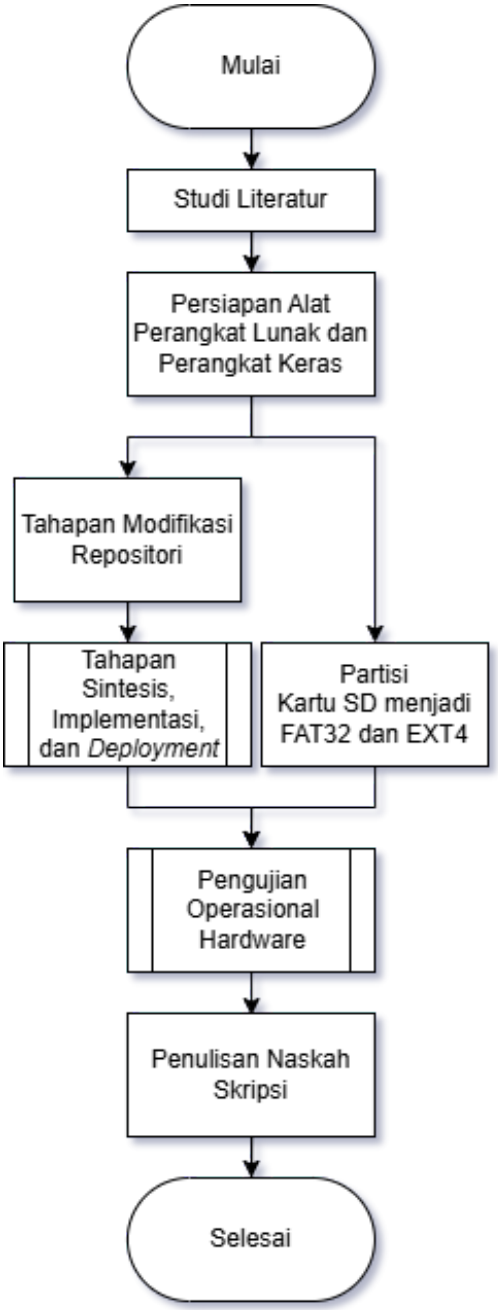
Penelitian diawali dengan implementasi System on Chip (SoC) PULPino pada FPGA Digilent Zybo *development board* berbasis Xilinx Zynq-7000. Tahapan penelitian meliputi proses adaptasi proyek PULPino, sintesis dan implementasi desain menggunakan perangkat lunak Vivado, penyusunan perangkat lunak berbasis bahasa C untuk inti prosesor RI5CY, hingga pengujian fungsional pada perangkat keras.

Analisis hasil penelitian dilakukan melalui dua tahapan. Tahap pertama berupa analisis laporan sintesis, yaitu dengan mengevaluasi hasil sintesis dan implementasi dari Vivado, seperti utilisasi sumber daya FPGA (*resource utilization*) serta *timing analysis*, kemudian membandingkannya dengan hasil yang diperoleh pada penelitian terdahulu. Tahap kedua berupa validasi operasional perangkat keras, yaitu dengan menguji secara langsung sistem yang telah diimplementasikan pada FPGA untuk memastikan bahwa inti prosesor RI5CY mampu menjalankan program serta berinteraksi dengan berbagai periferal, seperti LED, switch, button, PMOD, sensor PIR, dan OLED sesuai dengan

rancangan.

3.3 Alur Penelitian

Penelitian ini dilaksanakan secara bertahap mulai dari studi literatur hingga penyusunan naskah skripsi. Setiap tahapan saling berkaitan untuk menghasilkan implementasi SoC PULPino yang dapat dijalankan pada FPGA Digilent *development board* Zybo Z-7010. Diagram alur penelitian secara keseluruhan ditunjukkan pada Gambar 3.8.



Gambar 3.8. Diagram alur penelitian

Berdasarkan Gambar 3.8, penelitian dilaksanakan melalui tahapan sebagai beri-

kut.

1. **Studi Literatur**

   Tahap awal dilakukan dengan mempelajari berbagai referensi yang berkaitan dengan arsitektur RISC-V, SoC PULPino, FPGA Xilinx Zynq-7000, mekanisme *booting* pada SoC Zynq, serta dokumentasi resmi AMD dan penelitian terdahulu. Studi literatur digunakan sebagai dasar dalam proses adaptasi repositori, implementasi sistem, serta analisis hasil penelitian.

2. **Persiapan Perangkat Lunak dan Perangkat Keras**

   Tahap ini meliputi penyediaan perangkat keras berupa papan FPGA Digilent Zybo *development board* Z-7010, kartu SD beserta perangkat pendukung lainnya, serta instalasi perangkat lunak yang digunakan selama penelitian, seperti Xilinx Vivado, Xilinx Vitis, Windows Subsystem for Linux (WSL), Buildroot, dan *toolchain* RISC-V. Pada tahap ini juga dilakukan partisi kartu microSD menjadi partisi FAT32 sebagai media *boot* dan partisi ext4 sebagai media penyimpanan *root filesystem* Linux.

3. **Modifikasi Repositori**

   Repositori PULPino kemudian disesuaikan agar kompatibel dengan lingkungan pengembangan yang digunakan. Tahap ini mencakup penyesuaian struktur proyek, skrip sintesis, konfigurasi perangkat lunak, sistem kompilasi, serta beberapa berkas pendukung sehingga seluruh proses *build* dapat dijalankan menggunakan Vivado 2025.1, Vitis 2025.1, dan WSL.

4. **Tahapan Sintesis, Implementasi, dan *Deployment***

   Tahap ini merupakan proses utama penelitian yang menghasilkan sistem siap dijalankan pada FPGA. Proses tersebut terdiri atas beberapa subproses yang berjalan secara paralel, yaitu sintesis dan implementasi perangkat keras menggunakan Vivado, pembangunan sistem operasi Linux pada sisi PS, kompilasi perangkat lunak *bare-metal* RISC-V, pembuatan berkas *boot*, serta persiapan media *boot* dan *deployment* pada kartu microSD.

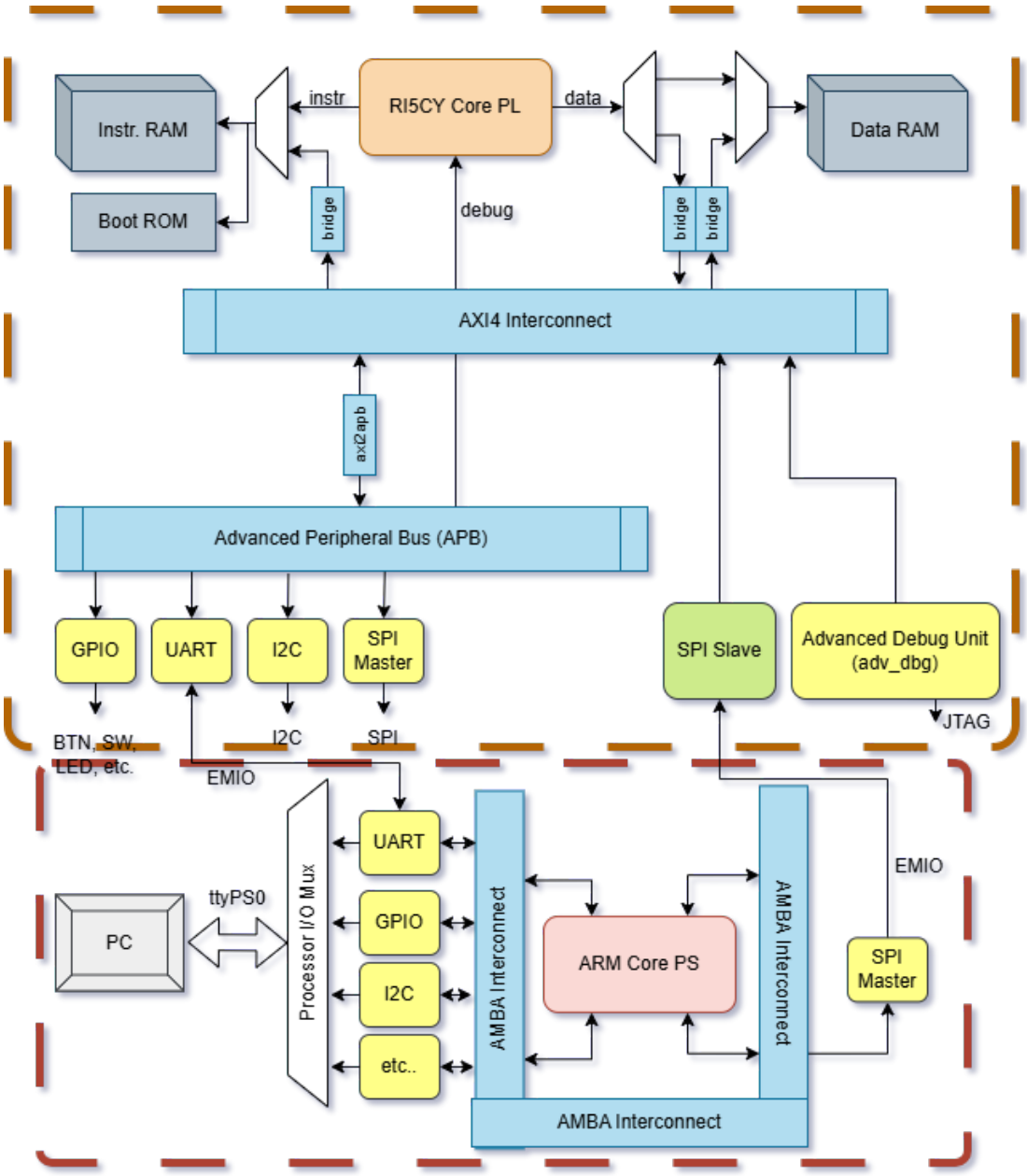
5. **Pengujian Operasional Perangkat Keras**

   Setelah proses *deployment* selesai, sistem dijalankan pada FPGA Zybo Z-7010 untuk memverifikasi keberhasilan implementasi. Pengujian dilakukan terhadap proses *booting*, komunikasi UART, antarmuka GPIO, modul PMOD, sensor PIR, serta modul OLED berbasis I2C guna memastikan seluruh periferal dapat berfungsi sesuai rancangan.

6. **Penulisan Naskah Skripsi**

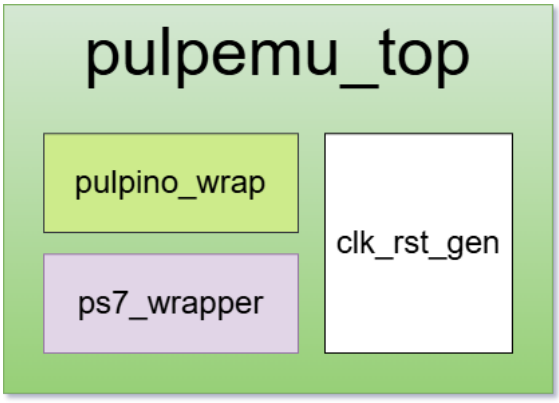
Tahap terakhir dilakukan dengan menyusun seluruh hasil implementasi, pengujian, serta analisis ke dalam naskah skripsi sesuai dengan sistematika penulisan tugas akhir.

3.4 Rancangan Hierarki Perangkat Keras



Gambar 3.9. Diagram Blok Hierarki Top-Level Arsitektur Zynq PS dan PULPino PL

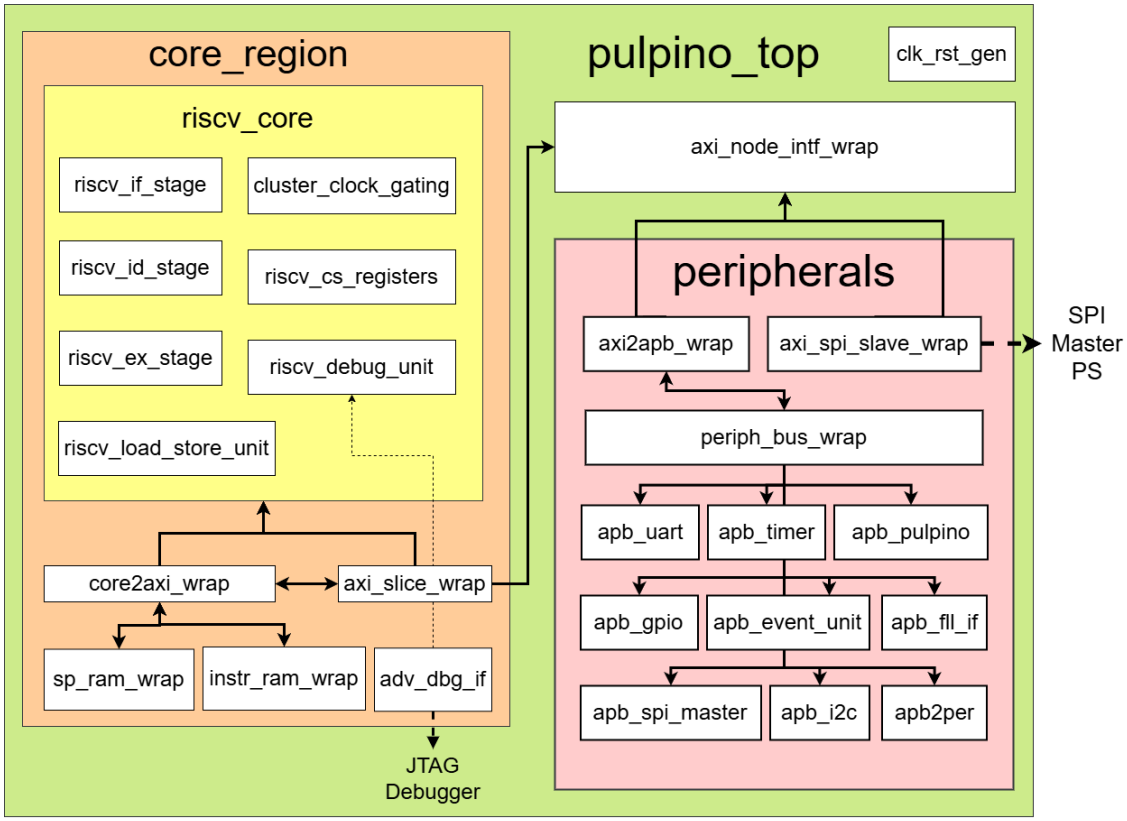
Implementasi PULPino pada FPGA Zybo tidak hanya melibatkan modul SoC PULPino, tetapi juga integrasi dengan subsistem *Processing System* (PS) milik Zynq-7000 seperti pada Gambar 3.9. Oleh karena itu, sebelum membahas struktur internal PULPino, perlu dipahami terlebih dahulu hierarki modul RTL yang menyusun keseluruhan desain FPGA.



Gambar 3.10. Modul tingkat teratas `pulpemu_top`

Pada penelitian ini, modul tingkat teratas (*top-level*) adalah `pulpemu_top`. Modul ini berfungsi sebagai integrator utama yang menghubungkan *Processing System* (PS) dengan *Programmable Logic* (PL), sebagaimana ditunjukkan pada Gambar 3.10. Di dalam modul ini terdapat tiga modul utama, yaitu `ps7_wrapper`, `pulpino_wrap`, dan `clk_rst_gen`. Modul `ps7_wrapper` merupakan hasil konfigurasi IP Zynq-7000 menggunakan Vivado IP Integrator yang menyediakan akses prosesor ARM Cortex-A9, antarmuka AXI, DDR, serta jalur EMIO menuju wilayah PL. Modul `pulpino_wrap` menjadi penghubung antara *Processing System* dan SoC PULPino yang diimplementasikan pada *Programmable Logic*. Selanjutnya, modul `clk_rst_gen` bertugas menghasilkan sinyal *clock* dan *reset* yang digunakan oleh seluruh logika pada FPGA.

Modul `pulpino_wrap` merupakan modul utama PL yang pada dasarnya hanya pembungkus modul `pulpino_top` untuk dipanggil oleh `pulpemu_top`. Modul `pulpino_top` merupakan *top-level* utama PULPino yang mengintegrasikan inti prosesor RISC-V (RI5CY), sistem memori, jaringan interkoneksi, serta seluruh periferal yang digunakan selama penelitian. Struktur modul tersebut ditunjukkan pada Gambar 3.11.



Gambar 3.11. Blok Diagram Verilog PULPino

Secara umum, `pulpino_top` tersusun atas empat berkas systemverilog utama, yaitu `core_region`, `axi_node_intf_wrap`, `peripherals`, dan modul *clock generator* `clk_rst_gen`. Keempat berkas tersebut saling berinteraksi untuk membentuk sebuah SoC yang lengkap, mulai dari proses eksekusi instruksi, akses memori, hingga komunikasi dengan periferal eksternal.Berikut merupakan fungsi masing-masing subsistem pada `pulpino_top`.

- **`core_region`** merupakan area komputasi utama yang berisi inti prosesor RI5CY, memori instruksi, memori data, serta *Advanced Debug Unit*. Seluruh proses *fetch*, *decode*, eksekusi instruksi, dan akses memori dilakukan pada subsistem ini.
- **`axi_node_intf_wrap`** merupakan interkoneksi utama yang menghubungkan inti prosesor, sistem memori, SPI Slave, serta periferal berkecepatan tinggi menggunakan protokol AXI.
- **`peripherals`** menampung seluruh periferal yang dipetakan ke ruang alamat prosesor, seperti UART, GPIO, SPI Master, I2C, Timer, Event Unit, dan SoC Control. Sebagian besar periferal tersebut diakses melalui bus APB yang dihubungkan menggunakan modul `axi2apb_wrap`.
- **`clk_rst_gen`** menghasilkan sinyal *clock* dan *reset* yang digunakan oleh seluruh modul pada SoC sehingga seluruh subsistem dapat bekerja secara sinkron.

3.4.1 Struktur Direktori dan Komposisi Proyek PULPino

Repositori PULPino terdiri atas beberapa direktori yang memisahkan desain perangkat keras, perangkat lunak, skrip otomatisasi, serta berkas pendukung implementasi. Struktur direktori ini memudahkan proses pengembangan karena setiap komponen sistem ditempatkan sesuai dengan fungsinya. Pada penelitian ini, hanya sebagian direktori yang digunakan secara langsung selama proses implementasi pada FPGA Zybo. Struktur direktori utama yang digunakan dalam penelitian beserta hubungan antar komponennya sebagai berikut.

```
pulpino/
|-- doc/
|-- rtl/
|-- ips/
|-- fpga/
|-- sw/
|-- ips_list.yml
|-- update-ips.py
|-- generate-scripts.py
\-- ...
```

Direktori `rtl` berisi modul-modul RTL yang membentuk keseluruhan SoC PULPino, termasuk modul tingkat atas (*top-level*), sistem interkoneksi, dan periferal. Direktori ini menjadi bagian utama yang digunakan pada proses sintesis menggunakan Xilinx Vivado.

```
rtl/
|-- axi2apb_wrap.sv
|-- axi_node_intf_wrap.sv
|-- axi_slice_wrap.sv
|-- axi_spi_slave_wrap.sv
|-- clk_rst_gen.sv
|-- core2axi_wrap.sv
|-- core_region.sv
|-- instr_ram_wrap.sv
|-- peripherals.sv
|-- periph_bus_wrap.sv
|-- pulpino_top.sv
|-- sp_ram_wrap.sv
|-- ...
```

```
|-- includes/
|   |-- apb_bus.sv
|   |-- axi_bus.sv
|   |-- debug_bus.sv
|   \-- ...
\-- components/
    |-- cluster_clock_gating.sv
    \-- ...
```

Direktori `ips` berisi kumpulan *Intellectual Property* (IP) yang digunakan oleh PULPino, seperti inti prosesor RI5CY, modul interkoneksi AXI dan APB, serta berbagai komponen pendukung lainnya. Sebagian besar modul pada direktori ini digunakan tanpa modifikasi sehingga tetap mempertahankan implementasi asli dari proyek PULPino.

```
ips/
|-- adv_dbg_if/
|-- axi/
|-- fpu/
|-- apb/
|   |-- apb_pulpino/
|   |-- apb_uart/
|   |-- apb_gpio/
|   |-- apb_i2c/
|   |-- apb_node/
|   |-- apb2per/
|   \-- ...
|-- riscv/
|   |-- riscv_core.sv
|   |-- riscv_if_stage.sv
|   |-- riscv_id_stage.sv
|   |-- riscv_ex_stage.sv
|   |-- riscv_load_store_unit.sv
|   \-- ...
\-- ...
```

Direktori `fpga` memuat berkas implementasi yang bersifat spesifik terhadap papan FPGA, seperti skrip *build* proyek Vivado (`Makefile` dan berkas tcl (.tcl)), konfigurasi untuk target perangkat Zybo, serta program `spiload` yang digunakan untuk memuat perangkat lunak ke memori instruksi PULPino.

```
fpga/
|-- ips/
|   |-- xilinx_clock_manager/
|   |-- xilinx_fp_fma/
|   |-- xilinx_mem_8192x32/
|   \-- ...
|-- pulpemu/
|   |-- tcl/
|   |   |-- run.tcl
|   |   |-- impl.tcl
|   |   \-- ps7_bd.tcl
|   \-- Makefile
|-- pulpino/
|   |-- tcl/
|   |   |-- run.tcl
|   |   \-- ...
|   \-- Makefile
|-- rtl/
|   |-- clk_rst_gen.sv
|   |-- pulpemu_top.sv
|   \-- pulpino_wrap.sv
|-- sw/
|   |-- apps/
|   |   |-- spiload/
|   |   \-- gpio_access/
|   |-- Makefile
|   |-- buildroot.config
|   |-- u-boot.config.zybo
|   \-- ...
\-- ...
```

Direktori `sw` digunakan untuk pengembangan perangkat lunak yang akan dijalankan oleh RI5CY. Program aplikasi ditulis dalam bahasa C, kemudian dikompilasi menggunakan *toolchain* RISC-V hingga menghasilkan berkas biner yang nantinya dimuat ke memori instruksi PULPino oleh program `spiload`.

```
sw/
|-- apps/
|   \-- helloworld/
```

```
|-- libs/
|-- ref/
|-- utils/
|    |-- crt0.riscv.S
|    |-- link.riscv.ld
|    \-- ...
|-- CMakeLists.txt
|-- cmake_configure.riscv.gcc.sh
\-- ...
```

3.5 Tahapan Modifikasi Repositori

Repositori PULPino yang digunakan pada penelitian ini merupakan proyek *open-source* yang dikembangkan untuk lingkungan perangkat lunak dan versi *toolchain* yang lebih lama. Oleh karena itu, implementasi pada FPGA Zybo dengan lingkungan pengembangan terkini tidak dapat dilakukan secara langsung. Diperlukan serangkaian penyesuaian terhadap struktur proyek, skrip kompilasi, maupun konfigurasi perangkat lunak agar keseluruhan sistem dapat dibangun dan dijalankan dengan benar.

Tahapan modifikasi dilakukan secara bertahap mulai dari proses inisialisasi repositori, penyesuaian dependensi, pembaruan skrip *build* proyek, hingga penyesuaian terhadap lingkungan pengembangan Vivado, Vitis, dan Linux. Berdasarkan hasil perombakan pada berkas-berkas proyek, proses modifikasi tersebut dikelompokkan ke dalam tiga lapisan sasaran utama:

3.5.1 Modifikasi Perangkat Keras FPGA

3.5.1.1 Penyesuaian Manajemen IP

Tahapan awal modifikasi dilakukan pada mekanisme pengelolaan *Intellectual Property* (IP). Berkas `ips_list.yml` diperbarui agar daftar versi IP yang digunakan sesuai dengan pustaka IP yang tersedia pada Vivado versi terbaru. Penyesuaian ini diperlukan agar seluruh modul pendukung dapat dikenali dan di-*build* kembali selama proses *build* proyek.

\ips_list.yml

```
67  adv_dbg_if:
68    commit: c440ae8 # Changed from pulpinov1 (pulpino's issue #399)
```

Gambar 3.12. Penyesuaian versi IP core pada `ips_list.yml`

Lalu, dilakukan penyesuaian pada berkas `ipstools/IPDatabase.py` untuk mengatasi perbedaan mekanisme *permission* antara Linux dan Windows. Perubahan ini menghilangkan pemanggilan perintah `chmod` yang tidak diperlukan pada lingkungan Windows sehingga proses inisialisasi repositori dapat berjalan tanpa menghasilkan galat. Ada dua pemanggilan yang perlu dihilangkan, yaitu pada baris 404 dan 444.

ipstools\IPDatabase.py

```
403  f.write(vcompile_script)
404  if hasattr(os, 'fchmod'):   # Added by Achmad Muhajir (22/500339/TK/54839)
405      os.fchmod(f.fileno(), os.fstat(f.fileno()).st_mode | stat.S_IXUSR | stat.S_IXGRP | stat.S_IXOTH)
```

Gambar 3.13. Penambahan blok *try-except* untuk *bypass* perintah `chmod` pada `IPDatabase.py`

3.5.1.2 Penyesuaian Skrip Otomasi FPGA

Proses *build* proyek FPGA dikendalikan melalui berkas `fpga/Makefile`. Pada penelitian ini dilakukan penyederhanaan alur kompilasi dengan memisahkan proses sintesis perangkat keras dan kompilasi perangkat lunak. Aturan `sw` dihapus dari aturan utama (`all`) karena kompilasi `sw` dilakukan pada lingkungan Linux menggunakan *toolchain* yang berbeda. Selain itu, beberapa perintah `clean` disesuaikan agar tidak dijalankan secara otomatis setiap kali proses *build* dimulai. Perintah `clean` hanya akan dijalankan jika dipanggil oleh pengguna.

fpga\Makefile

```
3   all: ips pulpino pulpemu
4
5   .PHONY: ips
6   ips: clk mem fp_fma
7
8   .PHONY: clk
9   clk: ips/xilinx_clock_manager/ip/xilinx_clock_manager.dcp
10
11  ips/xilinx_clock_manager/ip/xilinx_clock_manager.dcp:
12      make -C ips/xilinx_clock_manager/ all
```

Gambar 3.14. Modifikasi `fpga/Makefile` untuk menonaktifkan proses *clean* dan kompilasi perangkat lunak bawaan

3.5.1.3 Pembaruan Modul IP FPGA

Penyesuaian berikutnya dilakukan pada direktori `fpga/ips/` yang berisi skrip *build* berbagai IP FPGA, seperti *Clock Manager*, *Floating-Point Unit*, dan memori

BRAM.

Beberapa `Makefile` diperbarui agar keluaran proses *build* IP mengikuti struktur direktori yang digunakan oleh Vivado versi terbaru. Berkas hasil *build*, seperti `_stub.v`, `_stub.vhdl`, `.veo`, dan `.dcp`, diarahkan ke folder `.gen` sehingga dapat dikenali secara otomatis oleh proyek Vivado.

fpga\ips\xilinx_fp_fma\Makefile

```
4  all:
5      vivado -mode batch -source tcl/run.tcl
6      mkdir ip
7      cp ${PROJECT}.srcs/sources_1/ip/${PROJECT}/${PROJECT}.xci ./ip
8      cp ${PROJECT}.gen/sources_1/ip/${PROJECT}/${PROJECT}_stub.v ./ip
9      cp ${PROJECT}.gen/sources_1/ip/${PROJECT}/${PROJECT}_stub.vhdl ./ip
10     cp ${PROJECT}.gen/sources_1/ip/${PROJECT}/${PROJECT}.veo ./ip
11     cp ${PROJECT}.gen/sources_1/ip/${PROJECT}/${PROJECT}.dcp ./ip
```

Gambar 3.15. Pemindahan parameter target penyalinan fail *output* ke direktori `.gen`

Selain itu, parameter versi IP *Floating-Point* diperbarui menjadi versi 7.1 pada berkas `ips/xilinx_fp_fma/tcl/run.tcl` agar kompatibel dengan Vivado yang digunakan pada penelitian ini. Tanpa penyesuaian tersebut, proses *build* IP tidak dapat diselesaikan karena perbedaan versi.

fpga\ips\xilinx_fp_fma\tcl\run.tcl

```
14  set_property board $boardName [current_project]
15  # Changed to version 7.1 from 7.0
16  create_ip -name floating_point -vendor xilinx.com -library ip -version 7.1 -module_name xilinx_fp_fma
```

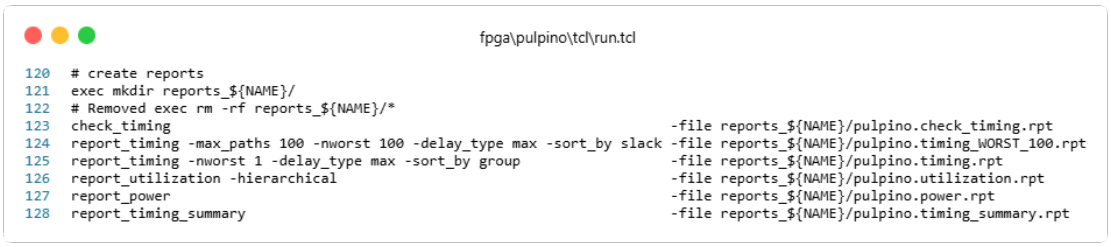
Gambar 3.16. Injeksi perbaikan rilis versi IP *Floating Point* ke rilis terbaru 7.1

3.5.1.4 Penyesuaian Proyek PULPino

Tahapan berikutnya dilakukan pada direktori `fpga/pulpino/` yang bertanggung jawab mem-*build* desain utama SoC PULPino.

Beberapa skrip Tcl diperbarui untuk menyesuaikan perintah yang sebelumnya hanya kompatibel dengan sistem operasi Linux. Perintah seperti `mkdir -p` dan `rm -rf` diganti dengan sintaks yang sesuai sehingga skrip dapat dijalankan langsung melalui Vivado pada lingkungan Windows.

Selain itu, skrip `run.tcl` diperluas dengan penambahan perintah pembangkitan laporan *timing* dan konsumsi daya. Laporan tersebut digunakan sebagai data evaluasi pada tahap analisis hasil implementasi di Bab IV.

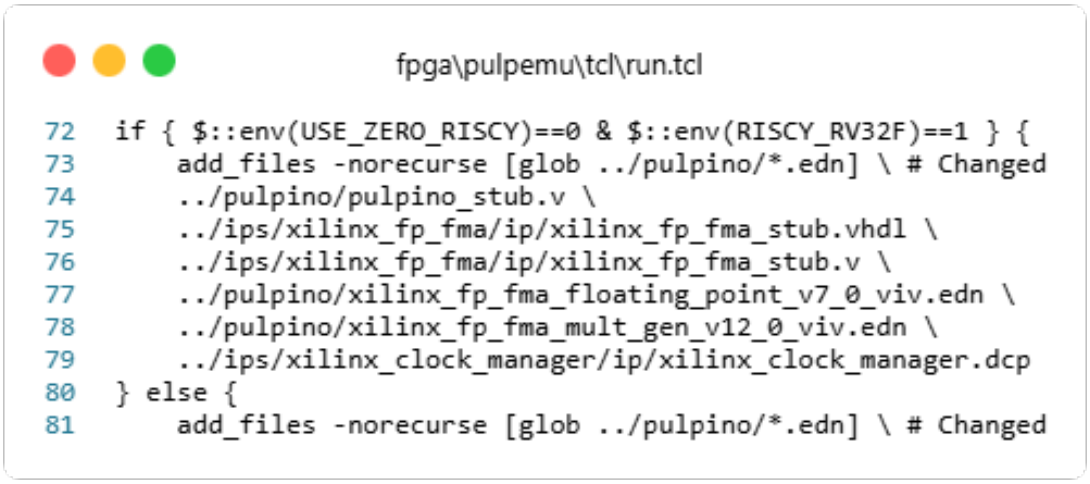


Gambar 3.17. Penyesuaian sintaks operasi dasar folder agar kompatibel pada lingkungan *host* Windows

3.5.1.5 Penyesuaian Proyek Emulasi Zybo

Tahapan selanjutnya dilakukan pada direktori `fpga/pulpemu/` yang berfungsi mengintegrasikan SoC PULPino dengan *Processing System* (PS) Zynq-7000. Pada berkas `run.tcl`, penambahan perintah `synth_checkpoint_mode` menjadi `None` untuk menghindari kegagalan sintesis yang muncul ketika Vivado me-*build Block Design*. Selain itu, mekanisme pencarian berkas menggunakan *wildcard* diperbaiki agar seluruh berkas hasil *build* IP dapat dimuat secara otomatis selama proses sintesis.





Gambar 3.18. Penambahan aturan *checkpoint* dan penyimpanan kendala otomatis pasca integrasi skema EDN

Selanjutnya, berkas `impl.tcl` diperbarui dengan menambahkan pemanggilan skrip `pre_opt.tcl` dan `pre_place.tcl` sebelum tahap optimasi dan *place-*

ment. Penambahan ini berfungsi untuk menghindari galat (*error*) benturan *Output Buffer* (OBUF) yang dapat timbul saat proses implementasi, sekaligus memastikan berkas *constraint* (`.xdc`) diterapkan secara konsisten sehingga pemetaan pin pada FPGA sesuai dengan rancangan.

fpga\pulpemu\tcl\impl.tcl

```
245  set_property STEPS.OPT_DESIGN.TCL.PRE [file normalize "tcl/pre_opt.tcl"] [get_runs impl_1]
246  set_property STEPS.PLACE_DESIGN.TCL.PRE [file normalize "tcl/pre_place.tcl"] [get_runs impl_1]
```

Gambar 3.19. Penyisipan instruksi pra-optimasi dan pra-penempatan perangkat

Penyesuaian juga dilakukan pada `ps7_bd.tcl` dengan melewati proses pemeriksaan versi Vivado. Hal ini bertujuan agar *Block Design Processing System* Zynq tetap dapat di-*build* meskipun menggunakan versi Vivado yang lebih baru.

fpga\pulpemu\tcl\ps7_bd.tcl

```
17  # if { [string first $scripts_vivado_version $current_vivado_version] == -1 } {
18  #   puts ""
19  #   puts "ERROR: This script was generated using Vivado <$scripts_vivado_version> ...
20  #
21  #   return 1
22  #}
```

Gambar 3.20. Klausa pelemahan penolakan ketidakcocokan versi rilis blok fungsional *Processing System*

3.5.2 Modifikasi Lingkungan Perangkat Lunak Linux (`fpga/sw/`)

3.5.2.1 Penyesuaian Sistem Kompilasi

Tahapan pertama dilakukan pada berkas `fpga/sw/Makefile` yang mengatur proses *build* seluruh komponen perangkat lunak. Penyesuaian dilakukan dengan menggganti metode pengambilan repositori dari protokol SSH (`git://github.com`) menjadi HTTPS sehingga proses pengunduhan dapat dilakukan tanpa autentikasi *SSH key*.

Selain itu, ditambahkan perintah `sed` untuk memperbaiki struktur `YYLTYPE` pada *Device Tree Compiler* (DTC). Penyesuaian ini diperlukan untuk mengatasi kesalahan kompilasi yang muncul akibat perubahan implementasi DTC pada distribusi Linux modern.



Gambar 3.21. Perbaikan orientasi repositori pengunduhan Linux dan penyisipan perintah `sed`

Pada skrip pembangkitan `uImage`, dilakukan penyesuaian pada deklarasi variabel lingkungan `PATH` dengan menambahkan tanda petik ("). Penyesuaian ini memastikan ekspansi variabel `PATH` diproses secara benar oleh shell pada lingkungan WSL sehingga proses kompilasi kernel dapat berjalan tanpa kesalahan.



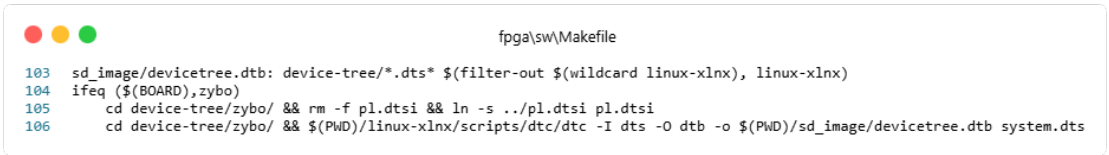
Gambar 3.22. Penambahan tanda petik pada deklarasi variabel `PATH`

Selanjutnya, diterapkan *patch* tambahan pada paket `fakeroot` untuk mengatasi ketidakcocokan dengan versi pustaka `glibc` yang digunakan pada Ubuntu.



Gambar 3.23. Integrasi *patch* pada `fakeroot`

Selain itu, mekanisme penyisipan berkas `pl.dtsi` juga disesuaikan agar deskripsi perangkat keras pada wilayah *Programmable Logic* dapat dimuat dengan benar selama proses *booting* Linux.



Gambar 3.24. Penyisipan berkas `pl.dtsi`

3.5.2.2 Penyesuaian Device Tree

Pada berkas `device-tree/zybo/system.dts` ditambahkan `serial1` untuk menyediakan antarmuka UART kedua yang digunakan sebagai jalur komunikasi dengan SoC PULPino. Dengan konfigurasi ini, sistem operasi Linux tetap menggunakan UART utama sebagai terminal sistem, sedangkan UART kedua dialokasikan secara khusus untuk komunikasi antara *Processing System* dan inti prosesor RI5CY.

fpga\sw\device-tree\zybo\system.dts

```
15      aliases {
16          ethernet0 = &ps7_ethernet_0;
17          i2c0 = &ps7_i2c_0;
18          serial0 = &ps7_uart_1;
19          serial1 = &ps7_uart_0;
20          spi0 = &ps7_qspi_0;
```

Gambar 3.25. Penambahan alias `serial1`

3.5.2.3 Penyesuaian Konfigurasi U-Boot

Pada berkas `u-boot.config.zybo`, dilakukan penyesuaian konfigurasi dengan menonaktifkan beberapa fitur verifikasi citra (*image verification*), seperti *FIT Signature* dan RSA.

fpga\sw\u-boot.config.zybo

```
218  CONFIG_FIT_VERBOSE=y
219  # CONFIG_FIT_SIGNATURE is not set # Changed
220  CONFIG_SYS_EXTRA_OPTIONS=""
```

fpga\sw\u-boot.config.zybo

```
443  # CONFIG_LIB_RAND is not set
444  # CONFIG_RSA is not set # Changed
445  # CONFIG_RSA_SOFTWARE_EXP is not set # Changed
```

Gambar 3.26. Penonaktifan fitur verifikasi citra pada U-Boot

3.5.2.4 Penyesuaian Buildroot

Konfigurasi Buildroot juga diperbarui untuk menyesuaikan versi *header kernel* yang digunakan oleh *toolchain*. Pada berkas `buildroot.config`, versi minimum

kernel headers ditetapkan menjadi 3.19 agar sesuai dengan konfigurasi *toolchain* eksternal yang digunakan dalam penelitian ini.

fpga\sw\buildroot.config

```
157  # BR2_TOOLCHAIN_EXTERNAL_HEADERS_4_0 is not set
158  BR2_TOOLCHAIN_EXTERNAL_HEADERS_3_19=y
159  # BR2_TOOLCHAIN_EXTERNAL_HEADERS_3_18 is not set
160  # BR2_TOOLCHAIN_EXTERNAL_HEADERS_3_17 is not set
161  # BR2_TOOLCHAIN_EXTERNAL_HEADERS_3_16 is not set
```

fpga\sw\buildroot.config

```
218  BR2_TOOLCHAIN_HEADERS_AT_LEAST_3_16=y
219  BR2_TOOLCHAIN_HEADERS_AT_LEAST_3_17=y
220  BR2_TOOLCHAIN_HEADERS_AT_LEAST_3_18=y
221  BR2_TOOLCHAIN_HEADERS_AT_LEAST_3_19=y
222  BR2_TOOLCHAIN_HEADERS_AT_LEAST="3.19"
```

Gambar 3.27. Deklarasi minimalisasi spesifikasi perangkat

3.5.2.5 Penyesuaian Program Spiload

Penyesuaian pada program `spiload` di folder `fpga/sw/apps/`, yaitu program yang digunakan untuk mengirimkan program RISC-V dari sisi *Processing System* menuju memori instruksi PULPino melalui antarmuka SPI.

Pada berkas `Makefile`, ditambahkan *compiler flag* `-DZYBO` pada pemanggilan CFLAG *board* Zybo agar program `spiload` dikompilasi menggunakan konfigurasi yang sesuai dengan perangkat keras Zybo.

fpga\sw\apps\spiload\Makefile

```
4  ifeq ($(BOARD),zybo)
5      CFLAGS += -DZYBO -static
6  endif
```

Gambar 3.28. Penambahan flag `-DZYBO` pada program `spiload`

Selain itu, pada berkas `main.c` dilakukan perubahan alamat awal eksekusi dari `0x00000000` menjadi `0x00000080`. Penyesuaian ini bertujuan agar alamat awal pro-

gram yang dimuat oleh `spiload` sesuai dengan konfigurasi memori instruksi PULPino pada FPGA Zybo. Setelah proses pemuatan selesai, inti prosesor RI5CY akan memulai eksekusi instruksi dari alamat tersebut sehingga program pengguna dapat dijalankan dengan benar.

fpga\sw\apps\spiload\main.c

```
425  // Start device and wait for timeout (if any)
426  set_boot_addr(0x00000080);
```

Gambar 3.29. Perubahan alamat awal eksekusi pada program `spiload`

3.5.3 Modifikasi Perangkat Lunak Bare-metal (`sw/`)

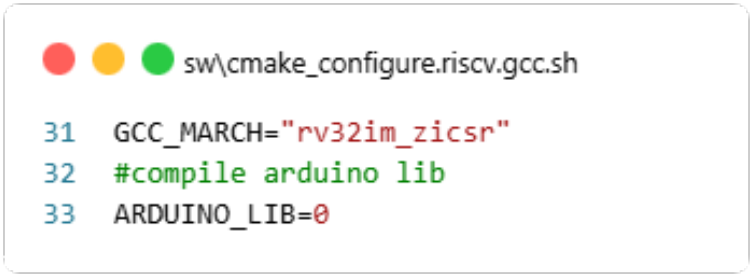
3.5.3.1 Penyesuaian Skrip Konfigurasi

Selanjutnya, pada berkas `cmake_configure.riscv.gcc.sh` dilakukan penyesuaian dengan mengubah *compiler prefix* menjadi `riscv64-unknown-elf-` agar sesuai dengan nama *toolchain* RISC-V yang digunakan. Selain itu, parameter kompilasi `-march` dan `-mabi` diperbarui menjadi `rv32im_zicsr` dan `ilp32` sehingga target kompilasi sesuai dengan konfigurasi ISA pada inti prosesor RI5CY. Perubahan tersebut ditunjukkan pada Gambar 3.30 dan Gambar 3.31.

sw\cmake_configure.riscv.gcc.sh

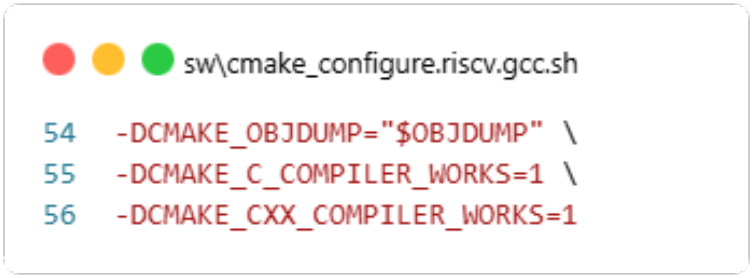
```
5   RISCV_PREFIX="riscv64-unknown-elf-"
6   OBJDUMP=`which ${RISCV_PREFIX}objdump`
7   OBJCOPY=`which ${RISCV_PREFIX}objcopy`
8   COMPILER=`which ${RISCV_PREFIX}gcc`
9   RANLIB=`which ${RISCV_PREFIX}ranlib`
10
11  VSIM=`which vsim`
12
13  TARGET_C_FLAGS="-O3 -mabi=ilp32 -march=rv32im_zicsr -g -D__riscv__ -nostdlib"
```

Gambar 3.30. Koreksi rujukan direktori target aplikasi *toolchain* pada skrip *configure*



Gambar 3.31. Injeksi spesifikasi target kompilasi mesin pada skrip *configure*

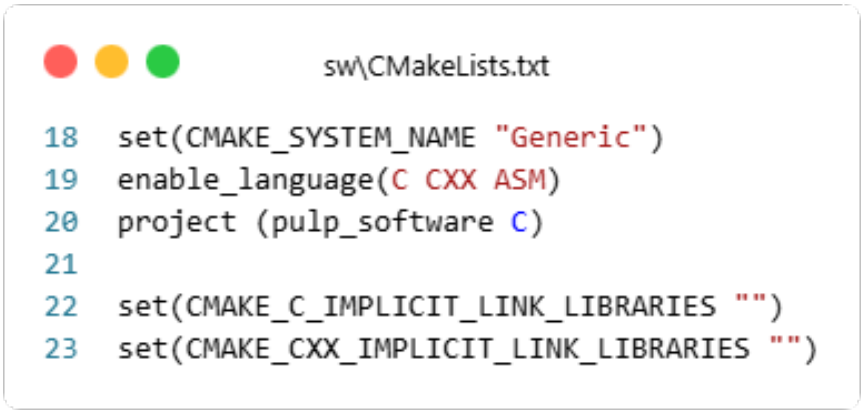
Terakhir, parameter ditambahkan pada perintah pemanggilan CMake, yaitu (*C & CXX compiler*) `DCMAKE_C_COMPILER_WORKS` dan `DCMAKE_CXX_COMPILER_WORKS`. Penambahan parameter ini digunakan untuk melewati proses pemeriksaan awal kompiler sehingga tahap konfigurasi proyek dapat berjalan dengan baik pada lingkungan WSL, sebagaimana ditunjukkan pada Gambar 3.32.



Gambar 3.32. Pembenahan argumen perintah pemanggilan utilitas CMake

3.5.3.2 Penyesuaian Konfigurasi `CMakeLists.txt`

Penyesuaian berikutnya dilakukan pada berkas `CMakeLists.txt`. Sistem target dideklarasikan sebagai `Generic` agar CMake mengenali bahwa aplikasi dibangun untuk lingkungan *bare-metal*. Selain itu, ditambahkan konfigurasi `CMAKE_C_IMPLICIT_LINK_LIB` dan `CMAKE_CXX_IMPLICIT_LINK_LIBRARIES` dengan nilai kosong. Hal ini mencegah CMake menautkan pustaka standar sistem operasi secara otomatis selama proses kompilasi.



Gambar 3.33. Deklarasi target sistem `Generic` untuk lingkungan *bare-metal*

Selanjutnya, parameter `CMAKE_C_FLAGS` disesuaikan dengan mengganti opsi `-m32` menjadi `-mabi=ilp32`. Selain itu, ditambahkan makro praprosesor `-D__riscv__` serta opsi `-nostdlib`. Penyesuaian ini memastikan proses kompilasi menggunakan ABI yang sesuai dengan arsitektur RISC-V dan tidak bergantung pada pustaka standar sistem operasi.



Gambar 3.34. Penonaktifan *implicit link libraries*

Pada tahap *linking*, parameter `CMAKE_EXE_LINKER_FLAGS` juga diperbarui dengan menambahkan opsi `-nostdlib`. Opsi ini memastikan proses *linking* tidak menggunakan pustaka standar sistem operasi sehingga berkas biner yang dihasilkan sepenuhnya mengikuti konfigurasi lingkungan *bare-metal*. Perubahan tersebut ditunjukkan pada Gambar 3.35.



Gambar 3.35. Penyesuaian compiler dan linker flags

3.5.3.3 Penyesuaian Berkas Inisialisasi dan *Linker*

Penyesuaian berikutnya dilakukan pada berkas `crt0.riscv.S`. Pemanggilan fungsi `__libc_init_array` dinonaktifkan karena sistem yang dikembangkan berjalan pada lingkungan *bare-metal* tanpa dukungan pustaka C standar. Dengan demikian,

proses inisialisasi yang bergantung pada `libc` tidak lagi diperlukan dan dapat dihilangkan untuk menghindari kegagalan saat program mulai dieksekusi.

sw\ref\crt0.riscv.S

```
76  /* call    __libc_init_array */ # Commented
```

Gambar 3.36. Penonaktifan pemanggilan inisialisasi awal `libc` pada blok *assembly*

Selanjutnya pada berkas `link.riscv.ld`, referensi terhadap pustaka `-lc`, `-lgloss`, dan `-lsupc++` dihapus. Penyesuaian ini memastikan proses *linking* hanya menggunakan komponen yang tersedia pada lingkungan *bare-metal*, sehingga program dapat dibangun tanpa ketergantungan terhadap pustaka sistem operasi.

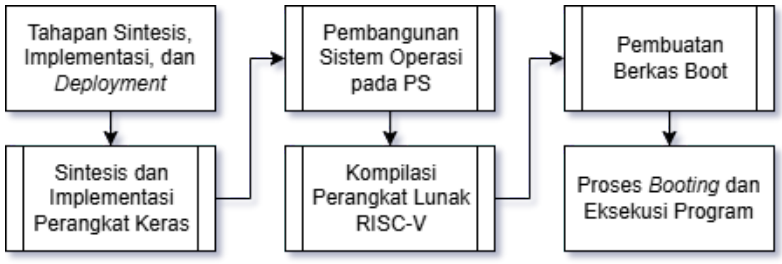
sw\ref\link.riscv.ld

```
5  GROUP( -lgcc ) # Deleted -lc -lgloss -lsupc++
```

Gambar 3.37. Penyingkiran parameter pustaka OS pendukung dari skrip *linker* RI5CY

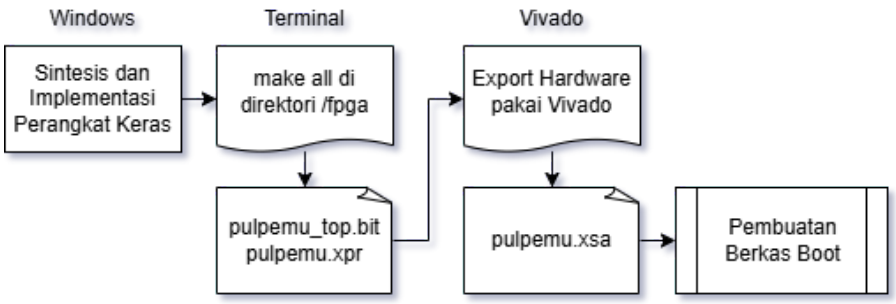
3.6 Tahapan Sintesis, Implementasi, dan *Deployment*

Setelah seluruh proses modifikasi terhadap repositori PULPino selesai dilakukan, tahapan berikutnya adalah membangun (*build*) keseluruhan sistem agar dapat diimplementasikan pada FPGA Zybo. Tahapan ini mencakup proses sintesis dan implementasi perangkat keras menggunakan Xilinx Vivado, pembangunan sistem operasi Linux pada sisi *host*, pembentukan berkas *boot*, kompilasi perangkat lunak RISC-V, hingga proses *deployment* ke perangkat keras. Alur tahapan ini ditunjukkan pada Gambar 3.38.



Gambar 3.38. Diagram alir tahapan sintesis, implementasi, dan *deployment*

3.6.1 Sintesis dan Implementasi Perangkat Keras



Gambar 3.39. Alur sintesis dan implementasi perangkat keras

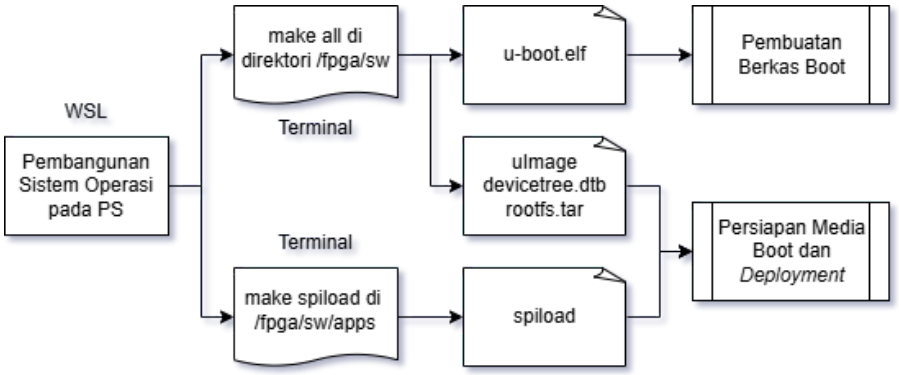
Proses pertama dilakukan pada direktori `fpga/` dengan menjalankan perintah `make all` melalui Command Prompt Windows. Perintah ini secara otomatis membangkitkan proyek Vivado berdasarkan seluruh berkas RTL, skrip TCL, serta konfigurasi perangkat keras yang telah dimodifikasi pada tahap sebelumnya. Setelah proses selesai, dihasilkan proyek Vivado (`pulpemu.xpr`) yang digunakan sebagai dasar proses sintesis dan implementasi.

Tahap sintesis (*synthesis*) merupakan proses penerjemahan seluruh deskripsi perangkat keras berbasis Verilog dan SystemVerilog menjadi representasi logika digital berupa *netlist*. Pada tahap ini Vivado melakukan pemeriksaan sintaks, analisis hierarki modul, optimasi logika, serta verifikasi keterhubungan antarmodul sesuai dengan rancangan SoC PULPino. Selain menghasilkan *netlist*, Vivado juga membangkitkan laporan awal mengenai utilisasi sumber daya FPGA, estimasi konsumsi daya, serta analisis awal terhadap karakteristik *timing* rancangan.

Setelah sintesis selesai, proses dilanjutkan ke tahap implementasi (*implementation*). Pada tahap ini, *netlist* hasil sintesis dipetakan ke sumber daya fisik FPGA melalui proses *placement* dan *routing*. Vivado menentukan lokasi setiap elemen logika pada FPGA serta membangun jalur interkoneksi antarblok sehingga seluruh rancangan dapat direalisasikan pada perangkat keras. Hasil implementasi kemudian digunakan untuk menghasilkan laporan utilisasi sumber daya, laporan *timing* akhir, serta berkas konfigurasi *bitstream* FPGA (`pulpemu_top.bit`)yang selanjutnya digunakan pada proses *deployment*.

Apabila seluruh kendala *timing* telah terpenuhi dan tidak ditemukan pelanggaran implementasi, rancangan perangkat keras diekspor dalam format *Xilinx Support Archive* (`.xsa`). Berkas ini memuat informasi konfigurasi perangkat keras yang diperlukan oleh Xilinx Vitis untuk membangun sistem *boot* pada PS.

3.6.2 Pembangunan Sistem Operasi pada *Processing System*



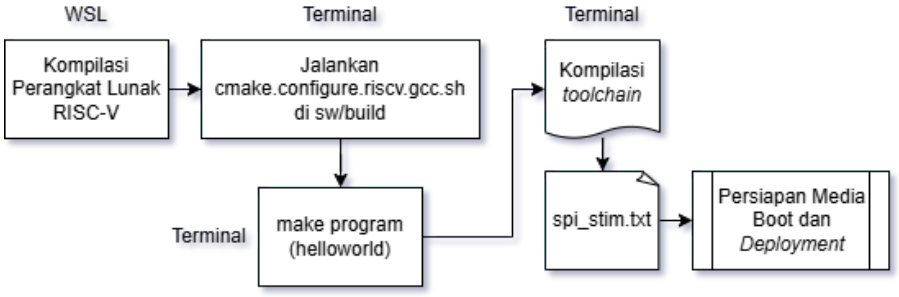
Gambar 3.40. Alur pembangunan sistem operasi pada *Processing System*

Setelah rancangan perangkat keras berhasil dibangun, proses dilanjutkan pada lingkungan Windows Subsystem for Linux (WSL). Tahap ini bertujuan membangun seluruh komponen perangkat lunak yang dijalankan oleh prosesor ARM pada PS. Proses dilakukan pada direktori `fpga/sw` dengan menjalankan perintah `make all`.

Eksekusi perintah tersebut menghasilkan beberapa berkas penting, yaitu `u-boot.elf` sebagai *second stage bootloader*, `uImage` sebagai kernel Linux ARM, `rootfs.tar` sebagai *root filesystem*, serta `devicetree.dtb` yang berisi deskripsi konfigurasi perangkat keras. Keempat berkas tersebut digunakan selama proses penyalaan sistem sehingga Linux dapat mengenali konfigurasi perangkat keras yang telah diimplementasikan pada FPGA.

Selain membangun sistem operasi, pada tahap ini juga dilakukan kompilasi program `spiload` yang berada pada direktori `fpga/sw/apps`. Program ini berfungsi sebagai utilitas komunikasi antara PS dan SoC PULPino yang berada pada wilayah PL. Utilitas tersebut digunakan untuk mentransfer program RISC-V ke memori instruksi PULPino setelah sistem operasi Linux berhasil dijalankan.

3.6.3 Kompilasi Perangkat Lunak RISC-V



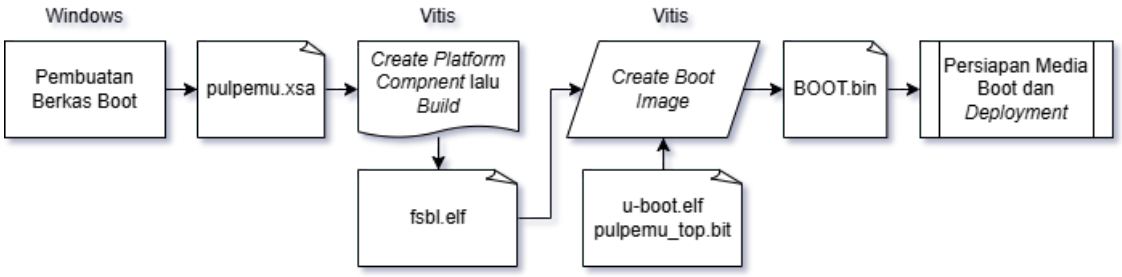
Gambar 3.41. Alur kompilasi perangkat lunak *bare-metal* RISC-V

Selain membangun perangkat keras dan sistem operasi host, penelitian ini juga memerlukan proses kompilasi perangkat lunak *bare-metal* yang akan dijalankan oleh inti prosesor RI5CY. Proses kompilasi dilakukan pada lingkungan WSL menggunakan *toolchain* RISC-V GCC yang telah dikonfigurasi pada tahap adaptasi repositori.

Tahap kompilasi diawali dengan membuat direktori `build` sebagai lokasi hasil kompilasi, kemudian berpindah ke direktori tersebut dan menjalankan skrip konfigurasi `cmake_configure.riscv.gcc.sh`. Skrip ini membangkitkan berkas-berkas konfigurasi CMake yang diperlukan untuk proses kompilasi menggunakan *toolchain* RISC-V GCC. Selanjutnya, proses kompilasi aplikasi dilakukan menggunakan perintah `make` terhadap aplikasi yang diinginkan (misalnya `make helloworld`) sehingga dihasilkan berkas *Executable and Linkable Format* (ELF) dari program `helloworld.c`.

Selanjutnya berkas ELF dikonversi menggunakan utilitas `objcopy` menjadi berkas stimulus memori `spi_stim.txt`. Berkas ini hanya berisi instruksi mesin dalam format heksadesimal sehingga dapat dimuat langsung ke dalam *Instruction RAM* PULPino melalui program `spiload`. Berkas `spi_stim.txt` kemudian disalin ke partisi FAT32 pada kartu microSD sebagai bagian dari proses *deployment*.

3.6.4 Pembuatan Berkas Boot

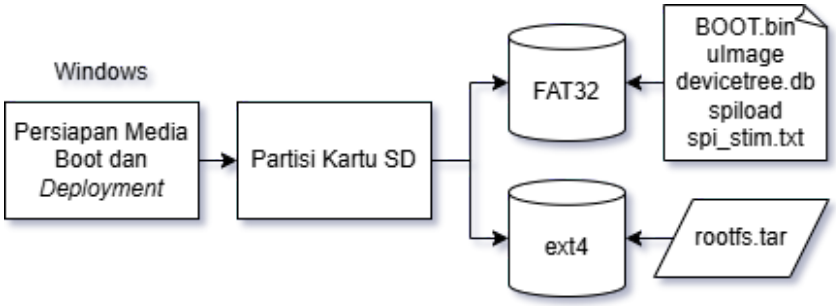


Gambar 3.42. Alur pembuatan berkas *boot*

Berkas `.xsa` hasil implementasi kemudian diimpor ke dalam Xilinx Vitis untuk membangun platform perangkat keras. Berdasarkan platform tersebut, Vitis menghasilkan *First Stage Bootloader* (FSBL) dalam bentuk berkas `fsbl.elf`. FSBL merupakan perangkat lunak pertama yang akan dijalankan setelah BootROM selesai melakukan proses inisialisasi awal pada SoC Zynq-7000.

Selanjutnya dibuat sebuah *Boot Image* dengan menggabungkan tiga komponen utama, yaitu `fsbl.elf`, berkas *bitstream* FPGA (`pulpemu_top.bit`), dan `u-boot.elf`. Ketiga komponen tersebut dikemas menjadi satu berkas `BOOT.BIN` menggunakan utilitas *Boot Image Format* (BIF). Berkas `BOOT.BIN` inilah yang nantinya dibaca oleh BootROM ketika papan Zybo dinyalakan.

3.6.5 Persiapan Media *Boot* dan *Deployment*



Gambar 3.43. Alur persiapan media *boot* dan *deployment*

Setelah seluruh komponen perangkat keras dan perangkat lunak berhasil dibangun, tahap berikutnya adalah menyiapkan media *boot* yang akan digunakan oleh Zybo. Pada penelitian ini digunakan kartu microSD yang dipartisi menjadi dua bagian. Partisi pertama menggunakan sistem berkas FAT32 sebagai partisi *boot*, sedangkan partisi kedua menggunakan sistem berkas `ext4` untuk *root filesystem* (`rootfs.tar`)Linux.

Partisi FAT32 diisi dengan berkas `BOOT.BIN`, `uImage`, `devicetree.dtb`, serta berkas program RISC-V (`spi_stim.txt`). Berkas `BOOT.BIN` berisi *First Stage Bootloader* (FSBL), *bitstream* FPGA, dan U-Boot, sedangkan `uImage` merupakan kernel Linux ARM yang akan dijalankan setelah proses *bootloader* selesai. Sementara itu, *root filesystem* (`rootfs.tar`) hasil pembangunan Buildroot akan diekstrak ke partisi `ext4`.

Setelah seluruh berkas tersebut ditempatkan pada partisi yang sesuai, kartu microSD dipasang pada papan Zybo yang telah dikonfigurasi pada mode SD *Boot*. Seluruh proses *deployment* selanjutnya dilakukan secara otomatis melalui mekanisme *boot* bawaan SoC Zynq-7000 hingga sistem operasi Linux berhasil dijalankan.

3.6.6 Proses *Booting* dan Eksekusi Program

Ketika papan Zybo dinyalakan, prosesor ARM pada PS terlebih dahulu mengeksekusi BootROM internal Zynq-7000. BootROM membaca berkas `BOOT.BIN` dari partisi FAT32 kartu microSD dan menjalankan komponen pertama di dalamnya, yaitu FSBL. Selanjutnya FSBL menginisialisasi subsistem PS, mengonfigurasi wilayah PL menggunakan *bitstream* FPGA yang juga terdapat di dalam `BOOT.BIN`, kemudian menyerahkan kendali kepada U-Boot yang merupakan komponen terakhir pada berkas tersebut.

Setelah memperoleh kendali, U-Boot memuat berkas `uImage` dan `devicetree.dtb` dari partisi FAT32 ke memori DDR, kemudian menjalankan kernel Linux. Selanjutnya kernel melakukan inisialisasi *root filesystem* yang tersimpan pada partisi `ext4` sehingga sistem operasi Linux dapat berjalan secara penuh.

Setelah sistem operasi Linux berhasil dijalankan, pengguna dapat mengakses terminal melalui antarmuka serial UART menggunakan perangkat lunak komunikasi serial (misalnya Tera Term atau PuTTY) dengan konfigurasi *baud-rate* 115200 bps. Selanjutnya partisi FAT32 pada kartu microSD di-*mount* ke dalam sistem Linux sehingga berkas `spi_stim.txt` dapat diakses oleh sistem operasi.

Program aplikasi RISC-V yang sebelumnya telah dikompilasi menjadi berkas instruksi mesin `spi_stim.txt`. Berkas ini selanjutnya dimuat ke dalam memori instruksi PULPino menggunakan program `spiload`. Program tersebut berkomunikasi dengan modul SPI Slave yang berada pada wilayah PL untuk mentransfer isi berkas `spi_stim.txt` menuju *Instruction RAM* PULPino.

Selama proses transfer berlangsung, program `spiload` menahan inti prosesor RI5CY pada kondisi *reset* sehingga tidak terjadi eksekusi instruksi sebelum seluruh isi memori selesai ditulis. Setelah seluruh data berhasil dipindahkan, sinyal *reset* dilepaskan sehingga inti prosesor RI5CY mulai melakukan proses *instruction fetch* dari alamat awal memori instruksi dan menjalankan program yang telah diunggah.

3.7 Skenario dan Rancangan Pengujian

Pengujian dilakukan secara bertahap pada perangkat keras Digilent Zybo (Zynq-7010) untuk memverifikasi keberhasilan implementasi SoC PULPino pada FPGA. Seluruh pengujian menggunakan satu program aplikasi `helloworld.c` yang dikembangkan secara bertahap sehingga mampu menguji komunikasi serial, antarmuka GPIO, serta integrasi periferal eksternal. Rancangan skenario pengujian ditunjukkan pada Tabel 3.4.

Tabel 3.4. Skenario dan rancangan pengujian perangkat keras

Jenis Pengujian	**Objek yang Diuji**	**Metode Pengujian**
Pengujian sistem	Booting, *SPI Loader*, dan UART	Sistem dijalankan menggunakan utilitas `spiload` melalui perintah `spiload -t100000 pulpino_test.txt`. Proses transfer program ke memori instruksi PULPino, pelepasan *reset* inti RI5CY, serta keluaran `printf()` diamati melalui terminal serial UART dengan *baud rate* 115200 bps.
Pengujian antarmuka digital	GPIO bawaan dan GPIO PMOD	Pengujian GPIO bawaan dilakukan menggunakan *slide switch*, *push button*, dan LED pada Zybo. Selanjutnya, GPIO eksternal diuji melalui port PMOD JB Pin 7–10 yang dipetakan ke pin FPGA Y18, Y19, W18, dan W19. Setiap pin dihubungkan ke GND secara bergantian menggunakan kabel *jumper* untuk menghasilkan perubahan level logika HIGH menjadi LOW, kemudian statusnya diamati melalui terminal UART.
Pengujian integrasi periferal	Sensor PIR dan OLED I^2C	Sensor PIR diberi catu daya eksternal 12 VDC, dengan GND dihubungkan ke GND Zybo dan pin OUT dihubungkan ke PMOD JC Pin 2 (FPGA pin W15/GPIO20). Modul OLED GM009605v4 dihubungkan melalui antarmuka I2C pada PMOD JD menggunakan jalur SDA di JD1 (FPGA pin T14) dan SCL di JD2 (FPGA pin T15). Program menampilkan perubahan status *switch*, *button*, dan sensor PIR pada layar OLED secara *real-time*.

Hasil dari masing-masing skenario pengujian tersebut disajikan dan dianalisis pada Bab IV.

3.8 Masalah dan Keterbatasan Penelitian

Selama proses replikasi dan implementasi SoC PULPino pada FPGA Zybo, terdapat beberapa kendala teknis serta keterbatasan penelitian. Kendala tersebut diuraikan sebagai berikut.

1. **Kompatibilitas Repositori terhadap Lingkungan Pengembangan Modern**
   Repositori PULPino sudah tidak dapat langsung dikompilasi menggunakan *toolchain* dan lingkungan pengembangan saat ini sehingga diperlukan modifikasi pada sistem *build*, konfigurasi proyek, serta perangkat lunak pendukung.
2. **Perbedaan Lingkungan Pengembangan**
   Repositori PULPino dikembangkan dengan lingkungan pengembangan berbasis Linux, sedangkan penelitian ini menggunakan sistem operasi Windows sebagai lingkungan kerja utama. Oleh karena itu, sebagian proses kompilasi dan pembangunan sistem dipindahkan ke Windows Subsystem for Linux (WSL) agar sesuai dengan kebutuhan repositori tanpa memerlukan instalasi Linux secara terpisah. Selain menyediakan lingkungan yang kompatibel dengan skrip *build* PULPino, WSL juga menghindari permasalahan pada sistem berkas Windows, seperti perbedaan mekanisme *case-sensitive* atau masalah format CRLF dan LF, yang dapat menyebabkan kegagalan kompilasi pada beberapa berkas di dalam repositori.
3. **Keterbatasan Evaluasi Penelitian**
   Evaluasi pada penelitian ini difokuskan pada keberhasilan implementasi perangkat keras dan validasi fungsional periferal pada FPGA Zybo. Analisis yang dilakukan mencakup laporan sintesis, utilisasi sumber daya FPGA, analisis *timing*, serta pengujian operasional melalui UART, GPIO, PMOD, sensor PIR, dan OLED. Penelitian ini belum mencakup evaluasi performa komputasi seperti pengukuran *benchmark*, *instruction throughput*, konsumsi daya aktual, maupun optimasi arsitektur inti prosesor RI5CY.

3.9 Penggunaan Kecerdasan Buatan dalam Penulisan Naskah

Selama proses penyusunan naskah skripsi ini, penulis memanfaatkan perangkat kecerdasan buatan (*Artificial Intelligence*/AI) berbasis *Large Language Model* (LLM), yaitu Google Gemini, OpenAI ChatGPT, dan Anthropic Claude, sebagai alat bantu dalam proses penulisan. Pemanfaatan AI dibatasi pada kegiatan *copyediting*, penyuntingan tata bahasa, penyempurnaan struktur kalimat agar sesuai dengan kaidah penulisan ilmiah, penyusunan ulang redaksi, serta membantu penelusuran informasi awal dan padanan istilah teknis.

AI tidak digunakan untuk menghasilkan data penelitian, melakukan pengambilan data, menyusun hasil eksperimen, maupun menarik kesimpulan penelitian. Seluruh pro-

ses perancangan, implementasi, pengujian, analisis hasil, interpretasi data, serta substansi ilmiah yang disajikan dalam skripsi ini merupakan hasil pekerjaan penulis dan menjadi tanggung jawab penuh penulis.

BAB IV
HASIL DAN PEMBAHASAN

Bab ini menyajikan tahapan replikasi dengan modifikasi, hasil sintesis dan implementasi, analisis performa, dan validasi operasional dari implementasi PULPino berbasis *core* RI5CY pada FPGA Digilent Zybo *development board* (Zynq-7010). Pembahasan pada bab ini dirancang untuk menjawab tujuan penelitian secara komprehensif, diawali dengan pembahasan alur modifikasi folder dan berkas proyek secara bertahap, laporan penggunaan sumber daya fisik, parameter *timing*, hingga pembuktian operasional sistem pada papan pengembangan FPGA.

4.1 Keberhasilan Sintesis dan *Deployment*

Repositori asli PULPino dapat diakses secara terbuka melalui tautan `https://github.com/pulp-platform/pulpino` [28]. Repositori tersebut menyediakan seluruh berkas RTL dan konfigurasi dasar yang menjadi titik awal sebelum tahapan modifikasi dilakukan. Proses adaptasi repositori PULPino yang telah dijelaskan pada Bab III berhasil menghasilkan rancangan perangkat keras yang dapat disintesis, diimplementasikan, dan dijalankan pada FPGA Zybo. Seluruh penyesuaian yang dilakukan terhadap struktur repositori, skrip kompilasi, konfigurasi *toolchain*, serta lingkungan pengembangan memungkinkan proses *build* berlangsung tanpa menghasilkan *fatal error*. Tahapan tersebut menghasilkan berkas *bitstream* FPGA (`pulpemu_top.bit`) beserta seluruh komponen perangkat lunak pendukung yang diperlukan untuk proses *booting* sistem.

Tahap *deployment* juga berhasil dilakukan menggunakan media kartu microSD yang memuat berkas `BOOT.BIN`, *device tree*, kernel Linux, dan *root filesystem*. Setelah sistem dinyalakan dalam mode SD Boot, Zybo berhasil menjalankan proses *booting* hingga sistem operasi Linux aktif dan siap berkomunikasi dengan perangkat keras PULPino melalui utilitas `spiload`. Keberhasilan sintesis dan *deployment* tersebut menjadi dasar untuk melakukan evaluasi terhadap hasil implementasi, baik melalui analisis laporan sintesis maupun pengujian operasional perangkat keras.

4.2 Evaluasi Laporan Hasil Sintesis

Keberhasilan proses sintesis dan implementasi menghasilkan sejumlah laporan yang secara otomatis dibangkitkan oleh perangkat lunak AMD Vivado 2025.1. Laporan tersebut digunakan untuk mengevaluasi karakteristik implementasi SoC PULPino pada FPGA Zybo (*part number* `xc7z010clg400-1`) berdasarkan tiga aspek utama, yaitu utilisasi sumber daya FPGA, analisis *timing*, dan estimasi konsumsi daya.

4.2.1 Analisis Penggunaan Sumber Daya (*Resource Utilization*)

Laporan *resource utilization* digunakan untuk mengevaluasi pemanfaatan sumber daya FPGA setelah proses sintesis dan implementasi selesai dilakukan. Laporan ini menunjukkan jumlah elemen logika yang digunakan oleh rancangan, meliputi *Look-Up Table* (LUT), *Flip-Flop* (FF), *Block RAM* (BRAM), dan *Digital Signal Processing* (DSP). Selain memberikan informasi mengenai tingkat utilisasi FPGA, laporan ini juga menunjukkan distribusi penggunaan sumber daya pada setiap modul penyusun SoC PULPino.

Tabel 4.1. Utilisasi sumber daya berdasarkan hierarki modul SoC PULPino

Hierarki Modul	**LUT**	**FF**	**BRAM**	**DSP**
AXI Interconnect	601	297	0	0
Core Region	7929	4452	16	11
RI5CY Core	5411	2576	0	11
Instruction Fetch (IF)	1213	321	0	0
Instruction Decode (ID)	2803	1577	0	0
Execute Stage (EX)	297	120	0	11
Advanced Debug Unit	613	481	0	0
Core2AXI Slice	516	286	0	0
Instruction Memory AXI IF	435	543	0	0
Instruction Memory	104	18	8	0
Data Memory AXI IF	559	533	0	0
Data Memory*	104	18	8	0
Peripheral	4866	5170	0	0
AXI-to-APB Bridge	2122	337	0	0
GPIO	167	513	0	0
UART	670	1485	0	0
SPI Master	590	870	0	0
SPI Slave	682	913	0	0
I2C	151	143	0	0
SoC PULPino (Top Level)	13400	9924	16	11
Kapasitas FPGA Zybo Z-7010	17600	35200	60	80
Utilisasi (%)	76,14	28,19	26,67	13,75

Tabel 4.1 menunjukkan bahwa implementasi SoC PULPino menggunakan seba-

nyak 13.400 LUT, 9.924 FF, 16 blok BRAM, dan 11 blok DSP. Dibandingkan dengan kapasitas FPGA Zybo Z-7010, penggunaan tersebut setara dengan 76,14% LUT, 28,19% FF, 26,67% BRAM, dan 13,75% DSP. Hasil tersebut menunjukkan bahwa rancangan masih dapat diimplementasikan pada FPGA Zybo tanpa melebihi kapasitas sumber daya yang tersedia

Sebagian besar sumber daya FPGA digunakan pada *Core Region*, yaitu sebanyak 7.929 LUT, 4.452 FF, 16 blok BRAM, dan seluruh 11 blok DSP yang digunakan pada rancangan. Di dalam *Core Region*, inti prosesor RI5CY merupakan modul dengan penggunaan sumber daya terbesar, yaitu 5.411 LUT dan 2.576 FF. Jika ditinjau berdasarkan tahapan *pipeline*, modul *Instruction Decode* (ID) memiliki penggunaan LUT paling tinggi, yaitu 2.803 LUT atau sekitar 51,8% dari total LUT pada RI5CY. Besarnya penggunaan logika pada tahap ini disebabkan oleh proses dekode instruksi, pembangkitan sinyal kendali, pembacaan register, pembentukan nilai *immediate*, serta pengaturan jalur data sebelum instruksi diteruskan ke tahap eksekusi. Sebaliknya, tahap *Instruction Fetch* (IF) menggunakan 1.213 LUT, sedangkan tahap *Execute* (EX) hanya menggunakan 297 LUT.

Meskipun penggunaan LUT pada tahap eksekusi relatif kecil, seluruh 11 blok DSP yang digunakan pada rancangan berasal dari modul `riscv_mult` yang berada di dalam `mult_i` pada `ex_stage_i`. Modul ini mengimplementasikan operasi perkalian yang berasal dari ekstensi instruksi RV32M sehingga Vivado memanfaatkan blok DSP48E1 bawaan FPGA sebagai unit aritmetika khusus. Pendekatan ini mengurangi kebutuhan logika kombinasional pada LUT sekaligus meningkatkan efisiensi implementasi operasi perkalian.

Implementasi memori menggunakan dua buah IP Xilinx `xilinx_mem_8192x32` yang masing-masing berfungsi sebagai memori instruksi dan memori data dengan kapasitas 8192 word berukuran 32 bit atau setara dengan 32 KB. Selama proses sintesis, kedua IP tersebut dipetakan ke blok BRAM (`RAMB36E1`) sehingga masing-masing menggunakan delapan blok BRAM atau total 16 blok BRAM. Pada laporan sintesis, hierarki *Instruction Memory* masih ditampilkan secara eksplisit, sedangkan hierarki *Data Memory* mengalami optimasi sehingga tidak muncul sebagai modul terpisah. Oleh karena itu, penggunaan BRAM pada memori data dianalisis berdasarkan struktur RTL yang diimplementasikan serta hasil sintesis keseluruhan.

`Peripheral` menggunakan 4.866 LUT dan 5.170 FF. Di antara seluruh periferal, modul `axi2apb_i` adalah pengguna LUT terbesar, yaitu sebanyak 2.122 LUT. Hal ini disebabkan modul tersebut berfungsi sebagai jembatan komunikasi antara bus AXI dan APB sehingga harus menangani proses dekode alamat, pengendalian transaksi, serta mekanisme *handshaking* pada kedua protokol bus. Sementara itu, modul UART, SPI Master, SPI Slave, GPIO, dan I2C menggunakan sumber daya yang relatif lebih kecil karena hanya mengimplementasikan fungsi antarmuka dengan perangkat eksternal.

Secara keseluruhan, hasil sintesis menunjukkan bahwa implementasi SoC PULPino masih berada dalam batas kapasitas FPGA Zybo Z-7010. Penggunaan sumber daya didominasi oleh inti prosesor RI5CY beserta sistem interkoneksinya, sedangkan penggunaan BRAM dan DSP dipengaruhi oleh implementasi memori internal serta unit perkalian pada tahap eksekusi. Hasil ini menunjukkan bahwa rancangan masih memiliki ruang sumber daya yang cukup untuk pengembangan lebih lanjut, seperti penambahan periferal maupun implementasi ekstensi arsitektur RISC-V lainnya.

Perbedaan tingkat utilisasi LUT dan FF juga menunjukkan bahwa implementasi SoC PULPino lebih didominasi oleh logika kombinasional dibandingkan sekuensial. Kondisi ini sesuai dengan karakteristik arsitektur prosesor dan sistem interkoneksi yang memerlukan proses dekode, pengendalian, serta pengolahan data melalui jaringan logika yang relatif kompleks.

4.2.2 Analisis *Timing* dan Frekuensi Kerja Maksimum

Analisis *timing* dilakukan untuk memastikan bahwa implementasi SoC PULPino memenuhi seluruh batasan waktu (*timing constraint*) yang telah ditentukan selama proses sintesis dan implementasi. Evaluasi dilakukan berdasarkan laporan *Clock Summary* dan *Design Timing Summary* yang dihasilkan oleh Xilinx Vivado setelah proses *placement* dan *routing* selesai. Parameter utama yang dianalisis meliputi *Worst Negative Slack* (WNS), *Total Negative Slack* (TNS), *Worst Hold Slack* (WHS), *Total Hold Slack* (THS), *Worst Pulse Width Slack* (WPWS), dan *Total Pulse Width Slack* (TPWS) pada domain *clock* utama (`clk`).

Tabel 4.2. Ringkasan hasil analisis *timing* pada domain *clock* `clk`

Parameter	**Nilai**	**Status**
Requirement clock	50,000 ns	–
WNS	+26,587 ns	MET
TNS	0 ns	MET
WHS	+0,077 ns	MET
THS	0 ns	MET
WPWS	+24,020 ns	MET
TPWS	0 ns	MET
Setup Failing Endpoints	0 dari 17.551 endpoint	PASS
Hold Failing Endpoints	0 dari 17.551 endpoint	PASS
Pulse Width Failing Endpoints	0 dari 9.025 endpoint	PASS

Berdasarkan Tabel 4.2, seluruh parameter *timing* menunjukkan status *MET* (*timing constraints met*), yang berarti seluruh lintasan logika pada domain *clock* utama berhasil memenuhi batasan waktu yang ditentukan. Nilai WNS sebesar +26,587 ns menunjukkan bahwa lintasan kritis masih memiliki margin waktu sebesar 26,587 ns terhadap batas *setup timing*. Selain itu, nilai TNS sebesar 0 ns dan tidak ditemukannya *failing endpoint* menunjukkan bahwa seluruh lintasan berhasil memenuhi persyaratan *setup timing*.

Hasil analisis *hold timing* juga menunjukkan bahwa implementasi telah memenuhi seluruh persyaratan waktu penahanan data. Nilai WHS sebesar +0,077 ns menunjukkan bahwa lintasan dengan margin *hold* terkecil masih berada di atas batas minimum, sedangkan THS sebesar 0 ns mengindikasikan tidak terdapat pelanggaran *hold timing* pada seluruh endpoint yang dianalisis.

Selain *setup* dan *hold timing*, Vivado juga melakukan analisis terhadap lebar pulsa (*pulse width*) sinyal *clock*. Nilai WPWS sebesar +24,020 ns dan TPWS sebesar 0 ns menunjukkan bahwa seluruh sinyal *clock* memenuhi persyaratan lebar pulsa minimum yang dibutuhkan oleh elemen sekuensial FPGA sehingga tidak ditemukan pelanggaran *pulse width* selama implementasi.

Frekuensi kerja maksimum teoritis dari rancangan dapat dihitung berdasarkan nilai periode *clock* dan WNS menggunakan Persamaan 4-1 [18].

$$F_{\max}(MHz) = \frac{1000}{T_{\text{clock}} - \text{WNS}} \tag{4-1}$$

Dengan periode *clock* sebesar 50 ns dan nilai WNS sebesar 26,587 ns, maka diperoleh

$$F_{\max} = \frac{1000}{50\text{ns} - 26,587\text{ns}} = \frac{1}{23,413\text{ ns}} \approx 42,71\text{ MHz} \tag{4-2}$$

Hasil tersebut menunjukkan bahwa lintasan kritis pada implementasi SoC PULPino secara teoritis masih mampu beroperasi hingga frekuensi sekitar 42,71 MHz. Nilai tersebut lebih tinggi daripada frekuensi target sebesar 20 MHz yang digunakan sebagai *timing constraint*, sehingga implementasi masih memiliki margin *timing* yang memadai. Selain itu, seluruh 17.551 endpoint pada analisis *setup* dan *hold timing* serta 9.025 endpoint pada analisis *pulse width* berhasil memenuhi seluruh batasan waktu tanpa ditemukan *failing endpoint*. Hasil ini menunjukkan bahwa rancangan SoC PULPino telah memenuhi seluruh persyaratan *timing* yang ditetapkan selama proses implementasi pada FPGA Zybo Z-7010.

4.3 Hasil Validasi Pengujian Operasional

Validasi operasional dilakukan pada perangkat keras Zybo *development board* untuk memastikan seluruh sistem PULPino yang telah diimplementasikan pada PL dapat beroperasi sesuai dengan rancangan. Pengujian dilakukan berdasarkan skenario yang telah dijelaskan pada Bab III, meliputi validasi operasional sistem, pengujian antarmuka GPIO, serta pengujian integrasi periferal eksternal.

4.3.1 Validasi Operasional Sistem

Pengujian pertama dilakukan untuk memverifikasi keberhasilan proses *booting*, pemuatan program ke *instruction RAM* PULPino, serta eksekusi program pada inti prosesor RI5CY. Program aplikasi dimuat menggunakan utilitas `spiload` melalui sistem operasi Linux ARM, kemudian seluruh proses diamati melalui terminal serial UART.

```
# /mnt/boot/spiload -t100000 /mnt/boot/pulpino_test.txt
Device has been reset
Sending block addr 00000000 with 256 entries
Sending block addr 000003FC with 256 entries
Sending block addr 000007F8 with 256 entries
Sending block addr 00000BF4 with 256 entries
Sending block addr 00000FF0 with 256 entries
Sending block addr 000013EC with 256 entries
Sending block addr 000017E8 with 256 entries
Sending block addr 00001BE4 with 256 entries
Sending block addr 00001FE0 with 256 entries
Sending block addr 000023DC with 256 entries
Sending block addr 000027D8 with 256 entries
Sending block addr 00002BD4 with 256 entries
Sending block addr 00002FD0 with 243 entries
Sending block addr 00100000 with 256 entries
Sending block addr 001003FC with 84 entries
Starting device
Waiting for EOC...
PULPino:
PULPino: ==================Hello World!!!==================
PULPino: ==================================================
PULPino: System ready. Monitoring actions... All feedback is displayed on OLED.
PULPino:
```

Gambar 4.44. Hasil validasi proses *bootloading* dan eksekusi program pada SoC PULPino

Berdasarkan Gambar 4.44, proses *bootloading* berlangsung tanpa kesalahan hingga program berhasil dijalankan oleh RI5CY. Setiap keluaran yang ditampilkan oleh program `spiload` merepresentasikan tahapan tertentu selama proses pemuatan program. Interpretasi terhadap setiap keluaran terminal tersebut dirangkum pada Tabel 4.3.

Tabel 4.3. Interpretasi keluaran utilitas `spiload` selama validasi operasional sistem

Keluaran Terminal	**Interpretasi**
`Device has been reset`	Program `spiload` berhasil mengendalikan sinyal *reset* sehingga inti prosesor RI5CY berada pada kondisi *reset* sebelum proses pemuatan program dimulai.
`Sending block addr ...`	Berkas program biner hasil kompilasi mulai ditransfer secara bertahap ke *Instruction RAM* PULPino melalui antarmuka SPI. Proses berlangsung hingga seluruh blok data selesai ditransfer.
`Starting device`	*Instruction RAM* telah berhasil ditulis, kemudian sinyal *reset* dilepaskan sehingga RI5CY mulai mengeksekusi program dari alamat awal memori.
`Waiting for EOC...`	Program `spiload` menunggu sinyal bahwa proses eksekusi program telah selesai.
`Hello World!!!`	Pesan dihasilkan oleh fungsi `printf()` pada program yang dijalankan oleh RI5CY melalui periferal UART.

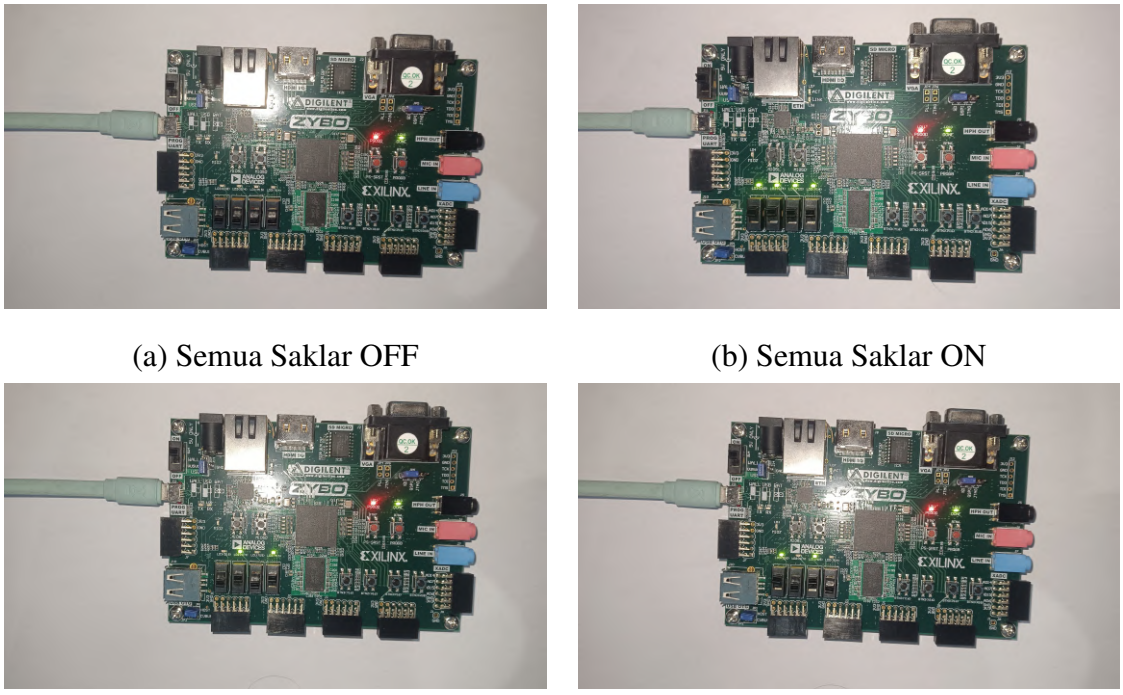
Secara keseluruhan, urutan keluaran pada terminal menunjukkan bahwa program berhasil ditransfer ke *instruction RAM*, RI5CY dapat mengeksekusi program dengan benar, serta komunikasi UART berfungsi sesuai rancangan. Hasil ini menjadi dasar bahwa implementasi SoC PULPino pada FPGA Zybo telah berhasil melewati tahap validasi operasional dan siap digunakan untuk pengujian fungsional periferal pada tahap berikutnya.

4.3.2 Pengujian Operasional Periferal Masukan/Keluaran (GPIO)

Pada pengujian ini, sebuah program aplikasi bahasa C (`helloworld.c`) yang lengkap dan terintegrasi telah dirancang untuk memvalidasi operasional seluruh port GPIO secara serentak. Pengujian mencakup validasi saklar, LED, tombol, hingga port PMOD yang terpasang pada Zybo guna memverifikasi koneksi fisik *pin mapping* dan kontrol register dari RI5CY ke perangkat I/O.

4.3.2.1 Pengujian Saklar (*Switch*) Fisik dan Lampu LED

Pengujian tahap pertama fokus pada membaca status masukan dari saklar dan menulisnya ke keluaran LED, serta menampilkan pesan teks pada terminal serial.



(a) Semua Saklar OFF

(b) Semua Saklar ON

(c) Saklar 0 & 2 ON

(d) Saklar 1 & 3 ON

Gambar 4.45. Dokumentasi Pengujian Fisik Saklar-LED pada Board Zybo

Selain divalidasi secara visual melalui kondisi LED pada Gambar 4.45, hasil pembacaan saklar juga ditampilkan pada terminal serial UART sebagai umpan balik sistem.

```
PULPino: [SWITCH] HEX: 0x03
PULPino: [SWITCH] HEX: 0x07
PULPino: [SWITCH] HEX: 0x0F
PULPino: [SWITCH] HEX: 0x0D
PULPino: [SWITCH] HEX: 0x05
PULPino: [SWITCH] HEX: 0x01
PULPino: [SWITCH] HEX: 0x00
PULPino: [SWITCH] HEX: 0x02
PULPino: [SWITCH] HEX: 0x0A
```

Gambar 4.46. Tangkapan Layar Log Interaksi Umpan Balik *Switch*

Seperti yang tertera pada Gambar 4.46, setiap perubahan kondisi saklar berhasil dibaca oleh sistem dan direpresentasikan melalui perubahan status LED yang sesuai. Nilai saklar yang terbaca juga dikirimkan ke terminal UART dalam format heksadesimal, sehingga memudahkan proses verifikasi hasil pembacaan.

4.3.2.2 Pengujian Interaksi Tombol (*Push Button*)

Tahap pengujian selanjutnya difokuskan pada membaca masukan dari tombol.

```
PULPino: [BUTTON] Pressed: BTN0
PULPino: [BUTTON] Pressed: BTN1
PULPino: [BUTTON] Pressed: BTN2
PULPino: [BUTTON] Pressed: BTN3
PULPino: [BUTTON] Pressed: BTN2
PULPino: [BUTTON] Pressed: BTN1
PULPino: [BUTTON] Pressed: BTN0
```

Gambar 4.47. Tangkapan Layar Log Respon *Push Button* pada UART

Berdasarkan tampilan sistem pada Gambar 4.47, sistem berhasil mendeteksi setiap penekanan tombol dan menampilkan informasi tombol yang aktif melalui antarmuka UART dengan tepat.

4.3.2.3 Pengujian Port Modul Periferal (*Peripheral Module* atau PMOD)

Tahap pengujian selanjutnya difokuskan pada verifikasi fungsi GPIO melalui port PMOD. Pengujian dilakukan dengan menghubungkan masing-masing pin PMOD JB Pin 7 hingga Pin 10 ke GND untuk memicu perubahan level logika pada masukan GPIO.

```
PULPino: [PMOD] PMOD JB Pin 7 -> LOW
PULPino: [PMOD] PMOD JB Pin 7 -> HIGH
PULPino: [PMOD] PMOD JB Pin 8 -> LOW
PULPino: [PMOD] PMOD JB Pin 8 -> HIGH
PULPino: [PMOD] PMOD JB Pin 9 -> LOW
PULPino: [PMOD] PMOD JB Pin 9 -> HIGH
PULPino: [PMOD] PMOD JB Pin 10 -> LOW
PULPino: [PMOD] PMOD JB Pin 10 -> HIGH
```

Gambar 4.48. Tangkapan Layar Deteksi Log Perubahan Transisi Level Tegangan PMOD

Berdasarkan tampilan sistem pada Gambar 4.48, seluruh pin PMOD JB yang diuji terbaca pada level logika HIGH saat tidak dihubungkan ke GND. Setelah masing-masing pin dihubungkan ke GND, nilai pembacaan berubah menjadi LOW dan berhasil ditampilkan pada terminal UART.

Secara keseluruhan, hasil pengujian GPIO menunjukkan bahwa seluruh antarmuka masukan dan keluaran pada sistem telah berfungsi sesuai dengan rancangan. Pembacaan masukan dari saklar, tombol, dan port PMOD berhasil dilakukan dengan benar, sedangkan keluaran melalui LED serta informasi yang ditampilkan pada UART sesuai dengan kondisi masukan yang diberikan. Hasil tersebut menunjukkan bahwa implementasi antarmuka GPIO pada sistem PULPino mampu menangani komunikasi dengan periferal internal maupun perangkat eksternal secara andal.

4.4 Pengujian Integrasi Periferal Tambahan Sensor PIR dan Antarmuka Layar I2C OLED

Tahap pengujian berikutnya dilakukan untuk memverifikasi integrasi beberapa periferal pada sistem PULPino. Program bahasa C dirancang untuk mengelola komunikasi dengan modul OLED GM009605v4 menggunakan protokol I2C, sekaligus membaca masukan dari saklar, tombol, dan sensor PIR secara real-time.

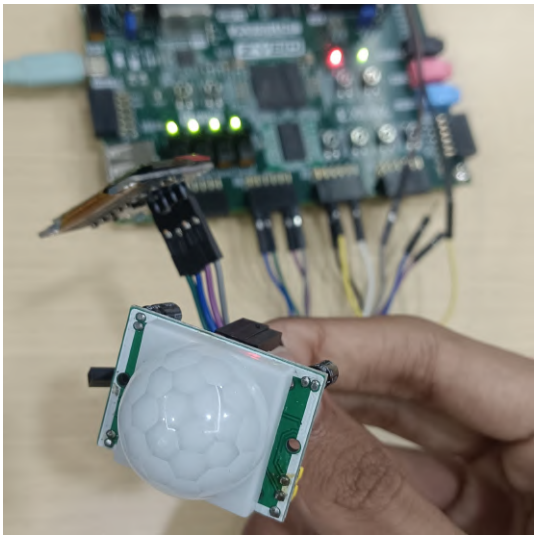
```
PULPino: [PIR SENSOR] Movement Detected
PULPino: [PIR SENSOR] Safe
PULPino: [PIR SENSOR] Movement Detected
PULPino: [PIR SENSOR] Safe
PULPino: [PIR SENSOR] Movement Detected
PULPino: [PIR SENSOR] Safe
```

Gambar 4.49. Tangkapan Layar Deteksi Log Sensor PIR

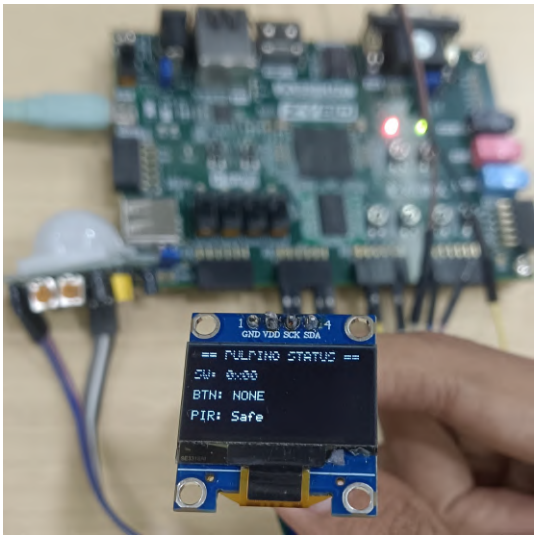
Berdasarkan Gambar 4.49 dapat terlihat bahwa pesan notifikasi pembacaaan sensor PIR berhasil ditampilkan pada sistem. Hal ini menunjukkan bahwa sensor PIR terbukti bekerja serta port JC02 mampu membaca sinyal dari sensor dengan baik. Kemudian dilakukan pengujian terhadap tiga skenario tampilan pada OLED sebagaimana ditunjukkan pada Tabel 4.4.

Tabel 4.4. Tabel Skenario Pengujian Aplikasi Periferal Tambahan

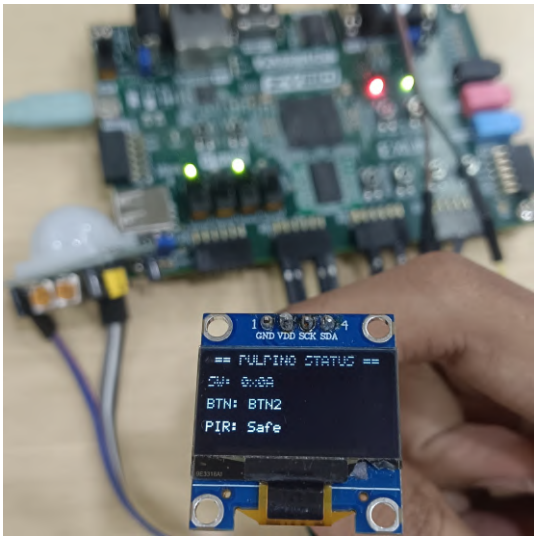
Skenario	Tombol Ditekan	Status Bit Saklar	Status PIR
1	Tidak Ada	0000	Tidak Ada Gerakan
2	Tombol 3	1010	Tidak Ada Gerakan
3	Tombol 4	1111	Ada Gerakan



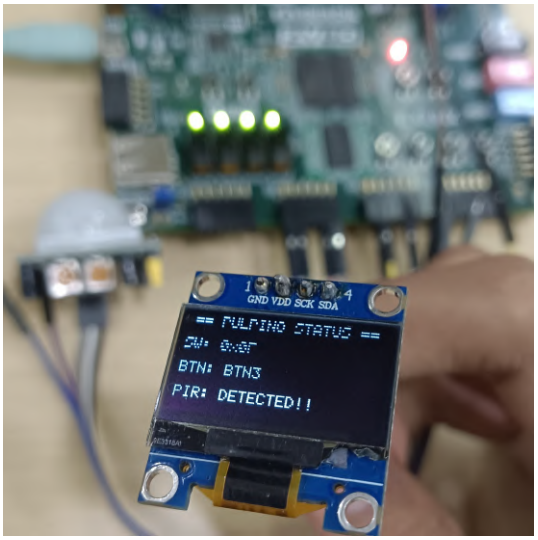
(a) Sensor PIR dan OLED I2C



(b) Status Skenario 1



(c) Status Skenario 2



(d) Status Skenario 3

Gambar 4.50. Dokumentasi Pengujian Periferal Lengkap pada Board Zybo

Dapat terlihat pada Gambar 4.50 (b) layar OLED menampilkan informasi SW: 0x00, BTN: NONE, dan PIR: Safe, serta seluruh LED tidak ada yang aktif. Lalu pada Gambar 4.50 (c) OLED menampilkan SW: 0x0A, BTN: BTN2, dan PIR: Safe, sementara LED menyala dengan pola biner 1010 yang sesuai dengan nilai hex dari saklar. Terakhir, pada Gambar 4.50 (d) OLED menampilkan SW: 0x0F, BTN: BTN3, dan PIR: DETECTED!!, dengan seluruh LED menyala membentuk pola biner 1111.

Hasil pengujian menunjukkan bahwa ketiga skenario dapat dijalankan sesuai dengan rancangan. Nilai saklar berhasil dikonversi dan ditampilkan dalam format heksadesimal, status tombol terakhir yang ditekan dapat teridentifikasi dengan benar, serta kondisi deteksi gerakan dari sensor PIR ditampilkan secara akurat pada layar OLED. Selain itu, pola LED yang menyala juga sesuai dengan nilai biner dari saklar yang diberikan.

Hal ini menunjukkan bahwa komunikasi I2C dengan modul OLED, pembacaan perangkat masukan, serta mekanisme pembaruan tampilan pada sistem telah berfungsi dengan baik.

4.4.1 Perbandingan Hasil Implementasi dengan Penelitian Terdahulu

Untuk memperoleh gambaran mengenai karakteristik sintesis *core* RI5CY ini dibandingkan dengan sintesis *core* Zero-Riscy pada platform FPGA Zybo *development board* Z-7010 yang sama sehingga perbandingan dapat dilakukan terhadap penggunaan sumber daya maupun karakteristik *timing* hasil sintesis.

Tabel 4.5. Perbandingan Kinerja Sintesis SoC Berbasis Zero-Riscy dan RI5CY.

Parameter Evaluasi	**Zero-Riscy (*Pipeline* 2-Tahap)**	**RI5CY (*Pipeline* 4-Tahap)**	**Keterangan Selisih**
Logic LUTs	10.320 (58,64%)	13.398 (76,13%)	RI5CY memakan +3.078 LUT (+29,8%)
Flip-Flops (FFs)	9.280 (26,36%)	9.924 (28,19%)	RI5CY membutuhkan +644 register FF
Blok DSP	1	11	RI5CY memiliki kapabilitas perkalian kompleks
Blok RAM (BRAM36)	16	16	Kapasitas memori sistem sama (64 KB)
***Worst Negative Slack* (WNS)**	+31,236 ns	+26,587 ns	Zero-Riscy memiliki margin pewaktuan lebih longgar
Frekuensi Maksimum (Fmax)	$\sim$53,29 MHz	$\sim$42,71 MHz	Zero-Riscy dapat dipacu $\pm$24,7% lebih cepat
Total Daya *On-Chip*	0,107 W	0,109 W	RI5CY sedikit lebih boros energi (+0,002 W)
Daya Dinamis	0,015 W	0,017 W	Aktivitas pensakelaran internal RI5CY lebih tinggi

Berdasarkan Tabel 4.5, sintesis SoC PULPino berbasis *core* RI5CY membutuhkan sumber daya FPGA yang lebih besar dibandingkan sintesis Zero-Riscy. Peningkatan penggunaan sumber daya paling signifikan terlihat pada LUT, yaitu dari 10.320 menjadi 13.400 LUT, serta pada blok DSP yang meningkat dari 1 menjadi 11 blok. Perbedaan tersebut menunjukkan bahwa RI5CY memiliki kompleksitas logika yang lebih tinggi, terutama akibat keberadaan unit perkalian perangkat keras (*hardware multiplier*) yang

memanfaatkan blok DSP FPGA.

Meskipun penggunaan sumber daya meningkat, kedua sintesis tetap menggunakan jumlah BRAM yang sama, yaitu 16 blok. Hal ini disebabkan kedua sintesis menggunakan konfigurasi memori instruksi dan memori data dengan kapasitas yang sama sehingga kebutuhan memori fisik FPGA tidak mengalami perubahan.

Perbedaan kompleksitas sintesis juga terlihat pada hasil analisis *timing*. Sintesis RI5CY menghasilkan nilai *Worst Negative Slack* (WNS) sebesar 26,587 ns, sedangkan sintesis Zero-Riscy memperoleh WNS sebesar 31,236 ns. Nilai WNS yang lebih kecil menunjukkan bahwa lintasan kritis *critical path* pada RI5CY lebih panjang sehingga margin *timing* yang tersedia menjadi lebih rendah dibandingkan Zero-Riscy.

Berdasarkan rumus frekuensi maksimum 4-1, sintesis RI5CY memiliki estimasi frekuensi kerja maksimum sebesar 42,71 MHz, sedangkan sintesis Zero-Riscy diperkirakan mampu mencapai 53,29 MHz. Hasil tersebut menunjukkan bahwa peningkatan kompleksitas arsitektur RI5CY memberikan konsekuensi terhadap bertambahnya penggunaan sumber daya logika sekaligus menurunkan frekuensi kerja maksimum yang dapat dicapai. Meskipun demikian, kedua sintesis masih memenuhi *timing constraint* pada frekuensi operasi 20 MHz sehingga dapat diimplementasikan dengan baik pada FPGA Zybo *development board* Z-7010.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil modifikasi, implementasi, pengujian, dan analisis yang telah dilakukan, diperoleh beberapa kesimpulan sebagai berikut:

1. Modifikasi repositori PULPino berhasil dilakukan sehingga seluruh proses sintesis, implementasi, dan *deployment* dapat dijalankan menggunakan Xilinx Vivado 2025.1, Vitis Unified IDE 2025.1, serta lingkungan Windows Subsystem for Linux (WSL). Penyesuaian terhadap skrip sintesis, sistem kompilasi, dan lingkungan *build* memungkinkan repositori PULPino yang dikembangkan pada ekosistem lama tetap kompatibel dengan perangkat lunak modern.
2. Implementasi SoC PULPino pada FPGA Zybo Z-7010 berhasil menggunakan 13.400 LUT (76,14%), 9.924 Flip-Flop (28,19%), 16 blok BRAM (26,67%), dan 11 blok DSP (13,75%). Penggunaan DSP seluruhnya berasal dari unit perkalian perangkat keras pada *core* RI5CY, sedangkan memori instruksi dan data masing-masing berkapasitas 32 KB dipetakan ke dalam 16 blok BRAM.
3. Hasil analisis *timing* menunjukkan bahwa seluruh *timing constraint* berhasil dipenuhi tanpa *failing endpoint*. Nilai WNS sebesar 26,587 ns pada target frekuensi 20 MHz menghasilkan estimasi frekuensi kerja maksimum sebesar 42,71 MHz, sehingga implementasi memiliki margin *timing* yang memadai untuk beroperasi secara stabil.
4. Pengujian operasional menunjukkan bahwa proses *bootloading* menggunakan utilitas `spiload` berhasil memuat program RISC-V ke dalam memori instruksi PULPino. Selain itu, komunikasi UART, antarmuka GPIO, konektor PMOD, sensor PIR, serta modul OLED berbasis I^2C berhasil beroperasi sesuai dengan rancangan sehingga membuktikan bahwa SoC PULPino dapat diimplementasikan dan dijalankan dengan baik pada FPGA Zybo Z-7010.
5. Hasil penelitian menunjukkan bahwa SoC PULPino berbasis *core* RI5CY masih dapat direplikasi dan diimplementasikan pada lingkungan pengembangan FPGA modern melalui proses modifikasi repositori. Hasil implementasi ini dapat menjadi referensi bagi pengembangan maupun penelitian lanjutan yang memanfaatkan platform PULPino pada perangkat FPGA berbasis Zynq-7000.

5.2 Saran

Beberapa saran yang dapat diajukan untuk pengembangan penelitian selanjutnya adalah sebagai berikut:

1. Melakukan optimasi pada jembatan bus AXI-to-APB atau memangkas periferal internal PULPino yang tidak digunakan aktif untuk mengurangi beban penggunaan LUT pada FPGA seri kecil.
2. Mengaktifkan dan menguji instruksi kompresi RISC-V (ekstensi RV32C) pada *core* RI5CY untuk memperkecil ukuran kode program aplikasi bare-metal sehingga lebih menghemat penggunaan memori BRAM.
3. Melakukan pengukuran konsumsi daya dinamis sirkuit PL FPGA secara langsung pada perangkat keras menggunakan alat ukur daya eksternal untuk memperoleh data efisiensi energi secara riil.
4. Melakukan porting skrip-skrip otomasi pembuatan proyek dan generator memori *legacy* PULPino dari Python 2.7 ke Python 3 guna mempermudah *reproducibility* proyek pada sistem operasi modern.

DAFTAR PUSTAKA

[1] A. Traber, M. Gautschi, and D. Pasotti, "PULPino: Datasheet," ETH Zurich, University of Bologna, Tech. Rep., 2017.

[2] OpenHW Group, *CORE-V CV32E40P User Manual*, OpenHW Group, 2026. [Online]. Available: https://docs.openhwgroup.org/projects/cv32e40p-user-manual/

[3] R. Höller, D. Haselberger, D. Ballek, P. Rössler, M. Krapfenbauer, and M. Linauer, "Open-source risc-v processor ip cores for fpgas — overview and evaluation," in *2019 8th Mediterranean Conference on Embedded Computing (MECO)*, 2019, pp. 1–6.

[4] C. Heinz, Y. Lavan, J. Hofmann, and A. Koch, "A catalog and in-hardware evaluation of open-source drop-in compatible risc-v softcore processors," in *2019 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, 2019, pp. 1–8.

[5] A. Verma, P. Sharma, and B. P. Das, "Risc-v core with approximate multiplier for error-tolerant applications," in *2022 25th Euromicro Conference on Digital System Design (DSD)*, 2022, pp. 239–246.

[6] "Implementation of fpga based 32-bit risc-v processor," *Engineering Science and Technology, an International Journal*, vol. 70, p. 102139, Aug. 2015. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/HOTCHIPS.2015.7477325

[7] A. Traber, M. Gautschi, and P. D. Schiavone, *RI5CY: User Manual*, ETH Zurich and University of Bologna, Apr. 2019, revision 4.0.

[8] *The RISC-V Instruction Set Manual. Volume I: Unprivileged ISA*, RISC-V International, Dec. 2019, document Version 20191213.

[9] D. J. Greaves, *Modern System-on-Chip Design on Arm*. Cambridge, UK: Arm Education Media, 2021.

[10] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed. Morgan Kaufmann, 2021.

[11] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.

[12] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[13] T. Barua, "DATA TRANSFER PERFORMANCE ANALYSIS FROM PROGRAMMABLE LOGIC TO PROCESSING SYSTEM OF ZYNQ 7000," 7 2020. [Online]. Available: https://hammer.purdue.edu/articles/thesis/DATA_TRANSFER_PERFORMANCE_ANALYSIS_FROM_PROGRAMMABLE_LOGIC_TO_PROCESSING_SYSTEM_OF_ZYNQ_7000/12743315

[14] *SPI Block Guide, V04.01*, Motorola, Inc., July 2004, document Number: S12SPIV4/D.

[15] P. Marwedel, *Embedded System Design: Embedded Systems, Foundations of Cyber-Physical Systems, and the Internet of Things*, 4th ed. Springer Nature, 2021.

[16] Digilent, Inc., *ZYBO FPGA Board Reference Manual*, Digilent, Inc., 2016. [Online]. Available: https://digilent.com/reference/programmable-logic/zybo/reference-manual

[17] *Zynq-7000 SoC Technical Reference Manual*, Advanced Micro Devices (AMD), 2026, accessed: 2026-06-29. [Online]. Available: https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM

[18] *UltraFast Design Methodology Guide for FPGAs and SoCs*, Advanced Micro Devices (AMD), 2026, section: Assessing the Maximum Frequency of the Design. Accessed: 2026-06-29. [Online]. Available: https://docs.amd.com/r/en-US/ug949-vivado-design-methodology

[19] *7 Series FPGAs Configurable Logic Block User Guide (UG474)*, Advanced Micro Devices (AMD) Xilinx, 2016. [Online]. Available: https://docs.amd.com/v/u/en-US/ug474_7Series_CLB

[20] *Zynq-7000 SoC Data Sheet: Overview (DS190)*, Advanced Micro Devices (AMD) Xilinx, 2021. [Online]. Available: https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview

[21] *7 Series DSP48E1 Slice User Guide (UG479)*, Advanced Micro Devices (AMD) Xilinx, 2018. [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

[22] *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*, Advanced Micro Devices (AMD) Xilinx, 2024. [Online]. Available: https://docs.amd.com/v/u/en-US/ug906-vivado-design-analysis

[23] Microsoft, "Windows subsystem for linux (wsl) documentation," https://learn.microsoft.com/en-us/windows/wsl/, 2026.

[24] The U-Boot Development Community, "Das u-boot: The universal boot loader," https://u-boot-project.org/, 2026.

[25] The Buildroot developers, *The Buildroot user manual*, Buildroot, 2026. [Online]. Available: https://buildroot.org/downloads/manual/manual.html

[26] *Devicetree Specification*, Devicetree.org, June 2023. [Online]. Available: https://www.devicetree.org/specifications/

[27] I. Kitware, "CMake: Cross-Platform Makefile Generator," https://cmake.org/, 2026.

[28] PULP Platform, "Pulpino: An open-source risc-v microcontroller system," https://github.com/pulp-platform/pulpino, 2026, gitHub repository, accessed: 2026-06-29.

LAMPIRAN

L.1 Repositori Modifikasi

Tautan repositori modifikasi PULPino untuk SoC RISCY-V pada FPGA Zybo dapat diakses pada repositori GitHub berikut:
`https://github.com/ziamat/pulpino`

L.2 Kode Program

L.2.1 `helloworld.c`

Berikut adalah kode sumber C (`helloworld.c`) yang digunakan untuk menguji kebenaran kompilasi, eksekusi, serta antarmuka UART, GPIO (termasuk kendali Switch, LED, Pmod, dan sensor PIR), serta I2C pada SoC PULPino:

```
1  // Copyright 2017 ETH Zurich and University of Bologna.
2  // Copyright and related rights are licensed under the Solderpad Hardware
3  // License, Version 0.51 (the "License"); you may not use this file except in
4  // compliance with the License.  You may obtain a copy of the License at
5  // http://solderpad.org/licenses/SHL-0.51. Unless required by applicable law
6  // or agreed to in writing, software, hardware and materials distributed under
7  // this License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
8  // CONDITIONS OF ANY KIND, either express or implied. See the License for the
9  // specific language governing permissions and limitations under the License.
10
11 #include "gpio.h"
12 #include "i2c.h"
13 #include "pulpino.h"
14 #include "string_lib.h"
15
16 #define NUM_SW_LED 4
17 #define NUM_BTN 4
18 #define NUM_PMOD 5
19
20 int sw_pins[NUM_SW_LED] = {0, 1, 2, 3};      // SW0 - SW3 (GPIO 0-3)
21 int led_pins[NUM_SW_LED] = {8, 9, 10, 11}; // LD0 - LD3 (GPIO 8-11)
22 int btn_pins[NUM_BTN] = {16, 17, 18, 19};  // BTN0 - BTN3 (GPIO 16-19)
23 const char *btn_messages[NUM_BTN] = {"BTN0", "BTN1", "BTN2", "BTN3"};
```

```
24 int pmod_pins[NUM_PMOD] = {
25     20, // PMOD JC Pin 2 (PIR)
26     5,  // PMOD JB Pin 7
27     4,  // PMOD JB Pin 8
28     7,  // PMOD JB Pin 9
29     6   // PMOD JB Pin 10
30 };
31
32 const char *pmod_names[NUM_PMOD] = {"PMOD JC Pin 2 (PIR)", "PMOD
       JB Pin 7",
33                                     "PMOD JB Pin 8", "PMOD JB
      Pin 9",
34                                     "PMOD JB Pin 10"};
35
36 // ========================================
37 // FONT 5x8 & STRING UTILS
38 // ========================================
39 const unsigned char font5x8[][5] = {
40     {0x00, 0x00, 0x00, 0x00, 0x00}, // 0x20 Space
41     {0x00, 0x00, 0x5F, 0x00, 0x00}, // 0x21 !
42     {0x00, 0x07, 0x00, 0x07, 0x00}, // 0x22 "
43     {0x14, 0x7F, 0x14, 0x7F, 0x14}, // 0x23 #
44     {0x24, 0x2A, 0x7F, 0x2A, 0x12}, // 0x24 $
45     {0x23, 0x13, 0x08, 0x64, 0x62}, // 0x25 %
46     {0x36, 0x49, 0x55, 0x22, 0x50}, // 0x26 &
47     {0x00, 0x05, 0x03, 0x00, 0x00}, // 0x27 '
48     {0x00, 0x1C, 0x22, 0x41, 0x00}, // 0x28 (
49     {0x00, 0x41, 0x22, 0x1C, 0x00}, // 0x29 )
50     {0x14, 0x08, 0x3E, 0x08, 0x14}, // 0x2A *
51     {0x08, 0x08, 0x3E, 0x08, 0x08}, // 0x2B +
52     {0x00, 0x50, 0x30, 0x00, 0x00}, // 0x2C ,
53     {0x08, 0x08, 0x08, 0x08, 0x08}, // 0x2D -
54     {0x00, 0x60, 0x60, 0x00, 0x00}, // 0x2E .
55     {0x20, 0x10, 0x08, 0x04, 0x02}, // 0x2F /
56     {0x3E, 0x51, 0x49, 0x45, 0x3E}, // 0x30 0
57     {0x00, 0x42, 0x7F, 0x40, 0x00}, // 0x31 1
58     {0x42, 0x61, 0x51, 0x49, 0x46}, // 0x32 2
59     {0x21, 0x41, 0x45, 0x4B, 0x31}, // 0x33 3
60     {0x18, 0x14, 0x12, 0x7F, 0x10}, // 0x34 4
61     {0x27, 0x45, 0x45, 0x45, 0x39}, // 0x35 5
62     {0x3C, 0x4A, 0x49, 0x49, 0x30}, // 0x36 6
```

```
63      {0x01, 0x71, 0x09, 0x05, 0x03}, // 0x37 7
64      {0x36, 0x49, 0x49, 0x49, 0x36}, // 0x38 8
65      {0x06, 0x49, 0x49, 0x29, 0x1E}, // 0x39 9
66      {0x00, 0x36, 0x36, 0x00, 0x00}, // 0x3A :
67      {0x00, 0x56, 0x36, 0x00, 0x00}, // 0x3B ;
68      {0x08, 0x14, 0x22, 0x41, 0x00}, // 0x3C <
69      {0x14, 0x14, 0x14, 0x14, 0x14}, // 0x3D =
70      {0x00, 0x41, 0x22, 0x14, 0x08}, // 0x3E >
71      {0x02, 0x01, 0x51, 0x09, 0x06}, // 0x3F ?
72      {0x32, 0x49, 0x79, 0x41, 0x3E}, // 0x40 @
73      {0x7E, 0x11, 0x11, 0x11, 0x7E}, // 0x41 A
74      {0x7F, 0x49, 0x49, 0x49, 0x36}, // 0x42 B
75      {0x3E, 0x41, 0x41, 0x41, 0x22}, // 0x43 C
76      {0x7F, 0x41, 0x41, 0x22, 0x1C}, // 0x44 D
77      {0x7F, 0x49, 0x49, 0x49, 0x41}, // 0x45 E
78      {0x7F, 0x09, 0x09, 0x09, 0x01}, // 0x46 F
79      {0x3E, 0x41, 0x49, 0x49, 0x7A}, // 0x47 G
80      {0x7F, 0x08, 0x08, 0x08, 0x7F}, // 0x48 H
81      {0x00, 0x41, 0x7F, 0x41, 0x00}, // 0x49 I
82      {0x20, 0x40, 0x41, 0x3F, 0x01}, // 0x4A J
83      {0x7F, 0x08, 0x14, 0x22, 0x41}, // 0x4B K
84      {0x7F, 0x40, 0x40, 0x40, 0x40}, // 0x4C L
85      {0x7F, 0x02, 0x0C, 0x02, 0x7F}, // 0x4D M
86      {0x7F, 0x04, 0x08, 0x10, 0x7F}, // 0x4E N
87      {0x3E, 0x41, 0x41, 0x41, 0x3E}, // 0x4F O
88      {0x7F, 0x09, 0x09, 0x09, 0x06}, // 0x50 P
89      {0x3E, 0x41, 0x51, 0x21, 0x5E}, // 0x51 Q
90      {0x7F, 0x09, 0x19, 0x29, 0x46}, // 0x52 R
91      {0x46, 0x49, 0x49, 0x49, 0x31}, // 0x53 S
92      {0x01, 0x01, 0x7F, 0x01, 0x01}, // 0x54 T
93      {0x3F, 0x40, 0x40, 0x40, 0x3F}, // 0x55 U
94      {0x1F, 0x20, 0x40, 0x20, 0x1F}, // 0x56 V
95      {0x3F, 0x40, 0x38, 0x40, 0x3F}, // 0x57 W
96      {0x63, 0x14, 0x08, 0x14, 0x63}, // 0x58 X
97      {0x07, 0x08, 0x70, 0x08, 0x07}, // 0x59 Y
98      {0x61, 0x51, 0x49, 0x45, 0x43}, // 0x5A Z
99      {0x00, 0x7F, 0x41, 0x41, 0x00}, // 0x5B [
100     {0x02, 0x04, 0x08, 0x10, 0x20}, // 0x5C backslash
101     {0x00, 0x41, 0x41, 0x7F, 0x00}, // 0x5D ]
102     {0x04, 0x02, 0x01, 0x02, 0x04}, // 0x5E ^
103     {0x40, 0x40, 0x40, 0x40, 0x40}, // 0x5F _
```

```
104     {0x00, 0x01, 0x02, 0x04, 0x00}, // 0x60 `
105     {0x20, 0x54, 0x54, 0x54, 0x78}, // 0x61 a
106     {0x7F, 0x48, 0x44, 0x44, 0x38}, // 0x62 b
107     {0x38, 0x44, 0x44, 0x44, 0x20}, // 0x63 c
108     {0x38, 0x44, 0x44, 0x48, 0x7F}, // 0x64 d
109     {0x38, 0x54, 0x54, 0x54, 0x18}, // 0x65 e
110     {0x08, 0x7E, 0x09, 0x01, 0x02}, // 0x66 f
111     {0x18, 0xA4, 0xA4, 0xA4, 0x7C}, // 0x67 g
112     {0x7F, 0x08, 0x04, 0x04, 0x78}, // 0x68 h
113     {0x00, 0x44, 0x7D, 0x40, 0x00}, // 0x69 i
114     {0x40, 0x80, 0x84, 0x7D, 0x00}, // 0x6A j
115     {0x7F, 0x10, 0x28, 0x44, 0x00}, // 0x6B k
116     {0x00, 0x41, 0x7F, 0x40, 0x00}, // 0x6C l
117     {0x7C, 0x04, 0x18, 0x04, 0x78}, // 0x6D m
118     {0x7C, 0x08, 0x04, 0x04, 0x78}, // 0x6E n
119     {0x38, 0x44, 0x44, 0x44, 0x38}, // 0x6F o
120     {0xFC, 0x24, 0x24, 0x24, 0x18}, // 0x70 p
121     {0x18, 0x24, 0x24, 0x18, 0xFC}, // 0x71 q
122     {0x7C, 0x08, 0x04, 0x04, 0x08}, // 0x72 r
123     {0x48, 0x54, 0x54, 0x54, 0x20}, // 0x73 s
124     {0x04, 0x3F, 0x44, 0x40, 0x20}, // 0x74 t
125     {0x3C, 0x40, 0x40, 0x20, 0x7C}, // 0x75 u
126     {0x1C, 0x20, 0x40, 0x20, 0x1C}, // 0x76 v
127     {0x3C, 0x40, 0x30, 0x40, 0x3C}, // 0x77 w
128     {0x44, 0x28, 0x10, 0x28, 0x44}, // 0x78 x
129     {0x1C, 0xA0, 0xA0, 0xA0, 0x7C}, // 0x79 y
130     {0x44, 0x64, 0x54, 0x4C, 0x44}, // 0x7A z
131     {0x00, 0x08, 0x36, 0x41, 0x00}, // 0x7B {
132     {0x00, 0x00, 0x7F, 0x00, 0x00}, // 0x7C |
133     {0x00, 0x41, 0x36, 0x08, 0x00}, // 0x7D }
134     {0x10, 0x08, 0x08, 0x10, 0x08}, // 0x7E ~
135     {0x00, 0x00, 0x00, 0x00, 0x00}  // 0x7F
136 };
137
138 void itoa_hex(int val, char *buf) {
139   const char hex_chars[] = "0123456789ABCDEF";
140   buf[0] = '0';
141   buf[1] = 'x';
142   buf[2] = hex_chars[(val >> 4) & 0xF];
143   buf[3] = hex_chars[val & 0xF];
144   buf[4] = '\0';
```

```
145 }
146
147 // ========================================
148 // OLED I2C CONFIGURATION
149 // ========================================
150 #define I2C_PRESCALER 0x63    // (soc_freq/(5*i2cfreq))-1 for 100
      Khz
151 #define OLED_ADDR_WRITE 0x78 // 0x3C << 1
152
153 void oled_send_cmd(unsigned char cmd) {
154   i2c_send_data(OLED_ADDR_WRITE);
155   i2c_send_command(I2C_START_WRITE);
156   i2c_get_ack();
157   i2c_send_data(0x00); // Co=0, D/C=0
158   i2c_send_command(I2C_WRITE);
159   i2c_get_ack();
160   i2c_send_data(cmd);
161   i2c_send_command(I2C_WRITE);
162   i2c_get_ack();
163   i2c_send_command(I2C_STOP);
164   while (i2c_busy())
165     ;
166 }
167
168 void oled_send_data_byte(unsigned char data) {
169   i2c_send_data(OLED_ADDR_WRITE);
170   i2c_send_command(I2C_START_WRITE);
171   i2c_get_ack();
172   i2c_send_data(0x40); // Co=0, D/C=1
173   i2c_send_command(I2C_WRITE);
174   i2c_get_ack();
175   i2c_send_data(data);
176   i2c_send_command(I2C_WRITE);
177   i2c_get_ack();
178   i2c_send_command(I2C_STOP);
179   while (i2c_busy())
180     ;
181 }
182
183 void oled_init() {
184   oled_send_cmd(0xAE); // Display OFF
```

```
185   oled_send_cmd(0xD5);
186   oled_send_cmd(0x80); // Set Clock Divide
187   oled_send_cmd(0xA8);
188   oled_send_cmd(0x3F); // Set Multiplex Ratio 64
189   oled_send_cmd(0xD3);
190   oled_send_cmd(0x00); // Set Display Offset
191   oled_send_cmd(0x40); // Set Display Start Line
192   oled_send_cmd(0x8D);
193   oled_send_cmd(0x14); // Enable Charge Pump
194   oled_send_cmd(0x20);
195   oled_send_cmd(0x00); // Memory Addressing Mode: Horizontal
196   oled_send_cmd(0xA1); // Segment Re-map
197   oled_send_cmd(0xC8); // COM Output Scan Direction
198   oled_send_cmd(0xDA);
199   oled_send_cmd(0x12); // COM Pins Hardware Configuration
200   oled_send_cmd(0x81);
201   oled_send_cmd(0xCF); // Contrast Control
202   oled_send_cmd(0xD9);
203   oled_send_cmd(0xF1); // Pre-charge Period
204   oled_send_cmd(0xDB);
205   oled_send_cmd(0x40); // VCOMH Deselect Level
206   oled_send_cmd(0xA4); // Entire Display ON Resume
207   oled_send_cmd(0xA6); // Set Normal Display
208   oled_send_cmd(0xAF); // Display ON
209 }
210
211 void oled_set_cursor(unsigned char col, unsigned char page) {
212   oled_send_cmd(0x21); // Set Column Address
213   oled_send_cmd(col);
214   oled_send_cmd(127);
215   oled_send_cmd(0x22); // Set Page Address
216   oled_send_cmd(page);
217   oled_send_cmd(7);
218 }
219
220 void oled_clear() {
221   oled_set_cursor(0, 0);
222   for (int i = 0; i < 1024; i++) {
223     oled_send_data_byte(0x00);
224   }
225 }
```

```
226
227 void oled_clear_line(unsigned char page) {
228   oled_set_cursor(0, page);
229   for (int i = 0; i < 128; i++) {
230     oled_send_data_byte(0x00);
231   }
232 }
233
234 void oled_print_char(char c) {
235   if (c < 32 || c > 127)
236     c = 32;
237   for (int i = 0; i < 5; i++) {
238     oled_send_data_byte(font5x8[c - 32][i]);
239   }
240   oled_send_data_byte(0x00); // 1 pixel spacing
241 }
242
243 void oled_print_string(const char *str, unsigned char col,
     unsigned char page) {
244   oled_set_cursor(col, page);
245   while (*str) {
246     oled_print_char(*str++);
247   }
248 }
249
250 // ========================================
251 // MAIN FUNCTION
252 // ========================================
253 int main() {
254   printf("\n==================Hello World!!!==================\n
     ");
255   printf("==================================================\n")
     ;
256
257   CGREG |= (1 << CGGPIO);
258
259   i2c_setup(I2C_PRESCALER, I2C_CTR_EN);
260   for (volatile int i = 0; i < 100000; i++)
261     ;
262   oled_init();
263   oled_clear();
```

```
264
265   oled_print_string("== PULPINO STATUS ==", 4, 0);
266
267   for (int i = 0; i < NUM_SW_LED; i++) {
268     set_pin_function(sw_pins[i], FUNC_GPIO);
269     set_gpio_pin_direction(sw_pins[i], DIR_IN);
270
271     set_pin_function(led_pins[i], FUNC_GPIO);
272     set_gpio_pin_direction(led_pins[i], DIR_OUT);
273     set_gpio_pin_value(led_pins[i], 0);
274   }
275
276   for (int i = 0; i < NUM_BTN; i++) {
277     set_pin_function(btn_pins[i], FUNC_GPIO);
278     set_gpio_pin_direction(btn_pins[i], DIR_IN);
279   }
280
281   for (int i = 0; i < NUM_PMOD; i++) {
282     set_pin_function(pmod_pins[i], FUNC_GPIO);
283     set_gpio_pin_direction(pmod_pins[i], DIR_IN);
284   }
285
286   // Read initial states
287   int last_sw_states[NUM_SW_LED] = {-1, -1, -1, -1};
288   int last_btn_states[NUM_BTN] = {0, 0, 0, 0};
289   int last_pmod_states[NUM_PMOD] = {-1, -1, -1, -1, -1};
290
291   // Display initial states on OLED
292   int initial_sw = 0;
293   for (int i = 0; i < NUM_SW_LED; i++) {
294     initial_sw |= (get_gpio_pin_value(sw_pins[i]) << i);
295     last_sw_states[i] = get_gpio_pin_value(sw_pins[i]);
296   }
297   char sw_buf[16] = "SW: ";
298   itoa_hex(initial_sw , sw_buf + 4);
299   oled_print_string(sw_buf, 0, 2);
300
301   oled_print_string("BTN: NONE", 0, 4);
302
303   // Initialize PIR sensor status (PMOD JC Pin 2)
304   int initial_pir = get_gpio_pin_value(pmod_pins[0]);
```

```
305   if (initial_pir == 1)
306     oled_print_string("PIR: DETECTED!", 0, 6);
307   else
308     oled_print_string("PIR: SAFE", 0, 6);
309
310   printf("System ready. Monitoring actions... All feedback is
       displayed on "
311          "OLED.\n\n");
312
313   while (1) {
314     // ------------------------------------------------
315     // 1. CONTROL LED FROM SWITCH & OLED FEEDBACK
316     // ------------------------------------------------
317     int current_sw_hex = 0;
318     int sw_changed = 0;
319
320     for (int i = 0; i < NUM_SW_LED; i++) {
321       current_sw_hex |= (get_gpio_pin_value(sw_pins[i]) << i);
322     }
323
324     for (int i = 0; i < NUM_SW_LED; i++) {
325       int sw_val = get_gpio_pin_value(sw_pins[i]);
326       set_gpio_pin_value(led_pins[i], sw_val);
327
328       if (sw_val != last_sw_states[i]) {
329         if (last_sw_states[i] != -1)
330           sw_changed = 1;
331         last_sw_states[i] = sw_val;
332       }
333     }
334
335     if (sw_changed) {
336       itoa_hex(current_sw_hex, sw_buf + 4);
337       oled_clear_line(2);
338       oled_print_string(sw_buf, 0, 2);
339       printf("[SWITCH] HEX: %s\n", sw_buf + 4);
340     }
341
342     // ------------------------------------------------
343     // 2. BUTTON DETECTION & OLED FEEDBACK
344     // ------------------------------------------------
```

```
345     for (int i = 0; i < NUM_BTN; i++) {
346       int current_btn_val = get_gpio_pin_value(btn_pins[i]);
347
348       if (current_btn_val == 1 && last_btn_states[i] == 0) {
349         printf("[BUTTON] Pressed: %s\n", btn_messages[i]);
350         oled_clear_line(4);
351         oled_print_string("BTN: ", 0, 4);
352         oled_print_string(btn_messages[i], 30, 4);
353       }
354
355       last_btn_states[i] = current_btn_val;
356     }
357
358     // ------------------------------------------------
359     // 3. SENSOR PIR & PMOD DETECTION
360     // ------------------------------------------------
361     for (int i = 0; i < NUM_PMOD; i++) {
362       int current_pmod_val = get_gpio_pin_value(pmod_pins[i]);
363
364       if (current_pmod_val != last_pmod_states[i]) {
365         if (last_pmod_states[i] != -1) {
366           if (i == 0) { // PMOD JC Pin 2 (Sensor PIR)
367             oled_clear_line(6);
368             if (current_pmod_val == 1) {
369               printf("[PIR SENSOR] Movement Detected\n");
370               oled_print_string("PIR: DETECTED!!", 0, 6);
371             } else {
372               printf("[PIR SENSOR] Safe\n");
373               oled_print_string("PIR: Safe", 0, 6);
374             }
375           } else {
376             printf("[PMOD] %s -> %s\n", pmod_names[i],
377                    (current_pmod_val == 1) ? "HIGH" : "LOW");
378           }
379         }
380         last_pmod_states[i] = current_pmod_val;
381       }
382     }
383
384     // Delay debounce
385     for (volatile int delay = 0; delay < 50000; delay++)
```

```
386             ;
387     }
388
389     return 0;
390 }
```

L.2.2 pre_opt.tcl

```
# pre_opt.tcl
# This script runs right before opt_design to resolve black boxes for Xilinx encrypted memory IPs.
# Because pulpino.edn was pre-synthesized, it contains the IP wrappers but leaves the encrypted cores as black boxes.
# This script replaces the EDIF wrappers with the actual .dcp files to restore the full IP hierarchy.
#
# NOTE: When this script runs as a pre-opt hook via launch_runs, the working directory
# is inside pulpemu.runs/impl_1/ and current_project may not be available.
# We must use absolute paths derived from the script's own location.

puts "
    ================================================================================
    "
puts "pre_opt.tcl: Starting memory black box resolution"
puts "
    ================================================================================
    "
puts "pre_opt.tcl: pwd = [pwd]"
puts "pre_opt.tcl: info script = [info script]"

# --- Strategy 1: Derive path from the script's own location ---
# This script lives at: <pulpemu>/tcl/pre_opt.tcl
# IPs live at:          <pulpemu>/../ips
set script_dir [file dirname [file normalize [info script]]]
set project_base [file dirname $script_dir]
set ips_dir [file normalize "$project_base/../ips"]

puts "pre_opt.tcl: script_dir   = $script_dir"
puts "pre_opt.tcl: project_base = $project_base"
puts "pre_opt.tcl: ips_dir      = $ips_dir"

# Primary DCP location: <ips>/xilinx_mem_8192x32/ip/xilinx_mem_8192x32.dcp
set dcp_path [file normalize "$ips_dir/xilinx_mem_8192x32/ip/xilinx_mem_8192x32.dcp"]
```

```
29
30 puts "pre_opt.tcl: Primary DCP path = $dcp_path"
31 puts "pre_opt.tcl: File exists = [file exists $dcp_path]"
32
33 # --- Strategy 2: Fallback paths if primary doesn't exist ---
34 if {![file exists $dcp_path]} {
35     puts "pre_opt.tcl: Primary DCP not found, trying fallback
    paths..."
36
37     # Try from the generated sources
38     set fallback_paths [list \
39         [file normalize "$ips_dir/xilinx_mem_8192x32/
    xilinx_mem_8192x32.gen/sources_1/ip/xilinx_mem_8192x32/
    xilinx_mem_8192x32.dcp"] \
40         [file normalize "$ips_dir/xilinx_mem_8192x32/
    xilinx_mem_8192x32.runs/xilinx_mem_8192x32_synth_1/
    xilinx_mem_8192x32.dcp"] \
41     ]
42
43     # Also try glob
44     set glob_dcps [glob -nocomplain $ips_dir/xilinx_mem_*/ip/
    *.dcp]
45     foreach g $glob_dcps {
46         lappend fallback_paths $g
47     }
48     set glob_dcps2 [glob -nocomplain $ips_dir/xilinx_mem_*/
    *.runs/*/xilinx_mem_*.dcp]
49     foreach g $glob_dcps2 {
50         lappend fallback_paths $g
51     }
52
53     foreach fb $fallback_paths {
54         puts "pre_opt.tcl: Trying fallback: $fb (exists=[file
    exists $fb])"
55         if {[file exists $fb]} {
56             set dcp_path $fb
57             puts "pre_opt.tcl: Using fallback DCP: $dcp_path"
58             break
59         }
60     }
61 }
```

```
62
63 # --- Resolve the black boxes ---
64 set mem_cells [list \
65     "pulpino_wrap_i/pulpino_i/core_region_i/data_mem/sp_ram_i" \
66     "pulpino_wrap_i/pulpino_i/core_region_i/instr_mem/
    sp_ram_wrap_i/sp_ram_i" \
67 ]
68
69 set resolved 0
70
71 if {[file exists $dcp_path]} {
72     puts "pre_opt.tcl: DCP file confirmed: $dcp_path ([file size
    $dcp_path] bytes)"
73
74     foreach cell $mem_cells {
75         puts "pre_opt.tcl: Attempting to resolve: $cell"
76         if {[catch {read_checkpoint -cell $cell $dcp_path} msg]}
    {
77             puts "pre_opt.tcl: FAILED to read checkpoint for
    $cell"
78             puts "pre_opt.tcl: Error message: $msg"
79         } else {
80             puts "pre_opt.tcl: SUCCESS - resolved $cell"
81             incr resolved
82         }
83     }
84 } else {
85     puts "pre_opt.tcl: CRITICAL ERROR - No DCP file found!"
86     puts "pre_opt.tcl: Expected: $dcp_path"
87     puts "pre_opt.tcl: Please regenerate the xilinx_mem_8192x32
    IP:"
88     puts "pre_opt.tcl:   cd fpga/ips/xilinx_mem_8192x32 && make"
89 }
90
91 puts "
    ================================================================
    "
92 puts "pre_opt.tcl: Resolved $resolved of [llength $mem_cells]
    memory black boxes"
93
94 # Verify by checking remaining black boxes
```

```
95 if {[catch {set bb_cells [get_cells -quiet -hierarchical -filter
       {IS_BLACKBOX == 1}]} ]} {
96     set bb_cells {}
97 }
98 if {[llength $bb_cells] > 0} {
99     puts "pre_opt.tcl: WARNING - Remaining unresolved black
       boxes:"
100    foreach bb $bb_cells {
101        catch {
102            puts "  - $bb (type: [get_property REF_NAME $bb])"
103        }
104    }
105 } else {
106    puts "pre_opt.tcl: All black boxes resolved successfully!"
107 }
108 puts "
       ================================================================
       "
109
110 #
       ----------------------------------------------------------------

111 # Fix Route 35-19 / REQP-127: Fix OBUFs from pulpino.edn
112 #
       ----------------------------------------------------------------

113 # Problem: pulpino.edn has OBUFs on its output ports. In
       pulpemu_top,
114 # some OBUF outputs drive BOTH a top-level port AND internal PS7
       logic.
115 # An OBUF at an IOB site can only drive the external pad, not
       internal
116 # fabric. We must move internal loads from OBUF output to OBUF
       input.
117 puts "
       ================================================================
       "
118 puts "pre_opt.tcl: Fixing OBUFs/IBUFs from pulpino
       pre-synthesized netlist..."
119
120 # --- Fix OBUFs: move internal loads to pre-buffer signal ---
```

```
121 set obuf_cells [get_cells -quiet -hierarchical -filter {REF_NAME
    == OBUF && NAME =~ "pulpino_wrap_i/*"}]
122 set obuf_fixed 0
123 set obuf_removed 0
124 puts "pre_opt.tcl: Found [llength $obuf_cells] OBUF(s) in
    pulpino_wrap_i"
125
126 foreach obuf $obuf_cells {
127     if {[catch {
128         set o_pin [get_pins -quiet $obuf/O]
129         set i_pin [get_pins -quiet $obuf/I]
130         if {[llength $o_pin] == 0 || [llength $i_pin] == 0} {
    error "missing pins" }
131
132         set o_net [get_nets -quiet -of_objects $o_pin]
133         set i_net [get_nets -quiet -of_objects $i_pin]
134         if {[llength $o_net] == 0 || [llength $i_net] == 0} {
    error "missing nets" }
135
136         # Check what the OBUF output drives
137         set ext_ports [get_ports -quiet -of_objects $o_net]
138
139         # Use -leaf to find actual register/LUT input pins (not
    hierarchical ports)
140         set leaf_pins [get_pins -quiet -leaf -of_objects $o_net]
141         set internal_loads {}
142         foreach p $leaf_pins {
143             set pdir [get_property -quiet DIRECTION $p]
144             set pcell [get_property -quiet PARENT_CELL $p]
145             if {$pdir eq "IN" && $pcell ne $obuf} {
146                 lappend internal_loads $p
147             }
148         }
149         # Also check non-leaf pins
150         set all_pins [get_pins -quiet -of_objects $o_net]
151         foreach p $all_pins {
152             set pdir [get_property -quiet DIRECTION $p]
153             set pcell [get_property -quiet PARENT_CELL $p]
154             if {$pdir eq "IN" && $pcell ne $obuf && $p ni
    $internal_loads} {
155                 lappend internal_loads $p
```

```
156             }
157         }
158 
159         # Aggressively remove the pre-synthesized OBUF and let
    opt_design infer a fresh top-level one
160         puts "  Removing OBUF and merging nets: $obuf"
161 
162         # 1. Move all internal loads
163         foreach lp $internal_loads {
164             disconnect_net -quiet -net $o_net -objects $lp
165             connect_net -quiet -net $i_net -objects $lp
166         }
167 
168         # 2. Move external ports
169         foreach port $ext_ports {
170             disconnect_net -quiet -net $o_net -objects $port
171             connect_net -quiet -net $i_net -objects $port
172         }
173 
174         # 3. Disconnect OBUF pins and remove cell
175         disconnect_net -quiet -net $o_net -objects $o_pin
176         disconnect_net -quiet -net $i_net -objects $i_pin
177         remove_cell -quiet $obuf
178         incr obuf_removed
179     } msg]} {
180         puts "  WARNING: Failed to process OBUF $obuf: $msg"
181     }
182 }
183 puts "pre_opt.tcl: Fixed $obuf_fixed OBUF(s), removed
    $obuf_removed OBUF(s)"
184 
185 # --- Fix IBUFs: move internal drivers to post-buffer signal ---
186 set ibuf_cells [get_cells -quiet -hierarchical -filter {REF_NAME
    == IBUF && NAME =~ "pulpino_wrap_i/*"}]
187 set ibuf_fixed 0
188 set ibuf_removed 0
189 puts "pre_opt.tcl: Found [llength $ibuf_cells] IBUF(s) in
    pulpino_wrap_i"
190 
191 foreach ibuf $ibuf_cells {
192     if {[catch {
```

```
193         set i_pin [get_pins -quiet $ibuf/I]
194         set o_pin [get_pins -quiet $ibuf/O]
195         if {[llength $i_pin] == 0 || [llength $o_pin] == 0} {
    error "missing pins" }
196 
197         set i_net [get_nets -quiet -of_objects $i_pin]
198         set o_net [get_nets -quiet -of_objects $o_pin]
199         if {[llength $i_net] == 0 || [llength $o_net] == 0} {
    error "missing nets" }
200 
201         set ext_ports [get_ports -quiet -of_objects $i_net]
202 
203         if {[llength $ext_ports] == 0} {
204             # IBUF not driven by a top-level port - remove it
205             puts "  Removing IBUF (internal only): $ibuf"
206             set load_pins [get_pins -quiet -leaf -of_objects
    $o_net -filter {DIRECTION == IN}]
207             disconnect_net -quiet -net $i_net -objects $i_pin
208             disconnect_net -quiet -net $o_net -objects $o_pin
209             foreach lp $load_pins {
210                 disconnect_net -quiet -net $o_net -objects $lp
211                 connect_net -quiet -net $i_net -objects $lp
212             }
213             remove_cell -quiet $ibuf
214             incr ibuf_removed
215         }
216     } msg]} {
217         puts "  WARNING: Failed to process IBUF $ibuf: $msg"
218     }
219 }
220 puts "pre_opt.tcl: Removed $ibuf_removed redundant IBUF(s)"
221 puts "pre_opt.tcl: Summary: $obuf_fixed OBUF fixed,
    $obuf_removed OBUF removed, $ibuf_removed IBUF removed"
222 puts "
    ================================================================
    "
```

L.2.3 pre_place.tcl

```
1 # pre_place.tcl
2 # Runs AFTER opt_design, BEFORE place_design.
```

```
3  # Fixes REQP-127 / Route 35-19: OBUFs from pulpino.edn driving
       internal fabric.
4  #
5  # The three problematic OBUFs:
6  #   gpio_out_OBUF[16]_inst  -> drives axi_gpio_emu syncstages
       register
7  #   gpio_out_OBUF[17]_inst  -> drives axi_gpio_emu syncstages
       register
8  #   tdo_o_OBUF_inst         -> drives axi_jtag_emu syncstages
       register
9  #
10 # Fix: Move internal fabric loads from OBUF output net to OBUF
       input net,
11 #      so the OBUF only drives the external pad.
12
13 puts "
       ================================================================
       "
14 puts "pre_place.tcl: Fixing OBUF DRC errors (REQP-127) before
       place_design"
15 puts "
       ================================================================
       "
16
17 # --- Target ALL OBUFs inside pulpino_wrap_i ---
18 set obuf_cells [get_cells -quiet -hierarchical -filter {REF_NAME
       == OBUF && NAME =~ "pulpino_wrap_i/*"}]
19 set obuf_fixed 0
20 set obuf_removed 0
21 puts "pre_place.tcl: Found [llength $obuf_cells] OBUF(s) in
       pulpino_wrap_i"
22
23 foreach obuf $obuf_cells {
24     puts ""
25     puts "  === Processing OBUF: $obuf ==="
26
27     if {[catch {
28         set o_pin [get_pins -quiet $obuf/O]
29         set i_pin [get_pins -quiet $obuf/I]
30         if {[llength $o_pin] == 0 || [llength $i_pin] == 0} {
       error "missing pins" }
```

```
31
32          set o_net [get_nets -quiet -of_objects $o_pin]
33          set i_net [get_nets -quiet -of_objects $i_pin]
34          if {[llength $o_net] == 0 || [llength $i_net] == 0} {
    error "missing nets" }
35
36          puts "    O pin net: $o_net"
37          puts "    I pin net: $i_net"
38
39          # --- Diagnostic: enumerate EVERYTHING on the output net
     ---
40          set ext_ports  [get_ports -quiet -of_objects $o_net]
41          set leaf_pins  [get_pins  -quiet -leaf -of_objects
    $o_net]
42
43          puts "    Ports on O net: [llength $ext_ports] ->
    $ext_ports"
44          puts "    Leaf pins on O net: [llength $leaf_pins]"
45          foreach p $leaf_pins { puts "      leaf: $p  dir=[
    get_property -quiet DIRECTION $p]" }
46
47          # --- Find internal load pins (leaf input pins,
    excluding OBUF's own pins) ---
48          set internal_loads {}
49          foreach p $leaf_pins {
50              set pdir [get_property -quiet DIRECTION $p]
51              set pcell [get_property -quiet PARENT_CELL $p]
52              if {$pdir eq "IN" && $pcell ne $obuf} {
53                  lappend internal_loads $p
54              }
55          }
56
57          puts "    Internal loads found: [llength $internal_loads
    ]"
58          foreach il $internal_loads { puts "      load: $il" }
59
60          if {[llength $ext_ports] > 0 && [llength $internal_loads
    ] > 0} {
61              # Case 1: Drives BOTH a top-level port AND internal
    logic
62              puts "    ACTION: Moving [llength $internal_loads]
```

```
    internal load(s) to input net (keeping OBUF for port)"
63              foreach lp $internal_loads {
64                  if {[catch {
65                      disconnect_net -quiet -net $o_net -objects
    $lp
66                      connect_net    -quiet -net $i_net -objects
    $lp
67                      puts "    Moved: $lp"
68                  } move_msg]} {
69                      puts "    WARNING: Failed to move $lp:
    $move_msg"
70                  }
71              }
72              incr obuf_fixed
73
74          } elseif {[llength $ext_ports] == 0} {
75              # Case 2: Drives ONLY internal logic (no top-level
    port)
76              puts "    ACTION: Moving [llength $internal_loads]
    internal load(s) to input net AND removing OBUF"
77              foreach lp $internal_loads {
78                  if {[catch {
79                      disconnect_net -quiet -net $o_net -objects
    $lp
80                      connect_net    -quiet -net $i_net -objects
    $lp
81                      puts "    Moved: $lp"
82                  } move_msg]} {
83                      puts "    WARNING: Failed to move $lp:
    $move_msg"
84                  }
85              }
86              disconnect_net -quiet -net $o_net -objects $o_pin
87              disconnect_net -quiet -net $i_net -objects $i_pin
88              remove_cell    -quiet $obuf
89              incr obuf_removed
90
91          } else {
92              # Case 3: Drives only external ports, no internal
    loads.
93              puts "    INFO: OBUF drives port only (no internal
```

```
    loads) - OK, skipping"
94          }
95      } msg]} {
96          puts "  WARNING: Failed to process OBUF $obuf: $msg"
97      }
98  }
99
100 puts ""
101 puts "pre_place.tcl: Fixed $obuf_fixed OBUF(s), removed
    $obuf_removed OBUF(s)"
102
103 # --- Safety net: waive/downgrade DRC in case any remain ---
104 catch { create_waiver -type DRC -id {REQP-127} -description "
    OBUF loads handled by pre_place.tcl" }
105 catch { set_property SEVERITY {Warning} [get_drc_checks REQP-127
    ] }
106
107 puts "
    ================================================================
    "
108 puts "pre_place.tcl: Done"
109 puts "
    ================================================================
    "
```